Database Design for Analysts

From business requirements to relational structures

> *© Patrick Dolinger, NSCC Institute of Technology*
>
> *Licensed under Creative Commons Attribution 4.0 International (CC BY 4.0)*
>
> *You are free to share and adapt this material for any purpose, provided appropriate credit is given.*

Chapter 1: The Relational Model Revisited — From Querying to Designing

> ***Database Design for Analysts*** *From business requirements to relational structures*
>
> *© Patrick Dolinger, NSCC Institute of Technology Licensed under Creative Commons Attribution 4.0 International (CC BY 4.0) You are free to share and adapt this material for any purpose, provided appropriate credit is given.*

Chapter Overview

In DBAS 5010 you learned to read databases. You queried `dim.Customer`, joined it to `fact.Sales`, filtered by territory, and aggregated by category. You became fluent in the language of databases as a *user*. This course starts the same databases from a different angle: not as things to query, but as things to design.

That shift — from user to designer — is the conceptual leap this chapter bridges. A user asks "what does this table contain?" A designer asks "why is this table structured the way it is, and what rules governed that structure?" The answers to those questions are the content of this course.

This chapter revisits the foundations of the relational model — not to repeat what DBAS 5010 covered, but to deepen your understanding of *why* the relational model works the way it does. By the time you finish, you will see the CabotTrail databases not just as collections of tables, but as formal representations of a business's reality.

By the end of this chapter you will be able to:

- Describe the three perspectives on a database: external, conceptual, and internal

- Define the terms entity, attribute, and relationship in the context of database design
- Distinguish between a logical data model and a physical data model
- Explain what a relation is and how it differs from a simple table
- Read an existing database schema and identify its design decisions
- Explain the purpose of primary and foreign keys at a deeper level than their mechanical function
- Describe the CabotTrail OLTP schema and explain the design choices it reflects

---

Table of Contents

1. The Designer's Perspective
2. The Three-Schema Architecture
3. Entities, Attributes, and Relationships
4. Relations, Not Just Tables
5. Logical vs Physical Models
6. Keys: Identity and Connection
7. Reading the CabotTrail OLTP Schema
8. Design Decisions in the Wild
9. The Cost of Bad Design
10. Chapter Summary
11. Review Questions
12. 🔍 Deeper Dive

---

1. The Designer's Perspective

When you queried `CabotTrailOutdoorsSales` in DBAS 5010, you worked with a finished product. The tables were there. The columns had names. The foreign keys connected the dimensions to the fact table. You wrote SQL against a structure someone else had already decided on.

Database design is the process of making those decisions. Before any table existed, a designer answered hundreds of questions:

- What information does the business actually need to store?
- What are the natural groupings of that information?

- How do different pieces of information relate to each other?
- What rules must data satisfy to be valid?
- What queries will analysts need to run, and how do we structure data to support them efficiently?

Some of those decisions are visible in the final schema — you can see that `fact.Sales` has a `CustomerKey` column that links to `dim.Customer.CustomerKey`. Some are invisible — the absence of a column is also a design decision. Why does `fact.Sales` not have a `CustomerName` column? Because that information is in `dim.Customer` and referencing it there via a key is more efficient, more consistent, and more maintainable.

Understanding why things are the way they are is what separates a database analyst from a database designer.

1.1 Design as Translation

Database design is fundamentally a translation activity. The input is a description of a business — usually given in natural language by people who know the business but not databases. The output is a formal specification in the language of the relational model.

This translation is not mechanical. Natural language is ambiguous. People describe things differently. Business rules are sometimes implicit, sometimes contradictory, sometimes simply unknown. A designer must ask questions, probe ambiguities, and make defensible decisions about things that are not clearly specified.

Consider this business description:

> *"CabotTrail sells outdoor gear products to wholesale and retail customers across Canada. Each customer can place multiple orders, and each order can include multiple products. We need to track what was ordered, at what price, and when."*

This description sounds complete. It is not. Questions a designer would ask:

- Can one order span multiple invoices, or is one order always one invoice?
- Can a product appear on an order more than once (two separate lines for the same product)?
- Is "price" the price at the time of the order, or the current catalogue price?
- What does "when" mean — the order date, the invoice date, or the delivery date?
- What happens to historical orders if a product is discontinued?
- What happens to historical invoices if a customer is deleted?

None of these are answerable from the description alone. Every one of them affects the database design. Chapter 2 covers techniques for surfacing these questions through entity-relationship diagramming.

1.2 The Designer Is Not the Last Word

An important professional truth: the database designer does not decide what a business does. The designer represents the business's reality in a formal structure. If the business says "one order can have one customer," the designer models that rule — even if the designer privately suspects it will change. If it does change, the design changes.

This is why database design requires close collaboration with business stakeholders. The designer brings technical expertise. The stakeholder brings business knowledge. Neither can produce a good design alone.

---

2. The Three-Schema Architecture

Database systems separate concerns into three levels, a framework called the **ANSI/SPARC three-schema architecture**. Understanding these levels explains why databases are designed the way they are.

2.1 The External Schema (View Level)

The **external schema** is what individual users and applications see — their personalised view of the data. Different users may see different subsets of the database, with different column names, different levels of detail, and different aggregations.

In the CabotTrail environment, the external schemas are the data marts: `CabotTrailOutdoorsSales`, `CabotTrailOutdoorsReturns`, `CabotTrailOutdoorsPurchasing`. Each mart presents a curated view of the data warehouse optimised for a specific audience. A sales analyst sees `fact.Sales` with customer and product dimensions. A purchasing manager sees `fact.Purchasing` with supplier and product dimensions.

These are not the "real" database. They are views onto it.

2.2 The Conceptual Schema (Logical Level)

The **conceptual schema** is the unified, complete description of the database — all entities, all relationships, all constraints — independent of how the data is physically stored and independent of which user sees what.

In the CabotTrail environment, the conceptual schema is the data warehouse (`CabotTrailOutdoorDW`) and, at a more fundamental level, the OLTP (`CabotTrailOutdoor`). These represent the full, integrated picture of the business's data.

The conceptual schema is what entity-relationship diagrams (Chapter 2) capture. It is the logical model — the definition of *what* the data is, not *how* it is stored.

2.3 The Internal Schema (Physical Level)

The **internal schema** is how the data is physically stored on disk — file organisation, index structures, page sizes, storage engine details. This level is largely invisible to SQL developers and analysts; it is the concern of database administrators and storage engineers.

In SQL Server, the internal schema includes the clustered index structure of each table, the non-clustered index definitions, the filegroup allocation, and the page compression settings. Chapter 7 touches the edge of the internal schema when discussing index design.

2.4 Why the Separation Matters

The three-schema architecture enforces **data independence** — the ability to change one level without breaking the others. You can add an index (internal schema change) without changing any queries (external schema). You can add a column to a table (conceptual schema change) without changing the physical storage format (internal schema). You can create a new data mart view (external schema change) without changing the underlying tables (conceptual schema).

This independence is one of the most practically important properties of relational databases. It is what makes it possible to optimise a query by adding an index without breaking the application that uses the database.

---

3. Entities, Attributes, and Relationships

The vocabulary of database design has precise meaning. These three terms — entity, attribute, relationship — are the building blocks of every data model.

3.1 Entities

An **entity** is a distinguishable thing in the real world about which the business needs to store information. Entities become tables.

An entity is not a specific instance — it is a category. "CabotTrail Outdoor" is not an entity; "Customer" is. "Cape Breton 3-Season Sleeping Bag" is not an entity; "Product" is.

Identifying entities is the first and most important step in database design. Good questions to ask:

- What are the main nouns in the business description?
- What things does the business track over time?
- What things have multiple attributes associated with them?
- What things would appear on a business document (invoice, purchase order, employee record)?

For CabotTrail, the core entities include: **Customer**, **Order**, **Product**, **Invoice**, **Supplier**, **Employee**, **City**, **Province**, **Country**.

3.2 Attributes

An **attribute** is a property of an entity — a piece of information that describes the entity. Attributes become columns.

The **Customer** entity has attributes: CustomerName, CreditLimit, AccountOpenedDate, DeliveryCity. Each attribute describes a fact about the customer.

Not everything that could be an attribute should be. A key design question: does this piece of information belong to this entity, or does it belong to a related entity? Customer's city is an attribute of Customer — but the city's province is not an attribute of Customer. It is an attribute of City. If you store `ProvinceCode` directly on the `Customers` table, you have duplicated information that belongs to `Cities`. When a city's province changes (unusual but possible), you must update every customer in that city rather than the one city record.

This reasoning is the foundation of normalization, covered in Chapters 4 and 5.

3.3 Relationships

A **relationship** describes how entities are associated with each other. Relationships become foreign keys (sometimes also tables, when the relationship has its own attributes).

Relationships have a **cardinality** — the numerical constraint on how many instances of one entity can be associated with instances of another:

Cardinality	Symbol	Meaning	CabotTrail example
One-to-one	1:1	One A is associated with exactly one B	Employee ↔ EmployeeContactDetails
One-to-many	1:N	One A is associated with many Bs	Customer → Orders (one customer, many orders)

Cardinality	Symbol	Meaning	CabotTrail example
Many-to-many	M:N	Many As are associated with many Bs	Products ↔ ProductCategories (one product, many categories; one category, many products)

Cardinality is a **business rule**, not a technical constraint. "One customer can place many orders" is a statement about how CabotTrail operates — it is not a physical limitation of the database. The database *encodes* that business rule by putting `CustomerID` as a foreign key on the `Orders` table (enforcing the one-to-many). If the business decided "each order must be placed by exactly one customer," that too is a business rule encoded in the design.

---

4. Relations, Not Just Tables

The word "relational" in relational database does not mean "things are related to each other." It refers to the mathematical concept of a **relation** — which is what most people would call a table with some additional formal properties.

4.1 The Formal Definition

A **relation** is a two-dimensional structure with these properties:

1. **Each column has a unique name** — no two columns in the same table may have the same name
2. **All values in a column are of the same type** — a column cannot mix integers and text
3. **Each row is unique** — no two rows in the same table are identical in all columns
4. **The order of rows is irrelevant** — a relation has no inherent row order; ORDER BY is something you apply to a query, not a property of the data
5. **The order of columns is irrelevant** — a relation is defined by its column names, not their sequence
6. **Each cell contains exactly one value** — a column cannot store a list of values in a single cell

These properties are formal. Understanding them explains several SQL behaviours:

- **Why ORDER BY is always optional:** Because a relation has no inherent order — the default return order of rows is genuinely undefined.
- **Why SELECT * is imprecise:** Because column order is irrelevant to the relation; a query relying on column position is fragile.
- **Why a table without a primary key violates relational principles:** Because if rows are not unique, the relation property (each row is distinguishable) is violated.

4.2 What Makes a Table Not a Relation

A SQL table is a physical object that approximates a relation. It can violate relational properties in ways that cause problems:

Violation: duplicate rows. A table without a primary key can have two identical rows. Queries against such a table produce unpredictable aggregate results and JOIN behaviours.

Violation: multi-valued attributes. A column that stores comma-separated lists (`'NS, NB, PE'`) violates property 6 — each cell does not contain exactly one value. This makes it impossible to filter by a single value using `=`.

Violation: inconsistent types. A `VARCHAR` column used to store phone numbers, dates, and codes indiscriminately violates property 2.

Normalization (Chapters 4 and 5) is the process of eliminating these violations from a database design.

4.3 The Tuple and Domain

The formal vocabulary of the relational model uses the term **tuple** for what SQL calls a row, and **domain** for the set of valid values for a column.

A domain is not just a data type — it is a semantic constraint. The domain for `MonthNumber` is the integers 1 through 12. The domain for `ProvinceCode` is a specific set of two-letter codes. A `TINYINT` column that stores month numbers has the right data type, but only a `CHECK (MonthNumber BETWEEN 1 AND 12)` constraint enforces the domain.

This distinction — between the physical type and the logical domain — explains why data types alone are insufficient for data integrity. Constraints (Chapter 6) are what enforce domains.

---

5. Logical vs Physical Models

Database design happens in two stages: logical design and physical design. The distinction is fundamental.

5.1 The Logical Model

A **logical data model** describes *what* data the system needs to store and how it is logically organised — independent of any specific database technology. A logical model answers:

- What entities exist?

- What attributes do they have?
- How are the entities related?
- What constraints apply?

A logical model is expressed as an entity-relationship diagram (Chapter 2). It should be understandable by business stakeholders without any database knowledge. It does not mention SQL Server, indexes, or storage.

5.2 The Physical Model

A **physical data model** describes *how* the logical model is implemented in a specific database system. A physical model answers:

- What SQL data type is each attribute stored as?
- What is the exact DDL for each table?
- Which columns are primary keys? Which are foreign keys?
- What indexes exist for performance?
- What constraints enforce which business rules?

A physical model is expressed as DDL — `CREATE TABLE` statements with data types, constraints, and index definitions. It is technology-specific: the physical model for SQL Server differs from the physical model for PostgreSQL or Oracle in syntax, data type names, and constraint options.

5.3 The Flow from Logical to Physical

The design process flows:

```
Business requirements
        ↓
[Analysis: identify entities, attributes, relationships]
        ↓
Logical model (ERD)
        ↓
[Normalization: eliminate redundancy]
        ↓
Normalized logical model
        ↓
[Physical design: choose data types, constraints, indexes]
        ↓
Physical model (DDL)
        ↓
[Implementation: CREATE TABLE statements]
        ↓
Populated database
```

This course follows this flow. Chapters 2–5 cover the logical design steps. Chapters 6–7 cover physical design. Chapter 8 applies both to the OLAP context. Chapter 9 executes the complete flow for a new database.

---

6. Keys: Identity and Connection

In DBAS 5010 you used primary and foreign keys mechanically — join on this key, look up that surrogate key. This section gives those concepts the depth they deserve.

6.1 Candidate Keys

A **candidate key** is any column or combination of columns that could serve as a unique identifier for rows in the table — no two rows have the same value, and no part of the key is redundant.

A table can have multiple candidate keys. For a `Customers` table, both `CustomerID` (a system-assigned number) and `CustomerEmail` (if unique) could be candidate keys.

6.2 The Primary Key

The **primary key** is the candidate key that the designer chooses as the official row identifier. The choice of primary key has implications:

- SQL Server creates a clustered index on the primary key by default, ordering the physical storage of rows by PK value
- All foreign keys in other tables that reference this table reference the primary key
- The primary key constraint enforces uniqueness and NOT NULL

Natural keys are candidate keys derived from real-world attributes: email address, tax ID, ISBN. They have meaning outside the database. The risk: real-world attributes sometimes change. A customer changes their email address; the natural key must be updated everywhere it is referenced.

Surrogate keys are system-generated identifiers with no meaning outside the database: `IDENTITY(1,1)` integer columns like `CustomerID`. They never change. They have no business meaning, but they are stable, compact, and index-efficient. CabotTrail uses surrogate keys throughout both the OLTP and the DW.

The surrogate vs natural key debate is one of the most persistent in database design. The CabotTrail databases use surrogate keys consistently — a deliberate design decision. Chapter 3 covers when each approach is appropriate.

6.3 Foreign Keys

A **foreign key** is a column in one table whose value must match the primary key of another table — or be NULL. It enforces the relationship between entities.

```
Sales.Orders.CustomerID  →  Sales.Customers.CustomerID
```

This foreign key enforces: "every order must reference a customer that exists." Attempting to insert an order with a `CustomerID` that does not exist in `Customers` fails.

Foreign keys also define **referential integrity actions** — what happens when the referenced PK is updated or deleted:

Action	Meaning
`NO ACTION` (default)	Block the update/delete if referencing rows exist
`CASCADE`	Propagate the update/delete to referencing rows
`SET NULL`	Set the FK column to NULL when the referenced row is deleted
`SET DEFAULT`	Set the FK column to its default value

The choice of referential action is a business rule. "What happens to orders when a customer is deleted?" is a question only the business can answer. The designer encodes the answer as a referential action.

6.4 Composite Keys

A **composite key** is a primary key made of two or more columns. Their combined values are unique; neither alone is sufficient.

```
-- Junction table with a composite primary key
CREATE TABLE Inventory.ProductCategoryAssignments
(
    ProductID           INT NOT NULL,
    ProductCategoryID   INT NOT NULL,
    CONSTRAINT PK_ProductCategoryAssignments
        PRIMARY KEY (ProductID, ProductCategoryID)
);
```

Neither `ProductID` alone nor `ProductCategoryID` alone is unique — a product can appear in multiple categories, and a category contains multiple products. Only the combination is unique. This is the standard pattern for implementing many-to-many relationships.

---

7. Reading the CabotTrail OLTP Schema

You have queried this database. Now read it as a designer.

7.1 Discovering the Schema

```
-- List all tables by schema in the OLTP
USE CabotTrailOutdoor;

SELECT
    s.name              AS SchemaName,
    t.name              AS TableName,
    p.rows              AS RowCount
FROM    sys.tables t
INNER JOIN sys.schemas s    ON s.schema_id = t.schema_id
INNER JOIN sys.partitions p ON p.object_id = t.object_id
                           AND p.index_id IN (0, 1)
ORDER BY s.name, t.name;
```

Run this. You will see tables organised into four schemas: `Sales`, `Purchasing`, `Inventory`, and `Application`. This organisation is itself a design decision — schemas group related tables, reflecting business domains.

7.2 Reading the Relationships

```
-- All foreign key relationships in the OLTP
SELECT
    fk.name                     AS RelationshipName,
    SCHEMA_NAME(tp.schema_id) + '.' + tp.name  AS ParentTable,
    cp.name                     AS ParentColumn,
    SCHEMA_NAME(tc.schema_id) + '.' + tc.name  AS ReferencedTable,
    cc.name                     AS ReferencedColumn
FROM    sys.foreign_keys fk
INNER JOIN sys.foreign_key_columns fkc ON fkc.constraint_object_id = fk.object_id
INNER JOIN sys.tables tp    ON tp.object_id = fkc.parent_object_id
INNER JOIN sys.tables tc    ON tc.object_id = fkc.referenced_object_id
INNER JOIN sys.columns cp   ON cp.object_id = fkc.parent_object_id
                           AND cp.column_id  = fkc.parent_column_id
INNER JOIN sys.columns cc   ON cc.object_id = fkc.referenced_object_id
                           AND cc.column_id  = fkc.referenced_column_id
ORDER BY SCHEMA_NAME(tp.schema_id), tp.name;
```

Study the output. For each relationship, ask:

- Which table is the "one" side and which is the "many" side?
- What business rule does this relationship enforce?
- What would happen if this FK constraint did not exist?

7.3 Reading the Constraints

```
-- All CHECK constraints in the OLTP
SELECT
    SCHEMA_NAME(t.schema_id) + '.' + t.name    AS TableName,
    cc.name                                     AS ConstraintName,
    cc.definition                               AS CheckCondition
FROM    sys.check_constraints cc
INNER JOIN sys.tables t ON t.object_id = cc.parent_object_id
ORDER BY TableName, ConstraintName;
```

Each check constraint is a business rule encoded in DDL. Reading them tells you something about what the database designer knew about the business — what values are valid, what ranges are permitted.

7.4 The CabotTrail OLTP Entity Map

Reading the schema systematically, the OLTP has these core entity groups:

Sales domain:

```
Customers ─────── Orders ─────── OrderLines
    │                │
Invoices ──────────────┘
    │
InvoiceLines ─────── Returns
```

Purchasing domain:

```
Suppliers ─────── PurchaseOrders ─────── PurchaseOrderLines
```

Inventory domain:

```
ProductCategories ─── ProductCategoryAssignments ─── Products ─── ProductHoldings
```

Application domain (shared lookups and geography):

```
Countries ─────── StateProvinces ─────── Cities ─────── (Sales.Customers.DeliveryCityID)

Employees ── People ── DeliveryMethods ── PackageTypes ── Colors ── PaymentMethods
```

The geography tables live in the `Application` schema (`Application.Countries`, `Application.StateProvinces`, `Application.Cities`); `Sales.Customers` references them through `DeliveryCityID`. The domain grouping is a reading aid — the schema name is what you use in queries.

Notice that `People` is a shared entity — both `Customers` and `Employees` reference it (or share its attributes). This is a **supertype/subtype** relationship — a design pattern covered in Chapter 2.

---

8. Design Decisions in the Wild

Every structural choice in the CabotTrail OLTP reflects a design decision. Identifying these decisions and understanding their implications is the core skill this course develops.

8.1 The Geography Hierarchy

Customer delivery addresses are stored across four tables:

```
Countries → StateProvinces → Cities → Customers.DeliveryCityID
```

A simpler design would store `City`, `Province`, and `Country` directly on the `Customers` table. Why was this not done?

Reason: Multiple customers share the same city. If `CityName` and `ProvinceCode` were columns on `Customers`, every customer in Halifax would store 'Halifax' and 'NS' redundantly. If Nova Scotia's province code ever changed, every customer record would need updating. Storing the geography in separate tables and linking by ID means:

- City data is stored once
- Changes propagate automatically
- Queries can aggregate by city or province without scanning customer records

This is the normalization principle in action — covered formally in Chapters 4 and 5.

8.2 The Junction Table for Categories

Products can belong to multiple categories. Categories contain multiple products. This many-to-many relationship is resolved with a junction table:

```
Products ——— ProductCategoryAssignments ——— ProductCategories
```

Why not store `CategoryID` directly on the `Products` table? Because a product can be in more than one category. A single column can hold only one value. To allow multiple categories per product requires a separate table — one row per (product, category) pair.

8.3 The Separation of Orders and Invoices

Orders and Invoices are separate tables even though they often have a one-to-one relationship. Why?

Reason: The business model allows for partial fulfilment — an order might be partially shipped and partially backordered, potentially resulting in multiple invoices per order. Even if this does not happen frequently, the design accommodates it without structural change. This is the principle of designing for the business's possible future, not just its current practice.

This is also why there are `OrderLines` *and* `InvoiceLines` — the quantity picked and invoiced may differ from the quantity ordered.

---

9. The Cost of Bad Design

Understanding why good design matters is motivating. Here are the specific costs that bad design inflicts.

9.1 Update Anomalies

When the same fact is stored in multiple places, updating it requires updating all copies — and missing one creates an inconsistency.

```
-- Bad design: storing city name directly on customer records
Customers table:
CustomerID | CustomerName          | CityName  | Province
42         | Fundy Bay Outfitters  | Halifax   | NS
17         | Atlantic Gear Ltd     | Halifax   | NS
91         | East Coast Supply     | Halifax   | NS

-- If Halifax is renamed to 'New Halifax' (hypothetical):
-- Must update 3 rows minimum, more if other customers are in Halifax
-- Missing any one row creates data inconsistency
```

With a proper geography hierarchy, the city name is stored once in `Cities`. Renaming it updates one row everywhere.

9.2 Insertion Anomalies

A bad design may make it impossible to record certain facts without simultaneously recording other facts you do not yet have.

```
-- Bad design: one table for customer + order information
-- Cannot record a new customer until they place their first order
-- (What order details do you fill in for a customer with no orders yet?)
```

```
CustomerOrderTable:
CustomerID | CustomerName | OrderID | OrderDate | ProductID | ...
```

With proper entity separation, you can create a Customer record before any orders exist.

9.3 Deletion Anomalies

A bad design may cause the loss of one piece of information when another is deleted.

```
-- Same bad design: if the last order for a customer is deleted,
-- all information about that customer disappears too
```

9.4 These Anomalies Have Names for a Reason

Update, insertion, and deletion anomalies are the formal names for the symptoms of under-normalization. Normalization (Chapters 4 and 5) is the process of eliminating these anomalies by structuring data correctly. Every normalization rule corresponds to eliminating one of these anomaly types.

---

10. Chapter Summary

- Database design is a **translation discipline** — converting business requirements into formal relational structures. It requires collaboration between technical designers and business stakeholders.
- The **three-schema architecture** separates the database into external (user views), conceptual (logical structure), and internal (physical storage) levels — enabling changes at one level without breaking others.
- The building blocks of data models are **entities** (things the business tracks), **attributes** (properties of entities), and **relationships** (associations between entities with cardinality).
- A **relation** is a formally defined structure with properties beyond a simple table: unique column names, consistent column types, unique rows, and single-valued cells. Violations of these properties cause anomalies.
- **Logical models** describe *what* data exists; **physical models** describe *how* it is stored. The flow is: requirements → ERD (logical) → normalization → DDL (physical).
- **Primary keys** identify rows; **foreign keys** enforce relationships. **Natural keys** have business meaning; **surrogate keys** are system-generated. **Composite keys** implement many-to-many relationships.

- Reading the CabotTrail OLTP schema reveals specific design decisions: the geography hierarchy (normalization), the junction table for categories (M:N resolution), and the separation of orders and invoices (future-proofing).
- **Update, insertion, and deletion anomalies** are the costs of bad design. Normalization eliminates them.

---

11. Review Questions

1. Explain the difference between an entity and an attribute. For each of the following, state whether it is an entity or an attribute, and explain your reasoning: **ProductName**, **Supplier**, **OrderDate**, **PaymentMethod**, **Province**, **TaxRate**.
2. The three-schema architecture defines external, conceptual, and internal schemas. Using the CabotTrail environment as your example, identify a specific object that belongs to each level and explain why it belongs there.
3. A colleague proposes storing customer orders in a spreadsheet where each row represents one customer, and multiple orders are stored as separate columns: `Order1Date`, `Order2Date`, `Order3Date`. Identify which relational property this design violates, what happens when a customer places a fourth order, and how you would redesign this correctly.
4. Explain the difference between a natural key and a surrogate key. Give a specific example of a column from `CabotTrailOutdoor` that could serve as a natural key for the `Customers` entity. What is the risk of using it as the primary key instead of the system-assigned `CustomerID`?
5. The CabotTrail OLTP stores geography across four tables: `Countries`, `StateProvinces`, `Cities`, and `Customers.DeliveryCityID`. Write out the specific update anomaly that would occur if city names were stored directly on the `Customers` table instead, using Halifax as a specific example.
6. Run the foreign key query from section 7.2 against `CabotTrailOutdoor`. Pick three foreign key relationships from the results and for each one: (a) state the business rule it enforces and (b) explain what would happen operationally if the constraint did not exist.
7. Describe the difference between a logical data model and a physical data model. For the CabotTrail `Customers` entity, give one example of a design decision that belongs in the logical model and one that belongs only in the physical model.

8. `Sales.InvoiceLines` has a composite primary key. Without looking it up, reason from the business context: which two columns form that composite key, and why is neither column alone sufficient to uniquely identify a row?

🔍 Deeper Dive

Going Further with the Relational Model

E.F. Codd's Twelve Rules

In 1985, E.F. Codd (who had proposed the relational model in 1970) published what became known as **Codd's Twelve Rules** — a set of criteria that a database management system must satisfy to be considered truly relational. They are:

1. **The information rule:** All information must be represented as data values in tables.
2. **The guaranteed access rule:** Every datum must be accessible via table name, primary key, and column name.
3. **Systematic treatment of NULL:** NULL must represent missing or inapplicable information, distinct from any other value.
4. **Active online catalogue:** The database schema must be stored in tables that can be queried using the same query language as user data.
5. **Comprehensive data sublanguage:** At least one language must support data definition, data manipulation, transaction management, and integrity constraints.
6. **View updating rule:** All views that are theoretically updatable must be updatable by the system.
7. **High-level insert, update, delete:** These operations must be applicable to sets of rows, not just individual rows.
8. **Physical data independence:** Changes to physical storage must not require changes to applications.
9. **Logical data independence:** Changes to the logical schema must not require changes to applications (beyond the changed objects).
10. **Integrity independence:** Integrity constraints must be definable in the data sublanguage and stored in the catalogue, not in application code.
11. **Distribution independence:** A distributed database must behave identically to a centralised one from the application's perspective.
12. **Non-subversion rule:** If a low-level access method exists, it must not be able to bypass the integrity constraints.

No commercial DBMS fully satisfies all twelve rules. SQL Server satisfies most of them. Understanding the rules helps explain why some SQL behaviours seem odd — they are the result of partial compliance with a strict theoretical framework.

Codd, E. F. (1985, October 14). Is your DBMS really relational? *Computerworld*.

The Relational Model vs the Object-Relational Model

The relational model treats data as two-dimensional tables of atomic values. Modern databases extend this with the **object-relational model** — allowing complex types, arrays, JSON, and hierarchical structures within columns.

SQL Server's support for XML columns, JSON stored in NVARCHAR, spatial data types, and user-defined CLR types are all object-relational extensions. They are useful when the strict relational model is impractical — storing a variable-length list of tags on a product, for example.

The trade-off: object-relational extensions are powerful but sacrifice some of the formal guarantees that make the relational model trustworthy. A JSON column containing a list is not queryable with standard SQL operators. Values inside it cannot have FK constraints enforced against them.

The principle: use the relational model by default. Reach for extensions only when the relational approach is genuinely impractical for a specific use case.

The Entity-Relationship Model vs the Relational Model

A subtle distinction that confuses many students: the **Entity-Relationship (ER) model** and the **Relational model** are two different things, though they work together.

The ER model, proposed by Peter Chen in 1976, is a conceptual modelling framework — a way of describing the real world in terms of entities and relationships. It produces entity-relationship diagrams.

The relational model, proposed by Codd in 1970, is a mathematical framework for data storage and query. It produces tables and SQL.

ER modelling is a technique for *designing* a relational database. The ER diagram is the input; the relational schema (tables, keys, constraints) is the output. Chapter 2 covers ER modelling. Chapter 3 covers the mapping from ER model to relational tables.

Chen, P. P. (1976). The entity-relationship model — Toward a unified view of data. *ACM Transactions on Database Systems*, 1(1), 9–36.

References and Further Reading

1. Date, C. J. (2019). *Database Design and Relational Theory: Normal Forms and All That Jazz* (2nd ed.). Apress. — The most rigorous treatment of the relational model's theoretical foundations.

2. Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387. — The foundational paper.

3. Chen, P. P. (1976). The entity-relationship model — Toward a unified view of data. *ACM Transactions on Database Systems*, 1(1), 9–36. — The paper that introduced ERDs.

4. Connolly, T., & Begg, C. (2015). *Database Systems: A Practical Approach to Design, Implementation, and Management* (6th ed.). Pearson. — Comprehensive textbook covering all topics in this course in depth.

5. Elmasri, R., & Navathe, S. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson. — Academic standard for database design courses.

6. Microsoft. (2024). *SQL Server data types*. https://learn.microsoft.com/en-us/sql/t-sql/data-types/data-types-transact-sql

7. Microsoft. (2024). *sys.foreign_keys (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/relational-databases/system-catalog-views/sys-foreign-keys-transact-sql

---

> ***Database Design for Analysts*** *| © Patrick Dolinger, NSCC Institute of Technology CC BY 4.0 — Share and adapt freely with attribution*

Chapter 2: Entity-Relationship Diagrams — Drawing the Business

Database Design for Analysts *From business requirements to relational structures*

© Patrick Dolinger, NSCC Institute of Technology Licensed under Creative Commons Attribution 4.0 International (CC BY 4.0) You are free to share and adapt this material for any purpose, provided appropriate credit is given.

Chapter Overview

Chapter 1 introduced the vocabulary of database design — entities, attributes, relationships, cardinality. This chapter puts that vocabulary to work. An **Entity-Relationship Diagram (ERD)** is the primary tool database designers use to capture and communicate the logical structure of a database before a single table is created.

An ERD is not a table diagram. It is not a list of SQL statements. It is a visual representation of the business's information model — a picture that shows what exists, what properties those things have, and how they are connected. A well-drawn ERD is readable by both a database developer and a business manager who has never written SQL.

This chapter covers the notation, the technique, and the thinking behind ERDs. The notation is straightforward to learn. The thinking — identifying the right entities, drawing the right relationships, getting cardinality correct — takes practice and is the core intellectual work of database design.

By the end of this chapter you will be able to:

- Identify entities, attributes, and relationships from a business narrative
- Distinguish between strong entities and weak entities
- Draw an ERD using Crow's Foot notation
- Correctly identify and represent cardinality for one-to-one, one-to-many, and many-to-many relationships
- Represent mandatory and optional participation
- Model derived attributes, multi-valued attributes, and composite attributes
- Apply supertypes and subtypes to model entity hierarchies
- Read and critique an ERD from an existing system

Table of Contents

1. What an ERD Is and Is Not
2. ERD Notation: The Crow's Foot Standard
3. Drawing Entities and Attributes
4. Special Attribute Types
5. Relationships and Cardinality
6. Participation: Mandatory and Optional
7. Strong and Weak Entities
8. Many-to-Many Relationships
9. Supertypes and Subtypes
10. From Narrative to ERD: A Worked Example
11. Reading the CabotTrail ERD
12. Chapter Summary
13. Review Questions
14. 🔍 Deeper Dive

1. What an ERD Is and Is Not

1.1 What an ERD Is

An ERD is a **conceptual model** — a formal representation of the logical structure of a business's information. It answers three questions:

1. **What exists?** The entities — the things the business tracks
2. **What do those things look like?** The attributes — the properties of each entity
3. **How are those things connected?** The relationships — the associations between entities

An ERD is drawn *before* any SQL is written. It is the specification that SQL implementations are derived from. If you change the ERD, the SQL changes. If you write SQL without an ERD, you have skipped the design step — and you will spend time discovering structural problems during implementation rather than on paper, where changes are free.

1.2 What an ERD Is Not

An ERD is **not** a table diagram. Tables have SQL data types, IDENTITY columns, DEFAULT constraints, and indexes. An ERD has none of these — they belong to the physical model, not the conceptual model.

An ERD is **not** a schema diagram (like the ones SSMS generates). Schema diagrams show physical tables with their actual columns. ERDs show logical entities with their semantic attributes.

An ERD is **not** a finished product. It is a working document — drawn in pencil, revised through conversation with stakeholders, and updated as understanding deepens. An ERD that has never been revised is either a very simple system or a design that was never questioned.

1.3 The ERD as a Communication Tool

The most important property of an ERD is that it is understandable to non-technical stakeholders. A business manager who has never seen SQL can look at a well-drawn ERD and say "yes, that's how our orders work" or "wait — that's not right, an order can have multiple delivery addresses."

This conversation — between the designer and the business — is what the ERD enables. The diagram externalises the designer's understanding of the business, making it verifiable by the people who know the business best.

---

2. ERD Notation: The Crow's Foot Standard

Several ERD notations exist. This book uses **Crow's Foot notation**, which is the most widely used in industry, supported by Microsoft Visio, Lucidchart, draw.io, and most database design tools.

2.1 The Basic Symbols

Entity: A rectangle containing the entity name. Names are singular nouns in uppercase or title case — `CUSTOMER`, `ORDER`, or `Customer`, `Order`.

Attribute: An oval connected to its entity with a line, or (in simplified notation) listed inside the entity rectangle. The primary key attribute is underlined.

Relationship: A line connecting two entities, with Crow's Foot symbols at each end indicating cardinality.

2.2 Crow's Foot Cardinality Symbols

The "crow's foot" is the symbol that looks like a three-pronged fork at the end of a relationship line. The symbols at each end of the line represent the minimum and maximum number of instances that can participate:

```
Minimum symbols (closest to entity):
  —|—   Exactly one (mandatory)
  —O—   Zero (optional)

Maximum symbols (farther from entity):
  —|—   One
  —<—   Many (the "crow's foot")
```

These are combined to express four cardinality patterns:

```
  ——|——|——   Exactly one (mandatory one)
  ——O——|——   Zero or one (optional one)
  ——|——<——   One or many (mandatory many — at least one)
  ——O——<——   Zero or many (optional many — none, one, or more)
```

2.3 Reading a Relationship Line

A relationship line is read in both directions, with the entity at each end as the subject:

```
CUSTOMER ——|——<—— ORDER
```

Reading left to right: "One Customer can have many Orders" Reading right to left: "One Order must belong to exactly one Customer"

The symbols at the Customer end (`|`) mean: from Order's perspective, exactly one Customer. The symbols at the Order end (`|<`) mean: from Customer's perspective, one or many Orders.

Always read both directions. Both statements express the complete cardinality rule.

2.4 Notation in Practice

In simplified ERD notation (used in most tools), attributes are listed inside the entity rectangle rather than as separate ovals. Primary key attributes are marked `PK`, foreign key attributes are marked `FK`:

```
┌──────────────────────┐          ┌──────────────────────┐
│ CUSTOMER             │          │ ORDER                │
├──────────────────────┤          ├──────────────────────┤
│ PK CustomerID (INT) |——|——O<——| PK OrderID (INT)     |
|    CustomerName     |          | FK CustomerID (INT)  |
```

```
|    CreditLimit       |          |    OrderDate          |
|    AccountOpenedDate|          |    DeliveryAddress    |
└──────────────────────┘          └──────────────────────┘
```

The relationship line at the Order end shows `O<` (zero or many) — a customer may exist with no orders yet. The line at the Customer end shows `|` (exactly one) — an order must always reference a customer.

---

3. Drawing Entities and Attributes

3.1 Identifying Entities

From a business narrative, entities are typically:

- **Nouns** that the business tracks over time: Customer, Product, Order, Supplier
- **Things that appear on business documents**: an invoice references customers, products, quantities, prices
- **Things with multiple instances**: there are many customers, many products, many orders
- **Things the business makes decisions about**: which supplier to use for which product

Not entities:

- Single-value facts (the company's tax number, the current tax rate) — these are better stored as configuration, not as entities
- Actions or processes (placing an order) — the *result* of the action is the entity (Order), not the action itself
- Relationships between entities — the relationship is captured as a line, not a box

The entity identification test: Can you have multiple distinct instances of this thing? Can two instances have the same attribute values? If yes to both, it is an entity.

3.2 Naming Entities

Entity names should be:

- **Singular** not plural: `Customer` not `Customers`, `Order` not `Orders`
- **Specific** to the domain: `PurchaseOrder` not `Order` when the system has both sales orders and purchase orders
- **Business-language** not technical: `Customer` not `UserAccount`, `Product` not `Item`

3.3 Identifying Attributes

For each entity, attributes are the facts the business needs to store about instances of that entity. Good questions to surface attributes:

- What information does the business record when a new [entity] is created?
- What does the business need to know about [entity] to do its work?
- What columns would appear on a report about [entity]?

For `Customer`, the CabotTrail attributes are: CustomerID, CustomerName, CustomerGroupName (category), CreditLimit, AccountOpenedDate.

For `Order`, the attributes are: OrderID, OrderDate, ExpectedDeliveryDate, IsUndersupplyBackordered — and a reference to CustomerID (the FK that links to Customer).

3.4 Choosing the Primary Key Attribute

Every entity must have a primary key — the attribute(s) that uniquely identify each instance. Mark the PK attribute with an underline (in traditional notation) or `PK` label (in simplified notation).

If no natural attribute is a reliable unique identifier, introduce a surrogate key (`CustomerID INT IDENTITY`). The discussion from Chapter 1 applies: surrogate keys are stable and index-efficient; natural keys carry business meaning but may change.

---

4. Special Attribute Types

Some attributes have properties that cannot be represented as simple single-valued columns. Recognising these in the design phase prevents structural problems later.

4.1 Multi-Valued Attributes

A **multi-valued attribute** is an attribute that can hold more than one value per entity instance. In the relational model, multi-valued attributes cannot be stored as a single column — they require a separate table.

Example: A `Product` can belong to multiple `ProductCategories`. `CategoryName` is a multi-valued attribute of `Product`.

Solution: Create a separate `ProductCategoryAssignment` entity (a junction table) with a row for each (Product, Category) pair. This is the standard resolution for multi-valued attributes and for many-to-many relationships (section 8).

In ERD notation, multi-valued attributes are shown with a **double oval**. They are always resolved to a separate table in the physical model.

4.2 Derived Attributes

A **derived attribute** is one that can be computed from other stored attributes. It does not need to be stored, because it can always be recalculated.

Example: `CustomerAge` can be derived from `DateOfBirth` and `GETDATE()`. `OrderTotal` can be derived from the sum of `InvoiceLines.LineTotal`.

In ERD notation, derived attributes are shown with a **dashed oval**. In the physical model, you have a design decision:

- **Compute at query time** — cheaper to store, always accurate, requires computation each query
- **Store as a computed column** (`AS expression PERSISTED`) — stored on disk, automatically maintained by SQL Server
- **Store as a regular column populated by ETL** — fast to query, but requires synchronisation logic

The CabotTrail DW stores `DaysUntilInvoice` and `GrossProfitMarginPct` as populated columns — the ETL computes and stores them. This is documented in the S2T mapping as derived attributes.

4.3 Composite Attributes

A **composite attribute** is one that is made up of multiple components that each have meaning independently.

Example: `FullName` is a composite attribute — it consists of `FirstName` and `LastName`. If the business only ever needs the full name as a unit, `FullName` is appropriate. If the business needs to sort by last name, search by first name, or generate personalised greetings, `FirstName` and `LastName` must be stored separately.

In ERD notation, composite attributes are shown with sub-ovals branching from the main oval. In the physical model, they become multiple columns.

The CabotTrail OLTP stores `Application.People.FullName` and `Application.People.PreferredName` as flat attributes — a decision that implies the business does not need to sort or filter by first name and last name independently. If that assumption is wrong, the schema would need to change.

---

5. Relationships and Cardinality

5.1 Naming Relationships

Relationships should be named with a verb phrase that describes the association. Read the name in both directions:

```
CUSTOMER ──places──> ORDER
ORDER ──is placed by──> CUSTOMER
```

Good relationship names make the ERD readable as prose: "A Customer *places* zero or many Orders. An Order *is placed by* exactly one Customer."

5.2 One-to-One (1:1)

A 1:1 relationship means each instance of entity A is associated with at most one instance of entity B, and vice versa.

```
EMPLOYEE ──|──|── EMPLOYEE_CONTRACT
```

One employee has exactly one contract; one contract belongs to exactly one employee.

1:1 relationships are rare because if two entities are always in a 1:1 relationship, they are usually the same entity. When 1:1 relationships do appear, they often reflect:

- **Security partitioning**: sensitive attributes are split to a separate table with tighter access control
- **Optional extension**: attributes that only apply to some instances are moved to a separate table rather than using NULLs
- **Legacy integration**: two systems that should share an entity do not, and a join table bridges them

CabotTrail example: `Application.People` and `Application.Employees` have a near-1:1 relationship — each employee is also a person. The separation allows the `People` table to be shared across multiple entities (employees, contacts, perhaps customers in future).

5.3 One-to-Many (1:N)

The most common relationship type. One instance of A is associated with many instances of B; each instance of B is associated with exactly one instance of A.

```
CUSTOMER ──|──O<── ORDER
```

One Customer has zero or many Orders. Each Order belongs to exactly one Customer.

The FK goes on the "many" side — `Orders.CustomerID` references `Customers.CustomerID`. This is the invariant rule: **the foreign key always lives on the many side of a one-to-many relationship.**

5.4 Many-to-Many (M:N)

Many instances of A are associated with many instances of B.

```
PRODUCT ──O<──>O── PRODUCT_CATEGORY
```

One Product belongs to zero or many Categories. One Category contains zero or many Products.

Many-to-many relationships **cannot be directly implemented** in the relational model with just two tables and a foreign key. They require a **junction table** (also called an associative entity, bridge table, or link table). Section 8 covers this in depth.

5.5 Self-Referencing Relationships

An entity can have a relationship with itself — a **recursive** or **unary** relationship. The classic example is a hierarchy: an Employee has a Manager who is also an Employee.

```
EMPLOYEE ──|──O<── EMPLOYEE (manages/is managed by)
```

In the physical model, this becomes a self-referencing foreign key: `Employees.ManagerID` references `Employees.EmployeeID`.

The CabotTrail `Application.Employees` table includes a `ReportsTo` or similar column referencing the same table. Recursive relationships underpin org charts, bill-of-materials structures, and category trees.

---

6. Participation: Mandatory and Optional

Participation defines whether every instance of an entity *must* participate in a relationship, or whether participation is optional.

6.1 Mandatory Participation

Mandatory participation (also called **total participation**): every instance of the entity must be involved in at least one instance of the relationship.

In Crow's Foot notation: the `|` symbol (vertical bar) on the relationship line nearest the entity indicates mandatory participation (minimum cardinality = 1).

Example: Every Order must have a Customer. An order cannot exist without a customer. This is mandatory participation — the minimum on the Customer side of the Order–Customer relationship is 1 (not 0).

In the physical model, mandatory participation translates to a `NOT NULL` foreign key: `Orders.CustomerID NOT NULL`. If the FK can be NULL, the participation is optional.

6.2 Optional Participation

Optional participation (also called **partial participation**): some instances of the entity may exist without participating in any instance of the relationship.

In Crow's Foot notation: the `O` symbol (circle/zero) on the relationship line nearest the entity indicates optional participation (minimum cardinality = 0).

Example: A Customer may or may not have placed any Orders. Participation of Customer in the Customer–Order relationship is optional from the Customer's perspective. A customer can exist in the database before placing their first order.

In the physical model, optional participation can mean either a nullable FK (on the "many" side) or simply that the referenced entity row may have no referencing rows.

6.3 Reading Participation in Both Directions

Every relationship has participation from both entities' perspectives:

```
CUSTOMER ──|──O<── ORDER
```

From Order's perspective: Every Order *must* have exactly one Customer (`|` at Customer end — mandatory). From Customer's perspective: A Customer *may* have zero or many Orders (`O<` at Order end — optional many).

Both participation decisions are business rules:

- "An order cannot exist without a customer" → mandatory on Order's side
- "A customer can exist before placing any orders" → optional on Customer's side

These two rules together define the complete semantics of the relationship.

---

7. Strong and Weak Entities

7.1 Strong Entities

A **strong entity** has its own primary key — it can be uniquely identified independently of any other entity. Most entities in a well-designed system are strong entities.

`Customer` is a strong entity — `CustomerID` uniquely identifies it without reference to anything else.

7.2 Weak Entities

A **weak entity** cannot be uniquely identified by its own attributes alone. It depends on a related **owner entity** for part of its identity.

Example: `OrderLine`. An order line is identified by its `OrderLineID` — but that ID only makes sense within the context of a specific `Order`. If two different orders both have an `OrderLineID = 1`, they are different line items. The uniqueness of `OrderLine` depends on knowing which Order it belongs to.

In the physical model, weak entities have a composite primary key that includes the FK to the owner entity:

```
-- OrderLine's identity depends on its Order
CONSTRAINT PK_OrderLines PRIMARY KEY (OrderID, OrderLineID)
-- or with a surrogate key:
-- The OrderLineID alone is unique system-wide (if using IDENTITY)
-- but conceptually, its meaning depends on the order context
```

In ERD notation, weak entities are shown in a **double rectangle** and the identifying relationship (the relationship to the owner entity) is shown as a **double diamond**.

The CabotTrail `Sales.OrderLines` is a weak entity — its rows are identified in the context of a specific order. `Sales.InvoiceLines` is similarly weak — each line is identified within the context of an invoice.

7.3 When Surrogate Keys Change the Picture

Surrogate keys complicate the strong/weak distinction. If every table has a system-generated IDENTITY column as its PK (as CabotTrail does), every entity *looks* like a strong entity from a physical standpoint — `OrderLineID IDENTITY(1,1)` makes each order line uniquely identifiable on its own.

The *conceptual* weakness remains: an order line without an order has no business meaning. The surrogate key makes it physically independent, but the business relationship still requires the owner entity. This is why the conceptual model (ERD) and the physical model (SQL) need to be understood separately.

---

8. Many-to-Many Relationships

Many-to-many relationships require special handling in both ERDs and physical models.

8.1 Why M:N Relationships Need Resolution

Consider a direct M:N relationship:

```
PRODUCT ──O<──>O── PRODUCT_CATEGORY
```

In the physical model, you cannot implement this with just two tables. There is no single column on either table that holds all the relationships. You cannot put `CategoryID` on `Products` (a product has many categories — you would need multiple `CategoryID` columns, with an unknown maximum). You cannot put `ProductID` on `Categories` (same problem).

8.2 The Junction Table (Associative Entity)

The solution is to introduce a **junction table** — an entity that represents the relationship itself. It has a row for each (Product, Category) pair:

```
PRODUCT ──|──O<── PRODUCT_CATEGORY_ASSIGNMENT ──>O──|── PRODUCT_CATEGORY
```

The junction table `PRODUCT_CATEGORY_ASSIGNMENT` has:

- `ProductID` FK → references `Products.ProductID`
- `ProductCategoryID` FK → references `ProductCategories.ProductCategoryID`
- A composite PK: `(ProductID, ProductCategoryID)`

This resolves the M:N relationship into two 1:N relationships — one from Product to the junction, one from Category to the junction.

8.3 Junction Tables with Their Own Attributes

Sometimes the relationship itself has attributes. A junction table can carry those attributes.

Example: If CabotTrail tracked the date a product was first assigned to each category, the `ProductCategoryAssignment` would have a `DateAssigned` column — an attribute of the relationship, not of either entity.

```
CREATE TABLE Inventory.ProductCategoryAssignments
(
```

```
    ProductID           INT     NOT NULL,
    ProductCategoryID   INT     NOT NULL,
    DateAssigned        DATE    NULL,       -- Attribute of the relationship

    CONSTRAINT PK_ProductCategoryAssignments
        PRIMARY KEY (ProductID, ProductCategoryID),
    CONSTRAINT FK_PCA_Product
        FOREIGN KEY (ProductID) REFERENCES Inventory.Products (ProductID),
    CONSTRAINT FK_PCA_Category
        FOREIGN KEY (ProductCategoryID) REFERENCES Inventory.ProductCategories
(ProductCategoryID)
);
```

When a junction table has its own attributes, it has elevated to a full **associative entity** — a concept that bridges between entity and relationship.

8.4 Identifying M:N Relationships

The signal for a potential M:N relationship: if you find yourself saying "one [A] can be associated with many [B]s, AND one [B] can be associated with many [A]s" — that is an M:N relationship requiring a junction table.

Common M:N relationships in business systems:

- Products ↔ Categories
- Employees ↔ Projects
- Students ↔ Courses
- Orders ↔ Promotions (an order can have multiple promotions; a promotion applies to many orders)
- Ingredients ↔ Recipes

---

9. Supertypes and Subtypes

9.1 The Problem They Solve

Sometimes multiple entities share a set of common attributes but also have their own distinct attributes. The brute-force solution — duplicate the common attributes in both tables — violates normalization. The elegant solution is a **supertype/subtype relationship**.

Example: CabotTrail has `Employees` and (potentially) `Customers` — both are `People` with a name, an address, a contact method. But employees have `JobTitle`, `HireDate`, `Department`; customers have `CreditLimit`, `AccountOpenedDate`, `CustomerGroupName`.

9.2 The Supertype

The **supertype** is the parent entity that holds attributes common to all subtypes. In CabotTrail, `People` is the supertype:

```
PERSON
├── PersonID (PK)
├── FullName
├── PreferredName
├── EmailAddress
└── PhoneNumber
```

9.3 The Subtypes

Subtypes are child entities that hold attributes specific to each subtype, plus a FK to the supertype:

```
EMPLOYEE                          CUSTOMER
├── EmployeeID (PK, FK→Person)    ├── CustomerID (PK, FK→Person — if applicable)
├── JobTitle                      ├── CreditLimit
├── HireDate                      ├── AccountOpenedDate
├── Department                    └── CustomerGroupName
└── IsActive
```

The supertype/subtype relationship is drawn with a triangle or an arc connecting the subtype entities to the supertype.

9.4 Disjoint vs Overlapping Subtypes

Disjoint subtypes: a person can be either an employee or a customer, but not both. (Exclusive membership.)

Overlapping subtypes: a person can be both an employee *and* a customer simultaneously. (Shared membership.)

The CabotTrail case is likely overlapping — an employee could also be a CabotTrail customer.

9.5 Physical Implementation Options

Option 1 — **Table per subtype** (CabotTrail's approach): `People` table + `Employees` table + `Customers` table. Each subtype table has a FK to `People`. Queries joining all attributes require a join between the supertype and subtype tables.

Option 2 — **Single table with NULLs**: All columns for all subtypes in one table. Non-applicable columns are NULL. Simple queries; wastes storage; difficult to enforce subtype-specific constraints.

Option 3 — **Table per subtype, no supertype table**: Each subtype table contains all common and specific attributes. Duplication of common attributes; simpler queries within a subtype.

Each option has trade-offs. Option 1 is the most normalized and most maintainable.

---

10. From Narrative to ERD: A Worked Example

This section demonstrates the complete process of building an ERD from a business narrative — the same process you will practice with the CabotTrail Loyalty Program in the labs.

10.1 The Narrative

> *CabotTrail Outdoor is launching a customer loyalty programme. Customers earn points on each purchase based on the amount spent. Points can be redeemed for rewards (discount vouchers, free products, gift items). Each customer has a loyalty account, and the account tracks their current point balance and their tier status (Bronze, Silver, Gold, Platinum). Every point transaction — whether earned or redeemed — must be recorded with a date and the number of points.*

10.2 Step 1: Identify Entities

Read the narrative and find the nouns:

- **Customer** — already exists; we extend it
- **LoyaltyAccount** — one per customer; tracks balance and tier
- **LoyaltyTransaction** — each point earn or redemption
- **Reward** — the things points can be redeemed for
- **Tier** — Bronze, Silver, Gold, Platinum (could be a lookup table)

10.3 Step 2: Identify Attributes

For each entity, list the attributes mentioned or implied:

```
LOYALTY_ACCOUNT
├── AccountID (PK)
├── CustomerID (FK → Customer)
├── PointBalance
├── TierID (FK → Tier)
└── AccountOpenedDate
```

```
LOYALTY_TRANSACTION
├── TransactionID (PK)
├── AccountID (FK → LoyaltyAccount)
├── TransactionDate
├── PointsAmount (positive = earned, negative = redeemed)
├── TransactionType ('EARN' or 'REDEEM')
├── RewardID (FK → Reward — nullable, for redeem transactions)
└── InvoiceID (FK → Sales.Invoices — nullable, for earn transactions)

REWARD
├── RewardID (PK)
├── RewardName
├── RewardDescription
├── PointCost
└── IsAvailable

TIER
├── TierID (PK)
├── TierName
├── MinimumPoints
└── BenefitDescription
```

10.4 Step 3: Identify Relationships

- `Customer` *has* one `LoyaltyAccount` (1:1 — each customer has at most one account)
- `LoyaltyAccount` *has* many `LoyaltyTransactions` (1:N)
- `LoyaltyTransaction` *may redeem* one `Reward` (optional — earn transactions have no reward) (1:N from Reward)
- `LoyaltyAccount` *is in* one `Tier` (N:1 from LoyaltyAccount to Tier)

10.5 Step 4: Check Cardinality

For each relationship, ask in both directions:

Customer ↔ LoyaltyAccount:

- "Can a customer have more than one loyalty account?" — per the narrative, no: one account per customer.
- "Can a customer exist without a loyalty account?" — yes: existing customers have no account until they join the programme.
- "Can a loyalty account exist without a customer?" — no: the account belongs to a customer.
- Result: 1:1, optional on the Customer side, mandatory on the LoyaltyAccount side.

LoyaltyAccount ↔ LoyaltyTransaction:

- "Can an account have zero transactions?" — yes, a new account has no transactions yet.

- "Can a transaction exist without an account?" — no.
- Result: 1:N from Account, optional on Account side, mandatory on Transaction side.

10.6 The Questions That Surface

A real design session would generate questions the narrative did not answer:

- Can points be transferred between accounts?
- What happens to points if a customer closes their account?
- Can a transaction earn points on a return/refund (negative purchase)?
- Is there an expiry date on points?
- Can a single sales transaction earn points for more than one customer (e.g., a shared account)?

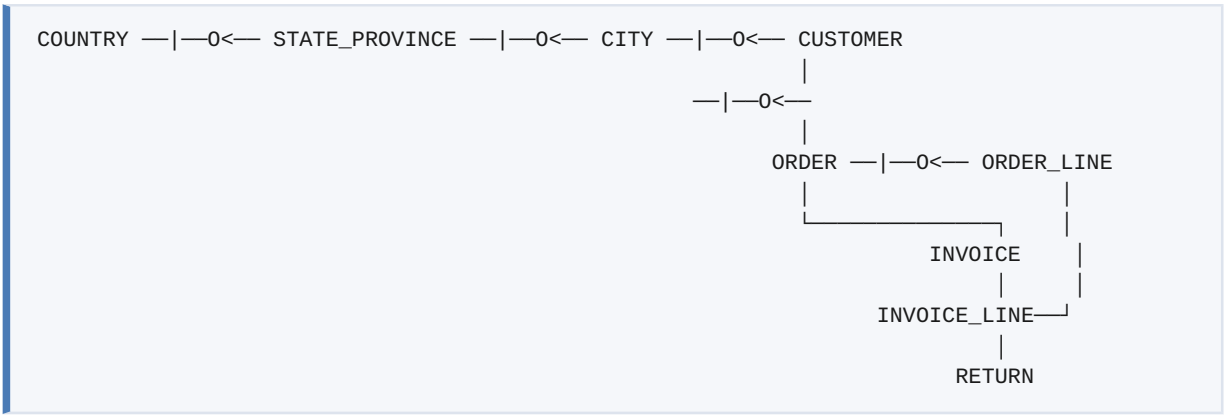
Each of these questions may change the ERD. Surfacing them before implementation is the value of the design process.

---

11. Reading the CabotTrail ERD

11.1 Sales Domain ERD

The core Sales domain can be read from the OLTP schema:



Reading this ERD:

- A Country has zero or many State/Provinces (countries with no provinces tracked = optional)
- A Province belongs to exactly one Country (mandatory)
- A City belongs to exactly one Province (mandatory)
- A Customer has a delivery city — one city has zero or many customers

- A Customer places zero or many Orders
- An Order has one or many Order Lines
- An Order generates zero or one Invoice (an order may not be invoiced immediately)
- An Invoice covers exactly one Order
- An Invoice Line references exactly one Order Line
- An Order Line may have zero or one Return

11.2 What the ERD Reveals

The ERD makes visible things that are easy to miss in SQL:

- **The separation of Orders and Invoices** is explicit in the diagram — it shows these are distinct entities with a 1:1 (or 1:0) relationship
- **The geography hierarchy** shows three levels of nesting before reaching Customer — making the normalization rationale visible
- **The Return entity** depends on InvoiceLine (weak entity) — a return cannot exist without an invoiced line

11.3 ERD as Documentation

The ERD is the most important documentation artifact for a database. A developer who reads the ERD before querying the database will understand the schema much faster than one who discovers it through trial-and-error SQL.

For the CabotTrail OLTP, an ERD generated from the schema (using SSMS's Database Diagrams feature or a tool like Lucidchart) shows the complete structure at a glance — every entity, every relationship, every cardinality constraint — in a form that is faster to read than dozens of CREATE TABLE statements.

---

12. Chapter Summary

- An ERD is a **conceptual model** of the business's information structure — readable by both technical and non-technical stakeholders. It is drawn before SQL and revised through stakeholder conversation.
- **Crow's Foot notation** uses rectangles (entities), lines with crow's foot symbols (relationships), and oval or in-box attributes. The symbols at each end of a relationship line express minimum and maximum cardinality.

- **Entities** are things the business tracks; identified from nouns in the narrative. **Attributes** are properties of entities; they become columns. Special attribute types: multi-valued (require a separate table), derived (computable from others), composite (multiple sub-components).
- **Cardinality** defines the numerical constraint on a relationship: 1:1, 1:N, M:N. **Participation** defines whether every instance must participate (mandatory) or may participate (optional). Both are business rules.
- **Weak entities** cannot be identified without their owner entity. They have a composite PK that includes the owner's PK. Surrogate keys make this partially invisible in physical models.
- **Many-to-many relationships** require a **junction table** (associative entity) that resolves the M:N into two 1:N relationships. Junction tables may carry their own attributes.
- **Supertypes and subtypes** model entity hierarchies where some entities share common attributes. The supertype holds shared attributes; subtypes hold specialised attributes plus a FK to the supertype.
- The ERD process: identify entities → identify attributes → identify relationships → determine cardinality → validate with stakeholders → surface unanswered questions.

---

13. Review Questions

1. Explain the difference between an ERD and a schema diagram. A colleague shows you a screenshot of SSMS's Database Diagram tool for `CabotTrailOutdoor` and calls it "the ERD." What is correct about this description and what is missing?
2. For each of the following pairs, determine the cardinality (1:1, 1:N, or M:N) and the participation (mandatory/optional for each side) of the relationship. Justify each answer with a business reason: a. `Supplier` ↔ `Product` (a supplier supplies products to CabotTrail) b. `Invoice` ↔ `InvoiceLine` c. `Employee` ↔ `SalesOrder` (employees take sales orders) d. `Product` ↔ `Color`
3. Is `LoyaltyTransaction` in the Loyalty Programme ERD from section 10 a weak entity? Explain your reasoning, and describe what its composite primary key would look like in the physical model if it were implemented without a surrogate key.
4. Draw (or describe in text notation) the ERD for the following narrative:

> *"CabotTrail sells products. Each product belongs to at least one category, and each category may contain many products. A product has a name, a unit price, and a weight. Each product is supplied by exactly one supplier, but a supplier may supply many products." Include entities, attributes, relationships, cardinality, and participation.*

5. Explain the difference between a multi-valued attribute and a one-to-many relationship. Give a specific example of each from the CabotTrail domain, and show how each is represented differently in the ERD.
6. The CabotTrail OLTP has `Application.People`, `Application.Employees`, and `Sales.Customers`. Identify the supertype/subtype relationship and explain which option was chosen for physical implementation (table per subtype, single table with NULLs, or combined). What are the trade-offs of this choice?
7. A new business requirement: CabotTrail wants to offer bundle deals — a bundle consists of two or more products sold together at a combined price. The bundle itself is treated as a product that can be ordered. Draw (or describe) the ERD extension needed to model this. Identify any junction tables required.
8. Run the FK relationship query from Chapter 1 against `CabotTrailOutdoor`. Pick the `Sales.Returns` entity and draw its immediate ERD neighbourhood — the entity itself, all entities it directly relates to, the cardinality, and participation of each relationship. What does the ERD tell you about the business rules governing returns?

🔍 Deeper Dive

Going Further with Entity-Relationship Modelling

The Original Chen Notation

The ERD notation introduced by Peter Chen in his 1976 paper uses different symbols from Crow's Foot:

- **Rectangles** for entities
- **Diamonds** for relationships (with the relationship name inside)
- **Ovals** for attributes
- **Lines** from entities to diamonds to show participation
- **Double diamonds** for identifying relationships (with weak entities)

- **Double rectangles** for weak entities

Chen notation is more visually expressive — relationships are first-class objects with their own symbol — but it becomes cluttered for large schemas. Crow's Foot notation, developed later, is more compact and is now the industry standard.

Understanding both notations is useful because academic textbooks (Elmasri & Navathe, Connolly & Begg) use Chen notation, while industry tools (Visio, Lucidchart, draw.io, ERDPlus) use Crow's Foot or variations of it.

Chen, P. P. (1976). The entity-relationship model — Toward a unified view of data. *ACM Transactions on Database Systems*, 1(1), 9–36.

UML Class Diagrams vs ERDs

UML (Unified Modelling Language) class diagrams are sometimes used in place of ERDs for database design, especially in object-oriented development contexts. A UML class diagram shows:

- Classes (equivalent to entities)
- Attributes (with types)
- Associations (equivalent to relationships)
- Multiplicity (equivalent to cardinality, written as `0..1`, `1..*`, `0..*`)

The key differences from ERDs:

- UML class diagrams include **methods** (operations) — not relevant to relational databases
- UML uses different multiplicity notation (`0..1` instead of Crow's Foot symbols)
- UML supports **inheritance** notation more explicitly than ERDs

In practice, both notations are used. Database professionals tend to use ERDs; software architects tend to use UML. Knowing both is an advantage in teams that mix both disciplines.

When ERDs Are Not Enough: The Relational Schema Diagram

An ERD captures logical structure. For communicating the complete physical schema — including all columns, data types, constraint names, and index definitions — a **relational schema diagram** (what SSMS Database Diagrams generates) is more appropriate.

The two documents serve different audiences at different stages:

- ERD: design phase, stakeholder communication, conceptual review
- Schema diagram: implementation phase, developer reference, DBA documentation

Both should exist for a production database. The ERD captures intent; the schema diagram captures implementation.

Entity Identification Anti-Patterns

Some common mistakes in entity identification:

Verbs as entities: "Purchasing" is not an entity — "Purchase Order" is. "Delivery" as a standalone entity may be premature — the delivery information may belong as attributes of `Order` (delivery address, expected delivery date).

Attributes as entities: "Product Colour" is probably an attribute of `Product`, not an entity in its own right — unless the business needs to track colour codes independently (e.g., for colour management across product lines). The test: does `Colour` have multiple attributes? Does it exist independently of any product?

Redundant entities: "RegularCustomer" and "WholesaleCustomer" as separate entities, when both are instances of `Customer` with a `CustomerType` attribute.

Missing entities: The classic example — missing the `OrderLine` entity by jumping straight from `Order` to `Product` with a many-to-many. The ordered quantity, unit price at time of order, and pick status belong to the line, not to either `Order` or `Product`.

References and Further Reading

1. Chen, P. P. (1976). The entity-relationship model — Toward a unified view of data. *ACM Transactions on Database Systems*, 1(1), 9–36. — The original ERD paper.

2. Connolly, T., & Begg, C. (2015). *Database Systems: A Practical Approach to Design, Implementation, and Management* (6th ed.). Pearson. — Chapters 12–14 cover ER modelling in depth with many worked examples.

3. Elmasri, R., & Navathe, S. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson. — Chapters 3–4 cover the ER model using Chen notation; Chapter 9 covers enhanced ER (supertypes/subtypes).

4. Hoffer, J. A., Ramesh, V., & Topi, H. (2019). *Modern Database Management* (13th ed.). Pearson. — Strong practical coverage of ERD drawing techniques and case studies.

5. Microsoft. (2024). *Database Diagrams (SSMS)*. https://learn.microsoft.com/en-us/sql/ssms/visual-db-tools/design-database-diagrams-visual-database-tools

6. Lucidchart. (2024). *ERD Tutorial*. https://www.lucidchart.com/pages/er-diagrams — Practical Crow's Foot notation reference and drawing guide.

> ***Database Design for Analysts*** *| © Patrick Dolinger, NSCC Institute of Technology CC BY 4.0 — Share and adapt freely with attribution*

Chapter 3: From ERD to Tables — Mapping Diagrams to DDL

Database Design for Analysts *From business requirements to relational structures*

© Patrick Dolinger, NSCC Institute of Technology Licensed under Creative Commons Attribution 4.0 International (CC BY 4.0) You are free to share and adapt this material for any purpose, provided appropriate credit is given.

Chapter Overview

Chapter 2 produced a logical data model — an entity-relationship diagram that represents the business's information structure. This chapter makes the crossing from logical to physical: translating every entity, attribute, and relationship in the ERD into concrete SQL Server DDL.

The translation is governed by a set of **mapping rules** — a systematic procedure that converts each ERD element into its physical equivalent. The rules are not arbitrary; each one reflects a deliberate design principle. Understanding the rules deeply means you can apply them confidently to new designs and adapt them when business requirements are unusual.

By the end of this chapter you will be able to:

- Apply the seven ERD-to-table mapping rules to produce a complete relational schema from any ERD
- Map strong entities, weak entities, and associative entities to tables with correct primary and foreign keys
- Choose between surrogate and natural primary keys with justification
- Map 1:1, 1:N, and M:N relationships to their correct physical implementations
- Map supertypes and subtypes to tables, choosing among implementation options
- Write complete DDL for a mapped schema including all constraints
- Verify the mapping by tracing every ERD element to a physical object
- Describe the design decisions made when building the CabotTrail OLTP schema

Table of Contents

1. The Mapping Process
2. Rule 1: Strong Entities
3. Rule 2: Simple Attributes
4. Rule 3: Multi-Valued Attributes
5. Rule 4: Weak Entities
6. Rule 5: One-to-Many Relationships
7. Rule 6: One-to-One Relationships
8. Rule 7: Many-to-Many Relationships
9. Mapping Supertypes and Subtypes
10. Choosing Data Types During Mapping
11. Naming Conventions
12. A Complete Mapping: The Loyalty Programme
13. Verifying the Mapping
14. Chapter Summary
15. Review Questions
16. 🔍 Deeper Dive

---

1. The Mapping Process

The ERD-to-table mapping is systematic, not creative. Given an ERD, two designers following the same mapping rules should produce the same relational schema. This is by design — the logical model (ERD) captures the creative and analytical decisions; the mapping is the mechanical step that converts those decisions to SQL.

1.1 What the Mapping Produces

For each ERD element, the mapping produces one or more physical objects:

ERD element	Physical result
Strong entity	One table
Weak entity	One table with composite or FK-assisted PK
Simple attribute	One column

ERD element	Physical result
Multi-valued attribute	One new table with FK back to the parent
Derived attribute	One computed column, or no column (compute at query time)
Composite attribute	Multiple columns (one per component)
1:N relationship	FK column on the "many" table
1:1 relationship	FK on one table (choose based on participation/optionality)
M:N relationship	One new junction table with FKs to both entities
Supertype/subtype	Two or more tables depending on strategy chosen

1.2 The Mapping is Not the Only Output

The mapping produces DDL — `CREATE TABLE` statements. But the designer must also decide:

- What data type does each attribute map to?
- What constraints enforce which business rules?
- What indexes are needed for query performance?
- What naming conventions are used?

Data types and constraints are covered in Chapters 6 and 7. This chapter focuses on the structural mapping — getting the right tables with the right keys — and introduces data type selection as part of the practical worked example.

---

2. Rule 1: Strong Entities

Every strong entity becomes one table.

The table takes the entity name. If the entity has a natural candidate key that is stable, unique, and not null, use it as the primary key. If not, introduce a surrogate key.

```
-- ENTITY: Customer
-- Strong entity → one table
CREATE TABLE Sales.Customers
(
    CustomerID      INT             NOT NULL IDENTITY(1,1),
    CustomerName    NVARCHAR(100)   NOT NULL,
    CreditLimit     DECIMAL(18,2)   NOT NULL DEFAULT 0,
    AccountOpenedDate DATE          NULL,
```

```
    CONSTRAINT PK_Customers PRIMARY KEY (CustomerID)
);
```

The entity name (`Customer`) becomes the table name (`Customers` — pluralised in the physical model; convention varies but CabotTrail uses plural table names).

The `CustomerID IDENTITY(1,1)` is a surrogate key — introduced because `CustomerName` is not reliably unique (two customers could have the same name) and business-meaningful attributes like email are not always available or stable.

2.1 When to Use a Natural Key as PK

Use a natural key as the primary key when:

- It is **guaranteed unique** across all past and future instances
- It will **never change** for the life of the entity
- It is **always known** when a new instance is created
- It is **compact** (short enough to be efficient as a join key)

Strong candidates in CabotTrail:

- `StateProvinceCode` for provinces (2-char code, stable, unique, always known)
- `CountryName` or an ISO country code for countries

Weak candidates: `CustomerName` (not guaranteed unique), `EmailAddress` (may change), `ProductCode` (may be NULL for some products).

When in doubt, use a surrogate key.

---

3. Rule 2: Simple Attributes

Every simple, single-valued attribute of an entity becomes a column in the entity's table.

The column inherits the attribute name and gets an appropriate data type:

```
-- ENTITY: Product attributes mapped to columns
CREATE TABLE Inventory.Products
(
    ProductID              INT             NOT NULL IDENTITY(1,1),
    ProductCode            NVARCHAR(20)    NULL,       -- Optional attribute
    ProductName            NVARCHAR(200)   NOT NULL,
    UnitPrice              DECIMAL(18,2)   NOT NULL DEFAULT 0,
    RecommendedRetailPrice DECIMAL(18,2)   NULL,
```

```
    TypicalWeightPerUnit    DECIMAL(18,3)   NULL,
    IsDiscontinued          BIT             NOT NULL DEFAULT 0,

    CONSTRAINT PK_Products PRIMARY KEY (ProductID)
);
```

Derived attributes (attributes computable from other stored values) have two mapping options:

Option 1 — **Omit from the physical model:** Do not store it. Compute it at query time using a SQL expression. This is the most storage-efficient approach and guarantees accuracy, but requires computation on every query.

```
-- Derived: LineTotal = Quantity × UnitPrice
-- Option 1: Compute in every query, do not store
SELECT Quantity * UnitPrice AS LineTotal FROM InvoiceLines;
```

Option 2 — **Store as a computed column:** Define it as a `AS expression` column, optionally `PERSISTED` to store the result physically:

```
-- Derived: LineTotalIncludingTax stored as a computed column
LineTotalIncludingTax AS (LineTotal + TaxAmount) PERSISTED
-- SQL Server computes and stores it; automatically maintained on UPDATE
```

The CabotTrail OLTP uses computed columns for `LineTotal` in `Sales.InvoiceLines`. The CabotTrail DW stores derived columns (`GrossProfit`, `GrossProfitMarginPct`) as regular populated columns loaded by ETL — a different approach appropriate for a read-optimised warehouse.

---

4. Rule 3: Multi-Valued Attributes

Every multi-valued attribute becomes a new table. The new table has:

- A FK referencing the parent entity's PK
- One column for the attribute value
- A PK (either the parent FK alone if each parent can have only one of each value, or a surrogate key if duplicate values per parent are allowed)

```
Entity: PRODUCT
Multi-valued attribute: CategoryName (a product can belong to many categories)
```

This becomes two tables plus a junction:

```
-- The multi-valued attribute becomes a separate entity
CREATE TABLE Inventory.ProductCategories
(
    ProductCategoryID   INT             NOT NULL IDENTITY(1,1),
    ProductCategoryName NVARCHAR(60)    NOT NULL,

    CONSTRAINT PK_ProductCategories   PRIMARY KEY (ProductCategoryID),
    CONSTRAINT UQ_ProductCategoryName UNIQUE (ProductCategoryName)
);

-- And a junction table to resolve the M:N
CREATE TABLE Inventory.ProductCategoryAssignments
(
    ProductID           INT NOT NULL,
    ProductCategoryID   INT NOT NULL,

    CONSTRAINT PK_ProductCategoryAssignments
        PRIMARY KEY (ProductID, ProductCategoryID),
    CONSTRAINT FK_PCA_Product
        FOREIGN KEY (ProductID) REFERENCES Inventory.Products (ProductID),
    CONSTRAINT FK_PCA_Category
        FOREIGN KEY (ProductCategoryID)
        REFERENCES Inventory.ProductCategories (ProductCategoryID)
);
```

Notice: the multi-valued attribute `CategoryName` becomes a full entity (`ProductCategories`) with its own PK, plus a junction table linking it to `Products`. This is because the categories have their own identity — they are not just tags, they are entities that can be described independently (names, descriptions, margins).

If the multi-valued attribute had no independent existence — for example, a product having multiple phone numbers — the mapping is simpler:

```
-- No independent entity: just a value table
CREATE TABLE dbo.ProductPhoneNumbers
(
    PhoneID     INT             NOT NULL IDENTITY(1,1),
    ProductID   INT             NOT NULL,
    PhoneNumber NVARCHAR(20)    NOT NULL,
    PhoneType   NVARCHAR(20)    NOT NULL DEFAULT 'Main',

    CONSTRAINT PK_ProductPhoneNumbers PRIMARY KEY (PhoneID),
    CONSTRAINT FK_ProductPhone_Product
        FOREIGN KEY (ProductID) REFERENCES Inventory.Products (ProductID)
);
```

---

5. Rule 4: Weak Entities

A weak entity becomes a table with a composite primary key that includes the FK to the owner entity.

Or, if a surrogate key is used, the composite logical identity is enforced with a UNIQUE constraint rather than being the PK itself.

```
Entity: ORDER_LINE (weak — identity depends on ORDER)
Owner: ORDER
```

```
-- Option A: Composite PK (pure relational approach)
CREATE TABLE Sales.OrderLines
(
    OrderID             INT             NOT NULL,
    OrderLineID         INT             NOT NULL,   -- Line number within order
    ProductID           INT             NOT NULL,
    Quantity            INT             NOT NULL,
    PickedQuantity      INT             NOT NULL DEFAULT 0,
    UnitPrice           DECIMAL(18,2)   NOT NULL,

    CONSTRAINT PK_OrderLines
        PRIMARY KEY (OrderID, OrderLineID),         -- Composite PK
    CONSTRAINT FK_OrderLines_Order
        FOREIGN KEY (OrderID) REFERENCES Sales.Orders (OrderID)
);

-- Option B: Surrogate PK + UNIQUE constraint (CabotTrail's actual approach)
CREATE TABLE Sales.OrderLines
(
    OrderLineID         INT             NOT NULL IDENTITY(1,1),  -- Surrogate PK
    OrderID             INT             NOT NULL,
    ProductID           INT             NOT NULL,
    Quantity            INT             NOT NULL,
    PickedQuantity      INT             NOT NULL DEFAULT 0,
    UnitPrice           DECIMAL(18,2)   NOT NULL,

    CONSTRAINT PK_OrderLines
        PRIMARY KEY (OrderLineID),                  -- Surrogate PK
    CONSTRAINT FK_OrderLines_Order
        FOREIGN KEY (OrderID) REFERENCES Sales.Orders (OrderID)
    -- No explicit UNIQUE(OrderID, LineNumber) unless duplicate lines
    -- in the same order are prevented
);
```

CabotTrail uses Option B — surrogate keys throughout. This makes joins slightly simpler (single-column FKs) at the cost of the composite identity being invisible in the schema structure.

---

6. Rule 5: One-to-Many Relationships

In a 1:N relationship, place a FK in the table on the "many" side, referencing the PK of the "one" side.

This is the most frequently applied rule — most relationships in a relational database are 1:N.

```
Relationship: CUSTOMER ──|──O<── ORDER
One side: Customer
Many side: Order
```

```
-- FK goes on the Order table (the "many" side)
CREATE TABLE Sales.Orders
(
    OrderID             INT     NOT NULL IDENTITY(1,1),
    CustomerID          INT     NOT NULL,   -- FK to Customer
    OrderDate           DATE    NOT NULL,
    ExpectedDeliveryDate DATE   NULL,
    SalesRepID          INT     NULL,       -- FK to Employee (nullable — optional)
    CityID              INT     NOT NULL,   -- FK to Cities

    CONSTRAINT PK_Orders PRIMARY KEY (OrderID),
    CONSTRAINT FK_Orders_Customer
        FOREIGN KEY (CustomerID) REFERENCES Sales.Customers (CustomerID),
    CONSTRAINT FK_Orders_SalesRep
        FOREIGN KEY (SalesRepID) REFERENCES Application.Employees (EmployeeID),
    CONSTRAINT FK_Orders_City
        FOREIGN KEY (CityID) REFERENCES Application.Cities (CityID)
);
```

6.1 Mandatory vs Optional in the FK

The participation rules from the ERD translate directly to `NOT NULL` vs `NULL` on the FK:

ERD participation (many side)	FK in table
Mandatory (every order has a customer)	`CustomerID INT NOT NULL`
Optional (an order may have no sales rep)	`SalesRepID INT NULL`

If `CustomerID` could be NULL, an order could exist without a customer — violating the mandatory participation rule from the ERD. The `NOT NULL` constraint enforces the ERD's business rule.

6.2 Multiple Relationships to the Same Entity

`Sales.Orders` has FKs to `Customers`, `Employees` (as SalesRep), and `Cities`. Multiple FKs to different entities on the same table are normal and common.

A table can also have multiple FKs to the **same** entity — called **role relationships** or **multiple relationships**. A classic example: an `Invoice` references both a `Customer` (for billing) and a different `Customer` (for delivery, if different). Each FK must have a distinct name and a distinct column:

```
-- Multiple FKs to the same entity with different roles
CREATE TABLE Sales.Invoices
(
    InvoiceID           INT NOT NULL IDENTITY(1,1),
    BillingCustomerID   INT NOT NULL,   -- FK to Customer (billing)
    DeliveryCustomerID  INT NULL,       -- FK to Customer (delivery — may differ)
    OrderID             INT NOT NULL,
    InvoiceDate         DATE NOT NULL,
    DueDate             DATE NOT NULL,
    ...

    CONSTRAINT FK_Invoices_BillingCustomer
        FOREIGN KEY (BillingCustomerID) REFERENCES Sales.Customers (CustomerID),
    CONSTRAINT FK_Invoices_DeliveryCustomer
        FOREIGN KEY (DeliveryCustomerID) REFERENCES Sales.Customers (CustomerID)
);
```

---

7. Rule 6: One-to-One Relationships

In a 1:1 relationship, place the FK on the table whose participation is optional — or on the table that logically extends the other.

1:1 relationships require a judgment call about which table carries the FK. The decision is guided by:

1. **Participation**: If participation is mandatory on one side and optional on the other, put the FK on the optional side. The optional side may or may not have a corresponding row, so the FK on that table can be NULL without breaking the other.
2. **Logical extension**: If one entity is the "extension" of the other (a weak specialisation), put the FK on the extension table.
3. **Query frequency**: If queries almost always access the two entities together, consider merging them into one table.

7.1 Mapping 1:1 with Optional Participation

```
Relationship: EMPLOYEE ──|──O── EMPLOYEE_PARKING_PERMIT (optional — not all employees
have parking)
```

```
-- FK on the optional side (Employee_Parking_Permit)
CREATE TABLE HR.EmployeeParkingPermits
(
    PermitID        INT     NOT NULL IDENTITY(1,1),
    EmployeeID      INT     NOT NULL UNIQUE,   -- UNIQUE enforces the 1:1
```

```
    PermitNumber    NVARCHAR(20) NOT NULL,
    ExpiryDate      DATE    NOT NULL,

    CONSTRAINT PK_EmployeeParkingPermits PRIMARY KEY (PermitID),
    CONSTRAINT FK_Permit_Employee
        FOREIGN KEY (EmployeeID) REFERENCES Application.Employees (EmployeeID),
    CONSTRAINT UQ_Permit_Employee        UNIQUE (EmployeeID)
    -- The UNIQUE constraint on EmployeeID enforces the "at most one permit per
employee" rule
);
```

The `UNIQUE` constraint on `EmployeeID` is what enforces the "one" in the 1:1 relationship. Without it, multiple parking permits per employee would be allowed — making it effectively 1:N.

7.2 The UNIQUE Constraint as a 1:1 Enforcer

This is a crucial physical design point: a FK alone does not enforce 1:1. A FK on `PermitID` → `EmployeeID` allows multiple permits for one employee (the FK allows repeated values). Adding `UNIQUE(EmployeeID)` to the permit table ensures each employee appears at most once — enforcing the 1:1 maximum.

---

8. Rule 7: Many-to-Many Relationships

Every M:N relationship becomes a junction table. The junction table:

- Has a FK referencing the PK of each participating entity
- Has a PK that is the composite of both FKs (or a surrogate key plus a UNIQUE constraint on the FK combination)
- May carry attributes of the relationship as additional columns

```
Relationship: PRODUCT ──O<──>O── PRODUCT_CATEGORY
```

```
CREATE TABLE Inventory.ProductCategoryAssignments
(
    ProductID           INT NOT NULL,
    ProductCategoryID   INT NOT NULL,

    CONSTRAINT PK_ProductCategoryAssignments
        PRIMARY KEY (ProductID, ProductCategoryID),
    CONSTRAINT FK_PCA_Product
        FOREIGN KEY (ProductID)
        REFERENCES Inventory.Products (ProductID),
    CONSTRAINT FK_PCA_ProductCategory
        FOREIGN KEY (ProductCategoryID)
```

```
        REFERENCES Inventory.ProductCategories (ProductCategoryID)
);
```

8.1 Adding a Surrogate Key to the Junction Table

Some designs add a surrogate key to the junction table for convenience — particularly when the junction is referenced by other tables:

```
-- Junction with surrogate key: useful when other tables need to FK to the assignment
CREATE TABLE Inventory.ProductCategoryAssignments
(
    AssignmentID        INT     NOT NULL IDENTITY(1,1),
    ProductID           INT     NOT NULL,
    ProductCategoryID   INT     NOT NULL,

    CONSTRAINT PK_ProductCategoryAssignments PRIMARY KEY (AssignmentID),
    CONSTRAINT UQ_PCA_Combination            UNIQUE (ProductID, ProductCategoryID),
    CONSTRAINT FK_PCA_Product
        FOREIGN KEY (ProductID) REFERENCES Inventory.Products (ProductID),
    CONSTRAINT FK_PCA_ProductCategory
        FOREIGN KEY (ProductCategoryID)
        REFERENCES Inventory.ProductCategories (ProductCategoryID)
);
```

The `UNIQUE(ProductID, ProductCategoryID)` constraint preserves the M:N resolution even with the surrogate key.

8.2 When the Junction Becomes a Full Entity

Sometimes a junction table grows attributes until it becomes a full entity in its own right. The test: does the junction table have attributes that are not derivable from either parent entity? If yes, it is an **associative entity** and should be named and treated as a first-class entity.

In CabotTrail, if `ProductCategoryAssignments` needed to track which product manager approved the category assignment and when, it would become a full entity — not just a structural bridge.

---

9. Mapping Supertypes and Subtypes

Chapter 2 identified three physical implementation options. This section shows the DDL for each, using `People`, `Employees`, and `Customers` as the example.

9.1 Option 1: Table per Subtype (CabotTrail's Approach)

```
-- Supertype table: common attributes
CREATE TABLE Application.People
(
    PersonID        INT             NOT NULL IDENTITY(1,1),
    FullName        NVARCHAR(100)   NOT NULL,
    PreferredName   NVARCHAR(60)    NOT NULL,
    EmailAddress    NVARCHAR(256)   NULL,
    PhoneNumber     NVARCHAR(20)    NULL,

    CONSTRAINT PK_People PRIMARY KEY (PersonID)
);

-- Subtype 1: Employee-specific attributes
CREATE TABLE Application.Employees
(
    EmployeeID      INT             NOT NULL IDENTITY(1,1),
    PersonID        INT             NOT NULL UNIQUE,   -- 1:1 link to People
    JobTitle        NVARCHAR(100)   NOT NULL,
    Department      NVARCHAR(60)    NULL,
    HireDate        DATE            NULL,
    IsActive        BIT             NOT NULL DEFAULT 1,

    CONSTRAINT PK_Employees        PRIMARY KEY (EmployeeID),
    CONSTRAINT FK_Employees_Person
        FOREIGN KEY (PersonID) REFERENCES Application.People (PersonID),
    CONSTRAINT UQ_Employees_Person UNIQUE (PersonID)
);

-- Subtype 2: Customer-specific attributes
CREATE TABLE Sales.Customers
(
    CustomerID          INT             NOT NULL IDENTITY(1,1),
    CustomerName        NVARCHAR(100)   NOT NULL,   -- May differ from PersonID.FullName
    CustomerGroupName   NVARCHAR(100)   NULL,
    CreditLimit         DECIMAL(18,2)   NOT NULL DEFAULT 0,
    AccountOpenedDate   DATE            NULL,

    CONSTRAINT PK_Customers PRIMARY KEY (CustomerID)
    -- Note: CabotTrail Customers does not directly FK to People
    -- (wholesale customers are businesses, not individuals)
);
```

Note the design decision: CabotTrail `Customers` does not reference `People` because wholesale customers are organisations, not individuals. `Employees` references `People` because employees are individuals. This asymmetry reflects business reality.

9.2 Option 2: Single Table with NULLs

```
-- All attributes in one table; NULLs for non-applicable subtypes
CREATE TABLE dbo.AllPersons
```

```
(
    PersonID            INT             NOT NULL IDENTITY(1,1),
    PersonType          NVARCHAR(20)    NOT NULL,   -- 'Employee' or 'Customer'
    FullName            NVARCHAR(100)   NOT NULL,
    EmailAddress        NVARCHAR(256)   NULL,
    -- Employee-specific (NULL for customers)
    JobTitle            NVARCHAR(100)   NULL,
    HireDate            DATE            NULL,
    -- Customer-specific (NULL for employees)
    CreditLimit         DECIMAL(18,2)   NULL,
    AccountOpenedDate   DATE            NULL,

    CONSTRAINT PK_AllPersons PRIMARY KEY (PersonID)
);
```

Drawbacks:

- NULL columns are semantically meaningless for non-applicable subtypes
- Cannot enforce subtype-specific NOT NULL constraints (e.g., employees must have `JobTitle`, but the column must allow NULL for customers)
- Table grows wide as subtypes multiply

9.3 Option 3: Table per Concrete Class (No Supertype Table)

```
-- No shared table: each subtype is fully self-contained
CREATE TABLE Application.Employees
(
    EmployeeID  INT NOT NULL IDENTITY(1,1),
    FullName    NVARCHAR(100) NOT NULL,      -- Duplicated from supertype
    EmailAddress NVARCHAR(256) NULL,         -- Duplicated from supertype
    JobTitle    NVARCHAR(100) NOT NULL,
    HireDate    DATE NULL,
    ...
);

CREATE TABLE Sales.Customers
(
    CustomerID  INT NOT NULL IDENTITY(1,1),
    FullName    NVARCHAR(100) NOT NULL,      -- Duplicated from supertype
    EmailAddress NVARCHAR(256) NULL,         -- Duplicated from supertype
    CreditLimit DECIMAL(18,2) NOT NULL DEFAULT 0,
    ...
);
```

Advantage: simple queries — no joins needed to get all attributes. Disadvantage: common attributes are duplicated; changes to common attributes require updating multiple tables.

CabotTrail uses Option 1 (table per subtype) — the most normalized and maintainable approach.

---

10. Choosing Data Types During Mapping

When mapping attributes to columns, every attribute requires a data type decision. Chapter 3 of *SQL for Analysts* covered the type families; this section applies them to design decisions.

10.1 The Data Type Selection Checklist

For each attribute, ask:

1. **What is the domain?** Numbers, text, dates, true/false values?
2. **What is the range of valid values?** Affects numeric type size.
3. **What is the maximum length?** Affects character type size.
4. **Is precision critical?** Financial amounts need DECIMAL, not FLOAT.
5. **Will this be compared or sorted?** Date types sort correctly; date-as-string does not.
6. **Will this be NULLable?** Mandatory attributes should be NOT NULL.

10.2 Common Mapping Decisions

Attribute type	Recommended SQL Server type	Notes
Surrogate key	`INT NOT NULL IDENTITY(1,1)`	Use `BIGINT` if > 2 billion rows possible
Short code (e.g., province code)	`NCHAR(2) NOT NULL`	Fixed-length for known-length codes
Name or description	`NVARCHAR(n) NOT NULL`	Choose n based on maximum expected length
Long text	`NVARCHAR(MAX)`	Only if truly variable, large content
Price / financial amount	`DECIMAL(18,2) NOT NULL`	Never FLOAT for money
Rate or percentage	`DECIMAL(8,4)`	4 decimal places for percentages/rates
Date only	`DATE`	Not DATETIME unless time is needed
Date and time	`DATETIME2(0-7)`	Prefer over legacy DATETIME
Boolean flag	`BIT NOT NULL DEFAULT 0`	1=yes/true, 0=no/false
Count or quantity	`INT NOT NULL DEFAULT 0`	TINYINT/SMALLINT for small ranges
Weight / measurement	`DECIMAL(10,3)`	Appropriate precision for measurement

10.3 Mapping the Loyalty Programme Attributes

Applying the checklist to each `LoyaltyAccount` attribute:

Attribute	Decision	Rationale
AccountID	`INT NOT NULL IDENTITY(1,1)`	Surrogate PK
CustomerID	`INT NOT NULL`	FK — must always reference a customer
PointBalance	`INT NOT NULL DEFAULT 0`	Points are whole numbers; cannot go below 0 (add CHECK)
TierID	`TINYINT NOT NULL`	Four tiers; TINYINT (0–255) is sufficient
AccountOpenedDate	`DATE NOT NULL DEFAULT GETDATE()`	Date only; defaults to today

---

11. Naming Conventions

Consistent naming makes a database self-documenting. CabotTrail follows conventions worth adopting.

11.1 Table Names

- **Plural** noun describing the entity: `Customers`, `Orders`, `Products`
- **PascalCase** (each word capitalised): `OrderLines`, `ProductCategories`, `StateProvinces`
- **Schema-qualified**: `Sales.Orders`, `Inventory.Products`, `Application.Employees`
- **No abbreviations** unless universal: `Qty` is ambiguous; `Quantity` is clear

11.2 Column Names

- **PascalCase**: `CustomerName`, `AccountOpenedDate`, `IsDiscontinued`
- **Descriptive**: `CreditLimit` not `Limit`; `AccountOpenedDate` not `OpenDate`
- **Consistent FK naming**: FK columns share the name of the referenced PK column: `CustomerID` in `Orders` references `CustomerID` in `Customers`
- **Boolean columns start with `Is` or `Has`**: `IsDiscontinued`, `IsActive`, `HasLoyaltyAccount`
- **Date columns end with `Date`**: `OrderDate`, `HireDate`, `AccountOpenedDate`

11.3 Constraint Names

Explicit constraint names follow a pattern that makes them identifiable in error messages:

Constraint	Pattern	Example
Primary key	`PK_TableName`	`PK_Customers`
Foreign key	`FK_TableName_ReferencedTable`	`FK_Orders_Customers`

Constraint	Pattern	Example
Unique	`UQ_TableName_Column(s)`	`UQ_Customers_Email`
Check	`CK_TableName_Column`	`CK_Products_UnitPrice`
Default	`DF_TableName_Column`	`DF_Orders_OrderDate`

When SQL Server generates constraint names automatically (unnamed constraints), the generated names are cryptic: `PK__Customers__A1B2C3D4E5`. These are impossible to reference in ALTER TABLE statements. **Always name constraints explicitly**.

---

12. A Complete Mapping: The Loyalty Programme

Applying all seven rules to the Loyalty Programme ERD from Chapter 2.

12.1 Step 1: Map Strong Entities

```
USE CabotTrailOutdoor;
GO

-- Rule 1: Strong entity TIER
CREATE TABLE Sales.LoyaltyTiers
(
    TierID              TINYINT         NOT NULL IDENTITY(1,1),
    TierName            NVARCHAR(20)    NOT NULL,
    MinimumPoints       INT             NOT NULL DEFAULT 0,
    BenefitDescription  NVARCHAR(200)   NULL,

    CONSTRAINT PK_LoyaltyTiers              PRIMARY KEY (TierID),
    CONSTRAINT UQ_LoyaltyTiers_TierName     UNIQUE (TierName),
    CONSTRAINT CK_LoyaltyTiers_MinimumPoints CHECK (MinimumPoints >= 0)
);
GO

-- Rule 1: Strong entity REWARD
CREATE TABLE Sales.LoyaltyRewards
(
    RewardID            INT             NOT NULL IDENTITY(1,1),
    RewardName          NVARCHAR(100)   NOT NULL,
    RewardDescription   NVARCHAR(500)   NULL,
    PointCost           INT             NOT NULL,
    IsAvailable         BIT             NOT NULL DEFAULT 1,

    CONSTRAINT PK_LoyaltyRewards           PRIMARY KEY (RewardID),
    CONSTRAINT CK_LoyaltyRewards_PointCost CHECK (PointCost > 0)
);
GO
```

12.2 Step 2: Map LoyaltyAccount (1:1 with Customer)

```
-- Rule 6: 1:1 relationship — LoyaltyAccount extends Customer
-- FK on the optional side (not every customer has an account — yet)
CREATE TABLE Sales.LoyaltyAccounts
(
    AccountID           INT             NOT NULL IDENTITY(1,1),
    CustomerID          INT             NOT NULL,   -- FK → Customers
    PointBalance        INT             NOT NULL DEFAULT 0,
    TierID              TINYINT         NOT NULL,   -- FK → LoyaltyTiers
    AccountOpenedDate   DATE            NOT NULL DEFAULT GETDATE(),

    CONSTRAINT PK_LoyaltyAccounts              PRIMARY KEY (AccountID),
    CONSTRAINT UQ_LoyaltyAccounts_CustomerID   UNIQUE (CustomerID),   -- Enforces 1:1
    CONSTRAINT FK_LoyaltyAccounts_Customers
        FOREIGN KEY (CustomerID) REFERENCES Sales.Customers (CustomerID),
    CONSTRAINT FK_LoyaltyAccounts_LoyaltyTiers
        FOREIGN KEY (TierID) REFERENCES Sales.LoyaltyTiers (TierID),
    CONSTRAINT CK_LoyaltyAccounts_PointBalance CHECK (PointBalance >= 0)
);
GO
```

12.3 Step 3: Map LoyaltyTransaction (1:N from LoyaltyAccount)

```
-- Rule 5: 1:N relationship — LoyaltyAccount has many Transactions
-- Rule 4: Weak entity — Transaction identity depends on Account context
CREATE TABLE Sales.LoyaltyTransactions
(
    TransactionID       INT             NOT NULL IDENTITY(1,1),
    AccountID           INT             NOT NULL,   -- FK → LoyaltyAccounts
    TransactionDate     DATE            NOT NULL DEFAULT GETDATE(),
    PointsAmount        INT             NOT NULL,   -- Positive = earn, negative =
redeem
    TransactionType     NVARCHAR(10)    NOT NULL,   -- 'EARN' or 'REDEEM'
    RewardID            INT             NULL,        -- FK → Rewards (nullable — earn
txns have none)
    InvoiceID           INT             NULL,        -- FK → Invoices (nullable — redeem
txns have none)
    Notes               NVARCHAR(200)   NULL,

    CONSTRAINT PK_LoyaltyTransactions PRIMARY KEY (TransactionID),
    CONSTRAINT FK_LoyaltyTransactions_LoyaltyAccounts
        FOREIGN KEY (AccountID) REFERENCES Sales.LoyaltyAccounts (AccountID),
    CONSTRAINT FK_LoyaltyTransactions_LoyaltyRewards
        FOREIGN KEY (RewardID) REFERENCES Sales.LoyaltyRewards (RewardID),
    CONSTRAINT FK_LoyaltyTransactions_Invoices
        FOREIGN KEY (InvoiceID) REFERENCES Sales.Invoices (InvoiceID),
    CONSTRAINT CK_LoyaltyTransactions_TransactionType
        CHECK (TransactionType IN ('EARN', 'REDEEM')),
    CONSTRAINT CK_LoyaltyTransactions_PointsDirection
        CHECK (
            (TransactionType = 'EARN'   AND PointsAmount > 0) OR
            (TransactionType = 'REDEEM' AND PointsAmount < 0)
```

```
        )
);
GO
```

The CHECK constraint on `PointsAmount` encodes a business rule: earn transactions must have positive points, redeem transactions must have negative points. This level of constraint design is covered in Chapter 6.

---

13. Verifying the Mapping

After completing the DDL, verify that every element of the ERD has been accounted for.

13.1 The Mapping Verification Checklist

```
☐ Every entity in the ERD has a corresponding table
☐ Every attribute in the ERD is a column in the appropriate table
☐ Every derived attribute is either a computed column or intentionally omitted
☐ Every multi-valued attribute has a separate table
☐ Every 1:N relationship has a FK on the "many" table
☐ Every 1:1 relationship has a FK with a UNIQUE constraint
☐ Every M:N relationship has a junction table
☐ Every weak entity has its ownership relationship enforced by FK + composite PK or
UNIQUE
☐ Every mandatory participation rule is enforced by NOT NULL on the FK
☐ Every optional participation is modeled as a nullable FK where appropriate
☐ Every constraint is explicitly named
```

13.2 Tracing Back to the ERD

For each table in the physical model, identify its ERD origin:

Physical table	ERD origin	Rule applied
`Sales.LoyaltyTiers`	`TIER` entity	Rule 1 (strong entity)
`Sales.LoyaltyRewards`	`REWARD` entity	Rule 1 (strong entity)
`Sales.LoyaltyAccounts`	`LOYALTY_ACCOUNT` entity + 1:1 relationship	Rules 1 + 6
`Sales.LoyaltyTransactions`	`LOYALTY_TRANSACTION` entity + 1:N relationships	Rules 4 + 5

No ERD element was skipped. No physical table exists without an ERD origin. The mapping is complete.

13.3 Querying the Schema to Verify

After creating the tables, use the system catalog to confirm the structure:

```
-- Verify all Loyalty tables exist
SELECT TABLE_SCHEMA, TABLE_NAME
FROM   INFORMATION_SCHEMA.TABLES
WHERE  TABLE_NAME LIKE 'Loyalty%'
ORDER BY TABLE_SCHEMA, TABLE_NAME;

-- Verify all FK relationships are in place
SELECT
    fk.name                          AS RelationshipName,
    SCHEMA_NAME(tp.schema_id) + '.' + tp.name AS ParentTable,
    cp.name                          AS ParentColumn,
    SCHEMA_NAME(tr.schema_id) + '.' + tr.name AS ReferencedTable,
    cr.name                          AS ReferencedColumn
FROM    sys.foreign_keys fk
INNER JOIN sys.foreign_key_columns fkc ON fkc.constraint_object_id = fk.object_id
INNER JOIN sys.tables tp    ON tp.object_id = fkc.parent_object_id
INNER JOIN sys.tables tr    ON tr.object_id = fkc.referenced_object_id
INNER JOIN sys.columns cp   ON cp.object_id = fkc.parent_object_id
                           AND cp.column_id  = fkc.parent_column_id
INNER JOIN sys.columns cr   ON cr.object_id = fkc.referenced_object_id
                           AND cr.column_id  = fkc.referenced_column_id
WHERE   tp.name LIKE 'Loyalty%'
ORDER BY tp.name, fk.name;
```

---

14. Chapter Summary

- The ERD-to-table mapping is **systematic and deterministic** — each ERD element maps to a physical object by a defined rule. Given the same ERD, two designers applying the same rules produce the same schema.
- **Rule 1:** Strong entities → tables. **Rule 2:** Simple attributes → columns. **Rule 3:** Multi-valued attributes → new tables with FK to parent. **Rule 4:** Weak entities → tables with composite PK or FK-assisted identity.
- **Rule 5 (1:N):** FK on the "many" side. Mandatory participation → NOT NULL FK. Optional participation → nullable FK.
- **Rule 6 (1:1):** FK on the optional/extension side, with a `UNIQUE` constraint to enforce the maximum cardinality. Without the UNIQUE constraint, a FK alone allows 1:N.

- **Rule 7 (M:N):** Junction table with FKs to both entities, composite PK, and optional surrogate key + UNIQUE constraint.
- **Supertype/subtype mapping:** Three options — table per subtype (most normalized), single table with NULLs (simplest queries), table per concrete class (no joins but duplication). CabotTrail uses table per subtype.
- **Data type selection** follows a checklist: domain, range, length, precision needs, sort behaviour, and nullability.
- **Naming conventions** — plural tables, PascalCase, descriptive names, explicit constraint names following `PK_/FK_/UQ_/CK_/DF_` patterns — make the schema self-documenting.
- **Verify the mapping** by tracing every ERD element to a physical object and every physical object back to an ERD element. Use system catalog queries to confirm the physical structure matches the design.

---

15. Review Questions

1. The mapping rules state that in a 1:N relationship, the FK goes on the "many" side. Explain *why* this is the rule — what would happen structurally if you put the FK on the "one" side instead? Use the Customer–Order relationship as your example.
2. A developer creates the following DDL for a 1:1 relationship between `Employee` and `ParkingPermit`:

   ```
   CREATE TABLE HR.ParkingPermits (
       PermitID   INT NOT NULL IDENTITY(1,1) PRIMARY KEY,
       EmployeeID INT NOT NULL,
       PermitCode NVARCHAR(20) NOT NULL,
       FOREIGN KEY (EmployeeID) REFERENCES Application.Employees (EmployeeID)
   );
   ```

   Identify the missing design element that should enforce the 1:1 maximum cardinality, explain what is wrong without it, and write the corrected DDL.
3. Write the complete DDL (applying all mapping rules with named constraints) for the following ERD narrative:

> *A **Warehouse** has a name, a city, and a capacity in cubic metres. A warehouse stores many **ProductBatches**. Each batch belongs to exactly one warehouse, has a batch number (unique within a warehouse), a product ID, a quantity, and a received date. A product can have many batches across multiple warehouses.*

4. The `LoyaltyTransaction` table in section 12.3 has two nullable FKs: `RewardID` and `InvoiceID`. Explain the business rules these nullable FKs represent. Now write a CHECK constraint that enforces: "An EARN transaction must reference an InvoiceID, and a REDEEM transaction must reference a RewardID."

5. For the supertype/subtype relationship between `People`, `Employees`, and `Customers`, compare Option 1 (table per subtype) and Option 2 (single table with NULLs). Write a specific query against each design that retrieves an employee's full name, email address, and job title — and compare the complexity of each version.

6. A business analyst tells you: "We need to store the delivery address for each order — street, city, province, and postal code." Describe two different ways to model this in an ERD and their corresponding physical implementations. What business question determines which approach is correct?

7. Convert the following unnormalised description to an ERD and then to DDL, applying all mapping rules:

> *"Each **sales rep** has an ID, a name, and an email. A sales rep manages many **accounts**. Each account has an account number, a company name, and a credit limit. An account can have many **contacts** — people at the company — each with a name, a phone number, and a primary contact flag."*

8. Explain the relationship between a well-named FK constraint and the ability to maintain a production database. Give a specific example of a maintenance task that becomes easier when FK constraints are named explicitly following the `FK_TableName_ReferencedTable` pattern.

🔍 Deeper Dive

Going Further with the Mapping Process

The Formal Relational Schema Notation

Academic database texts express a relational schema in formal notation rather than DDL. The notation uses an underline for primary key attributes and an asterisk (or italic) for foreign keys:

```
Customers(<u>CustomerID</u>, CustomerName, CreditLimit, AccountOpenedDate)
Orders(<u>OrderID</u>, *CustomerID, OrderDate, ExpectedDeliveryDate)
OrderLines(<u>OrderLineID</u>, *OrderID, *ProductID, Quantity, PickedQuantity,
UnitPrice)
```

The underlined attributes are the primary key. The asterisked attributes are foreign keys. This notation is useful for quickly communicating a schema without full DDL — for whiteboard discussions, exam answers, and academic papers.

Understanding the formal notation helps when reading database textbooks (Connolly & Begg, Elmasri & Navathe) which use it throughout.

The Chen-to-Relational Mapping Algorithm

The formal mapping algorithm from ER model to relational schema is described in Elmasri & Navathe as a 9-step procedure:

1. Map regular (strong) entity types
2. Map weak entity types
3. Map binary 1:1 relationship types
4. Map binary 1:N relationship types
5. Map binary M:N relationship types
6. Map multivalued attributes
7. Map N-ary relationship types (three or more entities)
8. Map specialisation/generalisation (supertypes/subtypes)
9. Map union types (categories — a single entity that can be one of several types)

This chapter covers steps 1–8 in the context of the Crow's Foot notation. N-ary relationships (step 7) and union types (step 9) are less common in business systems but are covered in Elmasri & Navathe Chapter 9.

Referential Integrity Actions in Depth

Chapter 1 introduced `NO ACTION`, `CASCADE`, `SET NULL`, and `SET DEFAULT` as referential integrity actions on FK constraints. These deserve more depth:

```
-- ON DELETE CASCADE: deleting a Customer deletes all their Orders automatically
CONSTRAINT FK_Orders_Customer
    FOREIGN KEY (CustomerID) REFERENCES Sales.Customers (CustomerID)
    ON DELETE CASCADE

-- ON DELETE SET NULL: deleting a SalesRep sets the FK to NULL (order keeps record)
CONSTRAINT FK_Orders_SalesRep
    FOREIGN KEY (SalesRepID) REFERENCES Application.Employees (EmployeeID)
    ON DELETE SET NULL

-- ON UPDATE CASCADE: if CustomerID changes, propagate to all Orders
CONSTRAINT FK_Orders_Customer
    FOREIGN KEY (CustomerID) REFERENCES Sales.Customers (CustomerID)
    ON UPDATE CASCADE
```

Cascade delete is powerful but dangerous — a single DELETE can remove thousands of rows across multiple tables. Use it only when child rows are truly meaningless without the parent (e.g., invoice lines without an invoice).

Set NULL on delete is appropriate when the referenced entity is optional — an order placed by a sales rep who has since left the company should not be deleted; the FK should become NULL.

No action (default) prevents deletion of referenced rows — safest default. Requires the application to handle the deletion of dependents before deleting the parent.

The choice of referential action is a **business rule**: what does the business want to happen when a customer is deleted? This question must be answered by the business, not decided unilaterally by the designer.

Microsoft documentation: FOREIGN KEY Constraints

References and Further Reading

1. Connolly, T., & Begg, C. (2015). *Database Systems: A Practical Approach to Design, Implementation, and Management* (6th ed.). Pearson. — Chapter 16 covers the ER-to-relational mapping with comprehensive worked examples.

2. Elmasri, R., & Navathe, S. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson. — Chapter 9 covers the formal 9-step ER-to-relational mapping algorithm.

3. Date, C. J. (2019). *Database Design and Relational Theory: Normal Forms and All That Jazz* (2nd ed.). Apress. — Chapter 2 covers the mapping from logical to physical model from a rigorous theoretical perspective.

4. Microsoft. (2024). *FOREIGN KEY Constraints*. https://learn.microsoft.com/en-us/sql/relational-databases/tables/primary-and-foreign-key-constraints

5. Microsoft. (2024). *Primary and Unique Key Constraints*. https://learn.microsoft.com/en-us/sql/relational-databases/tables/unique-constraints-and-check-constraints

6. Microsoft. (2024). *Computed Columns*. https://learn.microsoft.com/en-us/sql/relational-databases/tables/specify-computed-columns-in-a-table

---

> ***Database Design for Analysts*** *| © Patrick Dolinger, NSCC Institute of Technology CC BY 4.0 — Share and adapt freely with attribution*

Chapter 4: Normalization — 1NF and 2NF: Removing Redundancy

Database Design for Analysts *From business requirements to relational structures*

© Patrick Dolinger, NSCC Institute of Technology Licensed under Creative Commons Attribution 4.0 International (CC BY 4.0) You are free to share and adapt this material for any purpose, provided appropriate credit is given.

Chapter Overview

Chapter 1 introduced the three anomalies — update, insertion, and deletion — that arise from bad database design. Chapters 2 and 3 showed how to design and map a database correctly from the ground up. This chapter and the next address what happens when a database is *not* designed from scratch: how to take an existing flat, redundant structure and systematically improve it.

Normalization is the formal process of restructuring a relational schema to reduce data redundancy and eliminate the conditions that cause anomalies. It proceeds through a series of **normal forms** — increasingly strict structural requirements. Each normal form eliminates a specific class of problem. A table that satisfies a normal form is said to be "in" that form.

This chapter covers the first two normal forms: **First Normal Form (1NF)** and **Second Normal Form (2NF)**. Together, they eliminate the two most fundamental categories of structural problems: multi-valued cells and partial key dependencies.

By the end of this chapter you will be able to:

- Explain what functional dependency means and why it is the foundation of normalization
- Define First Normal Form and identify violations in any table
- Normalise a table to 1NF by removing multi-valued cells and repeating groups
- Define Second Normal Form and identify partial dependencies
- Normalise a table to 2NF by decomposing to remove partial dependencies
- Explain the preservation of data and lossless joins after normalization
- Apply normalization to realistic, messy source data from the CabotTrail context

Table of Contents

1. What Normalization Is and Why It Matters
2. Functional Dependency: The Foundation
3. The Unnormalised Form: Starting Point
4. First Normal Form (1NF)
5. Identifying 1NF Violations
6. Normalising to 1NF: Worked Examples
7. Second Normal Form (2NF)
8. Functional Dependency Diagrams
9. Identifying Partial Dependencies
10. Normalising to 2NF: Worked Examples
11. Lossless Decomposition
12. CabotTrail in 1NF and 2NF
13. Chapter Summary
14. Review Questions
15. 🔍 Deeper Dive

---

1. What Normalization Is and Why It Matters

1.1 The Problem Normalization Solves

Imagine you receive a spreadsheet from a colleague representing CabotTrail's sales data. It has been assembled by combining information from several sources, and it looks like this:

```
OrderID | CustomerName | CustomerCity | ProductCodes          |
ProductNames              | Quantities
1001    | Fundy Bay    | Halifax      | CT-083, CT-101        | Sleeping Bag,
Tent        | 2, 1
1002    | Trail Hut    | Truro        | CT-045                | Hiking
Boots             | 3
1003    | Fundy Bay    | Halifax      | CT-083, CT-200, CT-210| Sleeping Bag, Pad,
Stove  | 1, 2, 1
```

This spreadsheet has several structural problems:

- `ProductCodes`, `ProductNames`, and `Quantities` contain comma-separated lists — multiple values in a single cell
- `CustomerName` and `CustomerCity` are repeated on every order from the same customer
- If Halifax's spelling changes, multiple rows must be updated
- You cannot query "how many units of CT-083 were sold" without parsing comma-separated strings

These problems do not arise from bad data — they arise from bad **structure**. Normalization provides a systematic method for identifying and fixing structural problems.

1.2 The Normal Form Levels

Normal forms are numbered: 1NF, 2NF, 3NF, BCNF, 4NF, 5NF. Each higher form satisfies all lower forms plus an additional requirement. In practice, most business databases aim for **Third Normal Form (3NF)** — which eliminates the three main categories of redundancy anomalies. 4NF and 5NF address more exotic cases covered in Chapter 5's Deeper Dive.

```
UNF (Unnormalised Form)
  ↓  Eliminate multi-valued attributes and repeating groups
1NF (First Normal Form)
  ↓  Eliminate partial dependencies
2NF (Second Normal Form)
  ↓  Eliminate transitive dependencies
3NF (Third Normal Form)
  ↓  Eliminate all remaining non-trivial dependencies on non-candidate keys
BCNF (Boyce-Codd Normal Form)
```

1.3 Normalization Is Analysis, Not Redesign

Normalization does not change the *information* in the database — it reorganises how that information is stored. After normalization, the same facts can be retrieved (by joining the normalised tables). The data is preserved; the redundancy is removed.

This is the **lossless join** property: a properly normalised decomposition can be reconstructed to the original by joining the parts. Section 11 demonstrates this.

---

2. Functional Dependency: The Foundation

Before any normal form can be defined, a concept must be established: **functional dependency**. It is the mathematical backbone of normalization theory.

2.1 The Definition

A column B is **functionally dependent** on column A (written `A → B`, read "A determines B") if, for every possible value of A, there is exactly one corresponding value of B.

Knowing A tells you B without ambiguity.

```
CustomerID → CustomerName
```

If you know `CustomerID = 42`, you know exactly one `CustomerName`: 'Fundy Bay Outfitters'. No two customers have the same `CustomerID` with different names. So `CustomerName` is functionally dependent on `CustomerID`.

```
CustomerID → CustomerCity
```

`CustomerID = 42` always maps to the same city: 'Halifax'. So `CustomerCity` is functionally dependent on `CustomerID`.

2.2 Counter-Example: Not a Functional Dependency

```
CustomerCity → CustomerName   (NOT a functional dependency)
```

Knowing the city 'Halifax' does not tell you a single customer name — there may be many customers in Halifax. Multiple customers share the same city. So `CustomerName` is NOT functionally dependent on `CustomerCity`.

2.3 Full vs Partial Functional Dependency

When the primary key is **composite** (two or more columns), a non-key attribute may depend on:

- **The whole key** — a **full functional dependency**
- **Only part of the key** — a **partial functional dependency**

This distinction is the basis of Second Normal Form. A table that has attributes depending on only part of a composite key is in 1NF but not 2NF.

2.4 Notation Summary

```
A → B          A functionally determines B (B is fully dependent on A)
AB → C         The combination of A and B determines C
A → B, A → C   A determines both B and C
```

When writing functional dependencies, the left side is the **determinant** and the right side is the **dependent**.

---

3. The Unnormalised Form: Starting Point

The **Unnormalised Form (UNF)** is a table with no formal structure requirements — it may have multi-valued cells, repeating groups, or no defined primary key. This is the starting point for normalization.

3.1 A Realistic UNF Example

The sales spreadsheet from section 1.1 is typical UNF data. Here it is expressed as a SQL table (though SQL Server would struggle to store it correctly because of the multi-valued cells):

```
SALES_FLAT (unnormalised)
OrderID | CustomerID | CustomerName  | CustomerCity | Products (repeating group)
        |            |               |              | ProductID | ProductName | Qty |
UnitPrice
1001    | 42         | Fundy Bay     | Halifax      | 83        | Sleeping Bag| 2   |
249.99
        |            |               |              | 101       | Tent        | 1   |
649.99
1002    | 17         | Trail Hut     | Truro        | 45        | Hiking Boots| 3   |
159.99
1003    | 42         | Fundy Bay     | Halifax      | 83        | Sleeping Bag| 1   |
249.99
        |            |               |              | 200       | Sleeping Pad| 2   |
89.99
        |            |               |              | 210       | Camp Stove  | 1   |
119.99
```

Problems visible immediately:

- Each order row appears to span multiple sub-rows (the repeating product group)
- `CustomerName` and `CustomerCity` repeat for customer 42 across orders 1001 and 1003
- No clear primary key (OrderID alone doesn't identify a row if there are multiple product lines)

---

4. First Normal Form (1NF)

A table is in **First Normal Form** if and only if:

1. Every column contains **atomic (single-valued)** values — no lists, arrays, or repeating groups
2. Every column contains values from the **same domain** (consistent data type)
3. Every row is **unique** — there is a defined primary key
4. The **order of rows** is irrelevant (no row ordering dependency)

Condition 1 is the critical one in practice. Conditions 2–4 are properties of the relational model itself (Chapter 1); a properly defined SQL table satisfies them automatically with data types and PRIMARY KEY constraints.

4.1 The Atomicity Requirement

"Atomic" means indivisible — a single value that cannot be meaningfully subdivided for query purposes.

Not atomic:

- `'CT-083, CT-101'` in a ProductCodes column (a list of codes)
- `'Halifax, NS'` in a Location column (city and province combined)
- `'2, 1'` in a Quantities column (a list of quantities)

Atomic:

- `'CT-083'` — a single product code
- `'Halifax'` — a single city name
- `2` — a single quantity

The test for atomicity: does the business need to query, filter, or operate on individual components of this value? If yes, it must be split into separate columns or a separate table.

---

5. Identifying 1NF Violations

Three patterns consistently signal 1NF violations.

5.1 Pattern 1: Comma-Separated or Delimited Values

Any column storing multiple values separated by a delimiter violates 1NF.

```
-- 1NF violation: ProductCodes contains multiple values
ProductCodes = 'CT-083, CT-101, CT-200'

-- How would you query "all orders containing product CT-101"?
WHERE ProductCodes = 'CT-101'                  -- Misses 'CT-083, CT-101'
WHERE ProductCodes LIKE '%CT-101%'             -- Works but is fragile and non-sargable
```

The LIKE workaround is a common band-aid. It signals a structural problem: if you need LIKE to extract individual values from a column, the column is not atomic.

5.2 Pattern 2: Repeating Column Groups

A table with columns like `Product1`, `Product2`, `Product3` is structurally equivalent to a multi-valued attribute — just expressed as multiple columns rather than a list:

```
OrderID | Product1ID | Product1Name | Product1Qty | Product2ID | Product2Name |
Product2Qty
1001    | 83         | Sleeping Bag | 2           | 101        | Tent         | 1
1002    | 45         | Hiking Boots | 3           | NULL       | NULL         | NULL
```

Problems:

- What if order 1003 has 5 products? The table would need `Product5ID`, `Product5Name`, `Product5Qty`
- NULLs represent non-applicable values — structurally messy
- Queries aggregating across all product columns are complex and change as columns are added

5.3 Pattern 3: No Primary Key

A table without a primary key cannot guarantee unique rows — it violates the uniqueness requirement of 1NF. Even if rows happen to be unique, there is no enforcement mechanism. This is a 1NF violation.

---

6. Normalising to 1NF: Worked Examples

6.1 Converting Comma-Separated Values

Starting table (1NF violation — comma-separated products):

```
ORDERS_FLAT
OrderID | CustomerID | ProductCodes     | ProductNames              | Quantities
```

```
1001    | 42         | CT-083, CT-101   | Sleeping Bag, Tent         | 2, 1
1002    | 17         | CT-045           | Hiking Boots               | 3
```

Step 1 — Identify the repeating group: `ProductCode`, `ProductName`, `Quantity` repeat per order.

Step 2 — Move the repeating group to a new table, with a FK back to the original table and a PK on the new combination:

```
-- After 1NF: split into two tables
-- Table 1: Order-level information (no repeating attributes)
CREATE TABLE Orders_1NF
(
    OrderID     INT NOT NULL,
    CustomerID  INT NOT NULL,
    CONSTRAINT PK_Orders_1NF PRIMARY KEY (OrderID)
);

-- Table 2: Order line items (the repeating group becomes rows)
CREATE TABLE OrderLines_1NF
(
    OrderID     INT NOT NULL,
    ProductCode NVARCHAR(20) NOT NULL,
    ProductName NVARCHAR(100) NOT NULL,
    Quantity    INT NOT NULL,
    CONSTRAINT PK_OrderLines_1NF PRIMARY KEY (OrderID, ProductCode),
    CONSTRAINT FK_OrderLines_Order FOREIGN KEY (OrderID) REFERENCES Orders_1NF (OrderID)
);
```

The comma-separated list is gone. Each product on each order is now its own row. Order 1001 produces two rows in `OrderLines_1NF` (one for CT-083, one for CT-101).

6.2 Converting Repeating Column Groups

Starting table (1NF violation — repeating column groups):

```
ORDERS_COLUMNS
OrderID | CustomerID | Prod1Code | Prod1Qty | Prod2Code | Prod2Qty | Prod3Code |
Prod3Qty
1001    | 42         | CT-083    | 2        | CT-101    | 1        | NULL      | NULL
1002    | 17         | CT-045    | 3        | NULL      | NULL     | NULL      | NULL
```

Step 1 — Identify the repeating column group: `ProdNCode`, `ProdNQty` (where N = 1, 2, 3).

Step 2 — Same resolution: move to a new table, one row per (order, product) pair:

```
-- Same solution as comma-separated values: separate table for line items
CREATE TABLE OrderLines_1NF
(
    OrderID     INT             NOT NULL,
```

```
    ProductCode NVARCHAR(20)    NOT NULL,
    Quantity    INT             NOT NULL,
    CONSTRAINT PK_OrderLines_1NF PRIMARY KEY (OrderID, ProductCode),
    CONSTRAINT FK_OL_Order FOREIGN KEY (OrderID) REFERENCES Orders_1NF (OrderID)
);
```

The NULL-filled repeating columns become absent rows — clean and extensible.

6.3 Inserting the 1NF Data

The data transforms from the flat structure to the normalised rows:

```
-- Original flat row: Order 1001 with products CT-083 (qty 2) and CT-101 (qty 1)
-- Becomes:
INSERT INTO Orders_1NF VALUES (1001, 42);

INSERT INTO OrderLines_1NF VALUES
    (1001, 'CT-083', 2),
    (1001, 'CT-101', 1);

-- Original flat row: Order 1002 with one product CT-045 (qty 3)
INSERT INTO Orders_1NF VALUES (1002, 17);

INSERT INTO OrderLines_1NF VALUES
    (1002, 'CT-045', 3);
```

No information is lost. The original data can be reconstructed with a JOIN:

```
SELECT o.OrderID, o.CustomerID, ol.ProductCode, ol.Quantity
FROM   Orders_1NF o
INNER JOIN OrderLines_1NF ol ON ol.OrderID = o.OrderID;
```

---

7. Second Normal Form (2NF)

A table is in **Second Normal Form** if and only if:

1. It is in **First Normal Form**
2. Every non-key attribute is **fully functionally dependent on the entire primary key** — no partial dependencies

2NF is only relevant when the primary key is **composite** — a key made of two or more columns. If a table has a single-column primary key, it is automatically in 2NF (there is no "part of the key" to partially depend on).

7.1 Partial Dependency: The Violation

A **partial dependency** occurs when a non-key attribute is functionally dependent on *only part* of a composite key, rather than the whole key.

After 1NF, the `OrderLines_1NF` table has:

```
OrderLines_1NF (in 1NF)
PK: (OrderID, ProductCode)

OrderID | ProductCode | Quantity | ProductName         | UnitPrice
1001    | CT-083      | 2        | Sleeping Bag        | 249.99
1001    | CT-101      | 1        | Tent                | 649.99
1002    | CT-045      | 3        | Hiking Boots        | 159.99
1003    | CT-083      | 1        | Sleeping Bag        | 249.99
```

Check each non-key attribute:

- `Quantity` — depends on `(OrderID, ProductCode)` together: how many of CT-083 were on order 1001 is a fact about the specific combination. ✓ **Full dependency**
- `ProductName` — depends on `ProductCode` alone: 'CT-083' is always 'Sleeping Bag' regardless of which order it appears on. ✗ **Partial dependency** on just `ProductCode`
- `UnitPrice` — depends on `ProductCode` alone: the price of CT-083 is always $249.99 regardless of the order. ✗ **Partial dependency** on just `ProductCode`

`ProductName` and `UnitPrice` partially depend on `ProductCode` — only part of the composite PK. This violates 2NF.

7.2 Why Partial Dependencies Cause Anomalies

Update anomaly: If the price of CT-083 changes to $259.99, every row with `ProductCode = 'CT-083'` must be updated. Missing any one row means inconsistent prices across order lines.

Insertion anomaly: You cannot record information about a new product (its name and price) until it appears on at least one order. The product's name and price are facts about the product, not about any specific order line — but they can only be stored once an order line exists.

Deletion anomaly: If the only order containing CT-045 (Hiking Boots) is deleted, the fact that CT-045 is called 'Hiking Boots' and costs $159.99 is also deleted.

---

8. Functional Dependency Diagrams

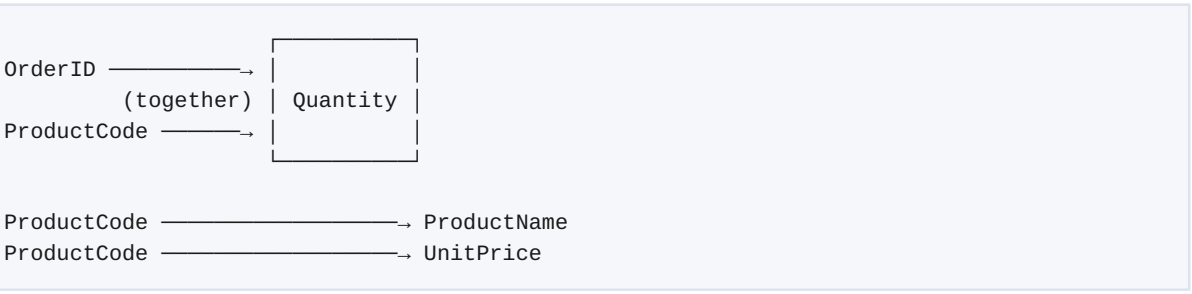
A **functional dependency diagram** (FD diagram) visualises all the functional dependencies in a table, making partial and transitive dependencies visible.

8.1 Drawing an FD Diagram

Conventions:

- Draw all columns as named boxes
- Draw arrows from determinant to dependent
- Show the primary key (or composite key components) clearly
- Identify partial dependencies (arrows from part of the PK) vs full dependencies (arrows from the whole PK)

For `OrderLines_1NF`:



The arrows from `ProductCode` alone (without `OrderID`) to `ProductName` and `UnitPrice` are the partial dependencies. They violate 2NF.

8.2 Identifying Full vs Partial Dependencies

The systematic check: for each non-key attribute, ask "Could this attribute's value change if I changed only one component of the composite key, while keeping the other constant?"

- Can `Quantity` change if I change `OrderID` while keeping `ProductCode` fixed? Yes — a different order for the same product could have a different quantity. So `Quantity` needs **both** key components. ✓ Full dependency.
- Can `ProductName` change if I change `OrderID` while keeping `ProductCode` fixed? No — the product name depends only on the product, not which order it's on. ✗ Partial dependency.

---

9. Identifying Partial Dependencies

9.1 The Standard Partial Dependency Pattern

Partial dependencies always follow the same pattern:

1. There is a composite PK: `(A, B)`
2. Some non-key attribute C depends only on A (or only on B)
3. C is a fact about A (or B) alone, not about the combination (A, B)

9.2 Practical Questions to Ask

When examining a table with a composite PK for 2NF violations, ask for each non-key column:

"Is this a fact about [A], [B], or [A and B together]?"

If the answer is "[A] alone" or "[B] alone" → partial dependency → 2NF violation. If the answer is "[A and B together]" → full dependency → no violation.

For `OrderLines_1NF`:

- `Quantity`: "Is this a fact about the order, the product, or the specific combination of order and product?" → The combination. ✓
- `ProductName`: "Is this a fact about the order, the product, or the combination?" → The product alone. ✗
- `UnitPrice`: "Is this a fact about the order, the product, or the combination?" → The product alone (assuming catalogue prices). ✗

9.3 A Subtler Case: Price at Time of Order

Is `UnitPrice` truly a fact about the product alone? Only if prices never change, or if the business does not need to know what price was charged on a specific order.

In reality, prices change. CabotTrail's invoices record the `UnitPrice` at the time of the sale — which may differ from today's catalogue price. So for historical invoice data, `UnitPrice` IS a fact about the specific order line (the combination), not just the product.

This is the key insight: **the correct functional dependencies depend on business rules, not just on the data that currently exists in the table.** If the business rule is "record the price at time of sale," then `UnitPrice` on `InvoiceLines` is a full dependency (on the line, which identifies the specific sale). If the business rule is "always use the current catalogue price," then `UnitPrice` is a partial dependency on the product.

The CabotTrail OLTP records `UnitPrice` on both `OrderLines` and `InvoiceLines` — correctly treating it as a fact about the specific transaction, not about the product.

---

10. Normalising to 2NF: Worked Examples

10.1 The Decomposition Step

To remove a partial dependency:

1. Create a **new table** for the partially dependent attributes, with the partial determinant as its PK
2. **Remove the partially dependent attributes** from the original table
3. **Keep the partial determinant column** in the original table as a FK to the new table

```
Before 2NF:
OrderLines_1NF (OrderID, ProductCode, Quantity, ProductName, UnitPrice)
PK: (OrderID, ProductCode)

Partial dependencies: ProductCode → ProductName, UnitPrice
```

```
After 2NF decomposition:
Table 1: Products_2NF (ProductCode, ProductName, UnitPrice)
         PK: ProductCode
         (Captures the facts about products)

Table 2: OrderLines_2NF (OrderID, ProductCode, Quantity)
         PK: (OrderID, ProductCode)
         FK: ProductCode → Products_2NF.ProductCode
         (Captures the facts about specific order lines)
```

```
-- 2NF: Products table (facts about products)
CREATE TABLE Products_2NF
(
    ProductCode     NVARCHAR(20)    NOT NULL,
    ProductName     NVARCHAR(100)   NOT NULL,
    UnitPrice       DECIMAL(18,2)   NOT NULL,

    CONSTRAINT PK_Products_2NF PRIMARY KEY (ProductCode)
```

```
);

-- 2NF: OrderLines table (facts about specific order lines)
CREATE TABLE OrderLines_2NF
(
    OrderID         INT             NOT NULL,
    ProductCode     NVARCHAR(20)    NOT NULL,
    Quantity        INT             NOT NULL,

    CONSTRAINT PK_OrderLines_2NF PRIMARY KEY (OrderID, ProductCode),
    CONSTRAINT FK_OL_Product FOREIGN KEY (ProductCode)
        REFERENCES Products_2NF (ProductCode),
    CONSTRAINT FK_OL_Order   FOREIGN KEY (OrderID)
        REFERENCES Orders_1NF (OrderID)
);
```

10.2 Populating the 2NF Tables

```
-- Products from the flat data
INSERT INTO Products_2NF VALUES
    ('CT-083', 'Sleeping Bag', 249.99),
    ('CT-101', 'Tent', 649.99),
    ('CT-045', 'Hiking Boots', 159.99);

-- Order lines (quantity only — product details are in Products_2NF)
INSERT INTO OrderLines_2NF VALUES
    (1001, 'CT-083', 2),
    (1001, 'CT-101', 1),
    (1002, 'CT-045', 3),
    (1003, 'CT-083', 1);
```

10.3 The 2NF Benefits Demonstrated

Update anomaly eliminated: To change CT-083's price, update one row in `Products_2NF`. All order lines referencing CT-083 automatically see the new price through the FK.

Insertion anomaly eliminated: A new product can be added to `Products_2NF` before it appears on any order line.

Deletion anomaly eliminated: Deleting all order lines containing CT-045 does not delete 'Hiking Boots' from `Products_2NF`.

10.4 A Second 2NF Example: Customer Data

The original flat `ORDERS_FLAT` also has a partial dependency if it has a composite PK:

```
ORDERS_FLAT (OrderID, CustomerID, CustomerName, CustomerCity, OrderDate)
```

```
If PK is OrderID alone: no partial dependency issue
If PK were somehow (OrderID, CustomerID): CustomerName and CustomerCity
  depend on CustomerID alone → partial dependency
```

In practice, `OrderID` is sufficient as the PK for orders. The issue with `CustomerName` and `CustomerCity` is a different problem — **transitive dependency** — which is the subject of 3NF (Chapter 5).

---

11. Lossless Decomposition

Every normalization step must be **lossless** — you must be able to reconstruct the original data by joining the decomposed tables. If the join produces more or fewer rows than the original, the decomposition introduced errors.

11.1 The Lossless Join Test

```
-- Original flat data (before normalization):
-- OrderID 1001 had CT-083 (qty 2) and CT-101 (qty 1)
-- OrderID 1002 had CT-045 (qty 3)

-- After 2NF decomposition, JOIN to reconstruct:
SELECT
    ol.OrderID,
    ol.ProductCode,
    p.ProductName,
    ol.Quantity,
    p.UnitPrice
FROM    OrderLines_2NF ol
INNER JOIN Products_2NF p ON p.ProductCode = ol.ProductCode
ORDER BY ol.OrderID, ol.ProductCode;
```

The result should match the original flat data exactly — same rows, same values. No rows added, no rows lost.

11.2 What Makes a Decomposition Lossless

A decomposition is lossless if the **join column is the primary key of one of the resulting tables**. In the 2NF decomposition:

- `OrderLines_2NF` and `Products_2NF` are joined on `ProductCode`
- `ProductCode` is the primary key of `Products_2NF`
- Therefore the join is lossless — each order line's `ProductCode` maps to exactly one product row

If the join column were not the PK of either table, the join could produce spurious (extra, incorrect) rows — a lossy decomposition.

11.3 Dependency Preservation

Along with lossless joins, a good normalization preserves **dependencies** — after decomposition, every functional dependency from the original table can still be enforced by a constraint in the new tables.

In the 2NF decomposition: `ProductCode → ProductName` is enforced by the UNIQUE constraint on `ProductCode` in `Products_2NF`. The dependency is preserved in the new table.

---

12. CabotTrail in 1NF and 2NF

12.1 Is the CabotTrail OLTP Already Normalised?

Yes — the CabotTrail OLTP was designed with normalization in mind. Examining it confirms this and reinforces the principles.

1NF compliance:

```
-- Check: are there any multi-valued columns?
-- Query for columns of XML, JSON, or text types that might store lists
SELECT TABLE_SCHEMA, TABLE_NAME, COLUMN_NAME, DATA_TYPE
FROM   INFORMATION_SCHEMA.COLUMNS
WHERE  DATA_TYPE IN ('xml', 'ntext', 'text')
AND    TABLE_CATALOG = 'CabotTrailOutdoor';
-- Expected: no results (or none storing lists)
```

Every column in `CabotTrailOutdoor` stores atomic values. There are no repeating groups, no comma-separated lists. **The OLTP is in 1NF.**

2NF compliance:

```
-- Check: does Sales.InvoiceLines have any partial dependencies?
-- PK: (InvoiceID, OrderLineID) — let's verify this is indeed the PK
SELECT
    c.name          AS ColumnName,
    c.is_nullable,
    CASE WHEN pk.column_id IS NOT NULL THEN 'PK' ELSE '' END AS KeyType
FROM    sys.tables t
INNER JOIN sys.schemas s    ON s.schema_id = t.schema_id
INNER JOIN sys.columns c    ON c.object_id = t.object_id
LEFT JOIN (
    SELECT ic.object_id, ic.column_id
```

```
    FROM   sys.index_columns ic
    INNER JOIN sys.indexes i ON i.object_id = ic.object_id
                            AND i.is_primary_key = 1
) pk ON pk.object_id = t.object_id AND pk.column_id = c.column_id
WHERE  s.name = 'Sales' AND t.name = 'InvoiceLines'
ORDER BY c.column_id;
```

For `Sales.InvoiceLines`, the non-key columns include `ProductID`, `Quantity`, `UnitPrice`, `LineTotal`, `TaxRate`, `TaxAmount`. Checking each:

- `ProductID` — depends on the invoice line (which product was on this line). ✓ Full dependency
- `Quantity` — depends on the invoice line (how many were invoiced). ✓ Full dependency
- `UnitPrice` — depends on the invoice line (price **at time of invoice**). ✓ Full dependency on the line

This is the key design decision: `UnitPrice` on `InvoiceLines` is the price charged for this specific invoice line — not the product's current catalogue price. It is a fact about the invoice line, not about the product. **This is what makes it a full dependency and not a 2NF violation.**

12.2 Where 2NF Would Be Violated

If CabotTrail had stored `ProductName` or `CategoryName` on `InvoiceLines` (as the UNF spreadsheet did), that would be a 2NF violation — those facts belong to the Product entity, not to any specific invoice line.

The absence of `ProductName` from `InvoiceLines` is not an oversight — it is correct design. When a report needs product names on invoice lines, the query joins `InvoiceLines` to `Products` (or in the DW, `FactSales` to `DimProduct`). The join is the mechanism that assembles normalised facts into readable output.

---

13. Chapter Summary

- **Normalization** eliminates data redundancy and anomalies (update, insertion, deletion) by restructuring tables according to formal rules.
- **Functional dependency** (A → B) means knowing A uniquely determines B. It is the mathematical foundation of every normal form.
- **First Normal Form (1NF)** requires: atomic (single-valued) cells, consistent column types, unique rows (primary key), and irrelevant row order. Violations include comma-separated values, repeating column groups, and missing primary keys.

- To normalise to 1NF: move repeating groups to a new table with a FK back to the original table. One row per atomic fact.
- **Second Normal Form (2NF)** requires: being in 1NF **plus** every non-key attribute is fully functionally dependent on the entire composite primary key. Tables with a single-column PK are automatically in 2NF.
- A **partial dependency** is when a non-key attribute depends on only part of a composite key. It causes update, insertion, and deletion anomalies.
- To normalise to 2NF: move partially dependent attributes to a new table keyed by the partial determinant. Keep the partial determinant column as a FK in the original table.
- Every normalization decomposition must be **lossless** — the original data can be reconstructed by joining the decomposed tables. This is guaranteed when the join column is the PK of one of the tables.
- The CabotTrail OLTP is already in at least 2NF — `UnitPrice` on `InvoiceLines` is a full dependency on the invoice line (price at time of sale), not a partial dependency on the product.

---

14. Review Questions

1. A table `COURSE_REGISTRATION` has columns: `(StudentID, CourseID, StudentName, CourseName, Grade, InstructorName)` with PK `(StudentID, CourseID)`. Identify all functional dependencies, classify each as full or partial, and explain which ones violate 2NF.
2. The following table stores supplier invoices in UNF:

   ```
   InvoiceNum | SupplierName | SupplierCity | Items (repeating)
              |              |              | ProductCode | Description | Qty | Cost
   ```

   (a) Normalise to 1NF — write the resulting tables with PKs. (b) Identify any 2NF violations in your 1NF result. (c) Normalise to 2NF — write the final tables with PKs and FKs.
3. Explain why 2NF is only relevant when a table has a composite primary key. What happens if a table with a single-column surrogate key has a column that logically depends on only a "part" of the data? Does this violate 2NF? If not, which normal form addresses this?
4. The CabotTrail OLTP stores `UnitPrice` on `Sales.InvoiceLines`. Explain why this is **not** a 2NF violation, even though `UnitPrice` is also an attribute of the `Products` entity. What business rule makes the difference?

5. Write the lossless join test for the following 2NF decomposition. Show the query that reconstructs the original data and explain why it produces exactly the same rows as the original flat table:
   - Original: `ENROLMENT(StudentID, CourseID, StudentName, CourseName, Grade)`
   - After 2NF: `STUDENT(StudentID, StudentName)`, `COURSE(CourseID, CourseName)`, `ENROLMENT(StudentID, CourseID, Grade)`
6. A table `PRODUCT_SUPPLIER(ProductID, SupplierID, ProductName, SupplierName, LeadTimeDays)` has PK `(ProductID, SupplierID)`. Identify all functional dependencies. Draw the FD diagram. Which attributes violate 2NF, and what tables result from the 2NF decomposition?
7. Consider this business rule: "The unit price charged on an order line may differ from the catalogue price because of negotiated discounts." Does this change the functional dependency analysis for `UnitPrice` in `OrderLines`? Explain your reasoning.
8. After normalising to 2NF, a developer complains that queries now require a JOIN that was not needed before. "You've made the queries more complicated," they say. Construct a counter-argument that explains the benefits of 2NF using a specific example of an anomaly the join eliminates.

🔍 Deeper Dive

Going Further with Functional Dependencies and 1NF/2NF

Armstrong's Axioms

The rules for deriving functional dependencies from known ones are called **Armstrong's Axioms** (after William Ward Armstrong, who formalised them in 1974). They are the logical foundation for computing the closure of a set of functional dependencies:

Reflexivity: If B is a subset of A, then A → B. (Trivial: {StudentID, CourseID} → StudentID)

Augmentation: If A → B, then AC → BC for any C. (If you already know A determines B, adding C to both sides preserves the dependency)

Transitivity: If A → B and B → C, then A → C. (This is the basis of transitive dependencies — the subject of 3NF)

From these three axioms, two additional rules are derived:

Union: If A → B and A → C, then A → BC. **Decomposition:** If A → BC, then A → B and A → C.

Understanding Armstrong's Axioms explains *why* normalization decompositions work: the functional dependencies in the original table are preserved (through derivation) in the decomposed tables.

Armstrong, W. W. (1974). Dependency structures of data base relationships. *Proceedings of IFIP Congress*, 74, 580–583.

The Minimal Cover

For a given set of functional dependencies, there may be **redundant dependencies** — ones that can be derived from the others. The **minimal cover** (also called the **canonical cover**) is the smallest set of functional dependencies from which all others can be derived.

Computing the minimal cover is the basis of automated normalization algorithms used by some database design tools. It eliminates:

- Redundant dependencies (A → B when B can already be derived from other dependencies)
- Redundant attributes on the left side of dependencies (AB → C when A → C alone)

For the purposes of this course, identifying and removing partial and transitive dependencies manually is sufficient. The minimal cover becomes important in advanced normalization (4NF, 5NF) and in algorithm-based schema synthesis.

1NF and the NoSQL Debate

First Normal Form's atomicity requirement — that every cell contains a single value — is contested in modern database practice. JSON document databases (MongoDB, Couchbase), key-value stores (Redis), and column-family databases (Cassandra) explicitly allow multi-valued, nested, and structured column values.

The argument for non-atomic columns in some modern systems:

- If you always retrieve all parts of a composite value together (e.g., a customer's complete address as a unit), storing them as a structured document is more efficient than normalising them into separate columns and joining
- Application-level access is often more natural with documents than with joins
- For systems with rapidly changing schemas, the rigidity of 1NF can be a liability

The argument for 1NF:

- Individual components of a non-atomic value cannot be efficiently filtered, sorted, or aggregated without parsing
- Referential integrity cannot be enforced across JSON values
- SQL's set-based operations assume atomic values

The practical position: relational databases with 1NF are the right choice for transactional systems with well-defined schemas, complex relationships, and integrity requirements. Document stores are appropriate for applications with flexible schemas, document-centric access patterns, and less complex relationships. The relational model remains the dominant choice for business data management precisely because of these guarantees.

References and Further Reading

1. Date, C. J. (2019). *Database Design and Relational Theory: Normal Forms and All That Jazz* (2nd ed.). Apress. — The most rigorous and complete treatment of normal forms, functional dependencies, and the theory underlying normalization.

2. Connolly, T., & Begg, C. (2015). *Database Systems: A Practical Approach to Design, Implementation, and Management* (6th ed.). Pearson. — Chapters 14–15 cover normalization from 1NF through 5NF with extensive worked examples.

3. Elmasri, R., & Navathe, S. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson. — Chapter 14 covers functional dependencies and normal forms rigorously; Chapter 15 covers the relational design algorithms.

4. Armstrong, W. W. (1974). Dependency structures of data base relationships. *Proceedings of IFIP Congress*, 74, 580–583. — The original paper establishing the inference axioms for functional dependencies.

5. Kent, W. (1983). A simple guide to five normal forms in relational database theory. *Communications of the ACM*, 26(2), 120–125. — A remarkably accessible explanation of the normal forms, freely available and widely cited.

6. Microsoft. (2024). *Database normalization description*. https://learn.microsoft.com/en-us/office/troubleshoot/access/database-normalization-description — Microsoft's practical introduction to normalization.

> ***Database Design for Analysts*** | *© Patrick Dolinger, NSCC Institute of Technology CC BY 4.0 — Share and adapt freely with attribution*

Chapter 5: Normalization — 3NF and Beyond: Completing the Model

Database Design for Analysts *From business requirements to relational structures*

© Patrick Dolinger, NSCC Institute of Technology Licensed under Creative Commons Attribution 4.0 International (CC BY 4.0) You are free to share and adapt this material for any purpose, provided appropriate credit is given.

Chapter Overview

Chapter 4 addressed the first two normal forms: 1NF eliminated multi-valued cells and repeating groups; 2NF eliminated partial dependencies on composite keys. A table in 2NF is significantly better than one in UNF or 1NF — but it can still harbour a specific class of redundancy: facts that depend on non-key columns rather than on the key itself.

Third Normal Form (3NF) eliminates this remaining class of dependency — the **transitive dependency** — to produce a schema where every non-key fact depends directly on the primary key, the whole key, and nothing but the key.

This chapter also introduces **Boyce-Codd Normal Form (BCNF)**, a stricter variant of 3NF that closes a subtle loophole, and gives an accessible overview of 4NF and 5NF for completeness. It closes with the practical question that every designer eventually faces: **when to stop normalising**.

By the end of this chapter you will be able to:

- Define transitive dependency and identify instances of it in a table
- Define Third Normal Form and explain its relationship to 2NF
- Normalise a table from 2NF to 3NF by decomposing transitive dependencies
- Explain Boyce-Codd Normal Form and when it differs from 3NF
- Describe 4NF and 5NF at a conceptual level
- Explain the concept of denormalization and when it is appropriate
- Apply the complete normalization process (UNF → 1NF → 2NF → 3NF) to a realistic example
- Evaluate whether the CabotTrail OLTP is in 3NF

Table of Contents

1. The Residual Problem After 2NF
2. Transitive Dependency: The Definition
3. Identifying Transitive Dependencies
4. Third Normal Form (3NF)
5. Normalising to 3NF: Worked Examples
6. The 3NF Test: Kent's Mnemonic
7. Boyce-Codd Normal Form (BCNF)
8. 4NF and 5NF: Brief Overview
9. The Complete Normalization Process
10. CabotTrail in 3NF
11. Denormalization: When to Break the Rules
12. Chapter Summary
13. Review Questions
14. 🔍 Deeper Dive

---

1. The Residual Problem After 2NF

After applying 2NF, the `Orders_2NF` table has only a single-column primary key (`OrderID`) and non-key attributes about the order:

```
Orders_2NF
OrderID | CustomerID | CustomerName  | CustomerCity | OrderDate
1001    | 42         | Fundy Bay     | Halifax      | 2024-03-01
1002    | 17         | Trail Hut     | Truro        | 2024-03-05
1003    | 42         | Fundy Bay     | Halifax      | 2024-04-10
1004    | 17         | Trail Hut     | Truro        | 2024-04-15
```

This table is in 2NF — there is no composite key, so no partial dependencies are possible. But it still has redundancy:

- `CustomerName = 'Fundy Bay'` appears on every order placed by customer 42
- `CustomerCity = 'Halifax'` appears on every order placed by customer 42
- If Fundy Bay's name changes, every one of their order rows must be updated

- If all of customer 42's orders are deleted, the fact that CustomerID 42 exists and is based in Halifax disappears

These are exactly the update and deletion anomalies from Chapter 1 — but they survived 2NF. The cause is a **transitive dependency**.

---

2. Transitive Dependency: The Definition

A **transitive dependency** occurs when a non-key attribute depends on another non-key attribute — rather than depending directly on the primary key.

Formally: given primary key A and non-key attributes B and C, a transitive dependency exists when:

```
A → B   and   B → C   (therefore A → C transitively, not directly)
```

In `Orders_2NF`:

```
OrderID → CustomerID       (knowing the order, you know the customer)
CustomerID → CustomerName  (knowing the customer, you know their name)
CustomerID → CustomerCity  (knowing the customer, you know their city)

Therefore (transitively):
OrderID → CustomerName  (but only through CustomerID — not directly)
OrderID → CustomerCity  (but only through CustomerID — not directly)
```

`CustomerName` and `CustomerCity` depend on `OrderID` — but only *through* `CustomerID`. They are facts about the customer, not about the order. The order merely knows which customer it belongs to; it does not "own" the customer's name or city.

2.1 Visualising the Chain

The transitive dependency forms a chain:

```
OrderID ————→ CustomerID ————→ CustomerName
                           └———→ CustomerCity
```

The arrow from `OrderID` to `CustomerName` is the transitive path. It is not a direct dependency — `CustomerName` would not change if you changed `OrderID` while keeping `CustomerID` the same. The direct dependency runs through `CustomerID`.

2.2 The Intermediate Non-Key Column

The distinguishing feature: the intermediate step in the chain (`CustomerID`) is a **non-key column** in the orders table (it is a FK, but it is not the PK). When a non-key column determines other non-key columns, a transitive dependency exists.

---

3. Identifying Transitive Dependencies

The systematic process for identifying transitive dependencies in a table:

Step 1: List all non-key attributes.

Step 2: For each non-key attribute, ask: "What does this attribute describe — the primary key entity, or something else?"

Step 3: If it describes "something else" — check whether that "something else" is another non-key column. If yes, you have a transitive dependency.

3.1 Applied to Orders_2NF

Non-key attributes: `CustomerID` , `CustomerName` , `CustomerCity` , `OrderDate`

- `CustomerID` — describes the order (which customer placed it). ✓ Direct dependency on `OrderID`
- `CustomerName` — describes the customer, not the order. The customer's name is a fact about `CustomerID` , not about `OrderID` . ✗ Transitive via `CustomerID`
- `CustomerCity` — describes the customer, not the order. ✗ Transitive via `CustomerID`
- `OrderDate` — describes the order (when it was placed). ✓ Direct dependency on `OrderID`

`CustomerName` and `CustomerCity` are transitively dependent on `OrderID` through `CustomerID` . They violate 3NF.

3.2 The "Describes" Test

The practical test: **"Does this attribute describe the primary key entity, or does it describe something referenced by a FK?"**

If it describes something referenced by a FK — it belongs in the table for that FK's entity, not in the current table.

`CustomerName` describes the customer (`CustomerID` entity), not the order (`OrderID` entity). It belongs in a `Customers` table, not in `Orders`. Storing it in `Orders` creates redundancy because the same customer places many orders.

---

4. Third Normal Form (3NF)

A table is in **Third Normal Form** if and only if:

1. It is in **Second Normal Form**
2. Every non-key attribute is **non-transitively dependent on the primary key** — that is, every non-key attribute depends directly on the PK, and on nothing other than the PK

Equivalently: no non-key attribute depends on another non-key attribute.

4.1 The Classic Formulation

Edgar Codd's original 1971 definition states: a table is in 3NF if every non-key attribute depends on "the key, the whole key, and nothing but the key."

This mnemonic is famous. Read it literally:

- **"The key"** → not on part of the key (eliminates 2NF violations — partial dependencies)
- **"The whole key"** → not on something derived from the key through intermediaries
- **"Nothing but the key"** → not on any non-key attribute (eliminates 3NF violations — transitive dependencies)

4.2 Why Transitive Dependencies Cause Anomalies

The three anomalies, demonstrated for `Orders_2NF`:

Update anomaly: Fundy Bay Outfitters changes their trading name to 'Fundy Bay Ltd.' Every row in `Orders` where `CustomerID = 42` must be updated. Missing even one produces inconsistent data — some orders show 'Fundy Bay Outfitters', others show 'Fundy Bay Ltd.'

Insertion anomaly: A new customer is created (CustomerID = 55, CustomerName = 'Cape Breton Supply'). The customer cannot be recorded until they place their first order — because `CustomerName` only exists in the `Orders` table, and an order is needed to create a row.

Deletion anomaly: All orders for customer 17 are deleted (perhaps the customer had a dispute and all records are to be purged). The fact that CustomerID 17 is 'Trail Hut' in 'Truro' disappears along with the orders.

---

5. Normalising to 3NF: Worked Examples

5.1 The Decomposition Step

To remove a transitive dependency A → B → C:

1. Create a **new table** for the transitively dependent attributes, keyed by the intermediate non-key attribute (B)
2. **Remove the transitively dependent attributes** (C) from the original table
3. **Keep the intermediate attribute** (B) in the original table as a FK to the new table

```
Before 3NF (transitive dependency):
Orders_2NF (OrderID, CustomerID, CustomerName, CustomerCity, OrderDate)
Transitive chain: OrderID → CustomerID → CustomerName, CustomerCity
```

```
After 3NF decomposition:
Table 1: Customers_3NF (CustomerID, CustomerName, CustomerCity)
         PK: CustomerID
         (Facts about customers — no longer in Orders)

Table 2: Orders_3NF (OrderID, CustomerID, OrderDate)
         PK: OrderID
         FK: CustomerID → Customers_3NF.CustomerID
         (Facts about orders — CustomerID is a FK, not carrying customer details)
```

```
-- 3NF: Customers table
CREATE TABLE Customers_3NF
(
    CustomerID      INT             NOT NULL,
    CustomerName    NVARCHAR(100)   NOT NULL,
    CustomerCity    NVARCHAR(60)    NOT NULL,

    CONSTRAINT PK_Customers_3NF PRIMARY KEY (CustomerID)
);

-- 3NF: Orders table
CREATE TABLE Orders_3NF
(
    OrderID         INT     NOT NULL,
    CustomerID      INT     NOT NULL,
    OrderDate       DATE    NOT NULL,
```

```
    CONSTRAINT PK_Orders_3NF PRIMARY KEY (OrderID),
    CONSTRAINT FK_Orders_Customer
        FOREIGN KEY (CustomerID) REFERENCES Customers_3NF (CustomerID)
);
```

5.2 The 3NF Data

```
-- One row per customer
INSERT INTO Customers_3NF VALUES
    (42, 'Fundy Bay Outfitters', 'Halifax'),
    (17, 'Trail Hut Ltd',        'Truro');

-- Orders reference customers by ID only
INSERT INTO Orders_3NF VALUES
    (1001, 42, '2024-03-01'),
    (1002, 17, '2024-03-05'),
    (1003, 42, '2024-04-10'),
    (1004, 17, '2024-04-15');
```

Customer details are stored once. The update anomaly is eliminated: change 'Fundy Bay Outfitters' to 'Fundy Bay Ltd' in one row in `Customers_3NF` — all orders referencing CustomerID 42 automatically reflect the change through the FK relationship.

5.3 A Second Example: Employee and Department

Consider:

```
EMPLOYEE (EmployeeID, EmployeeName, DepartmentID, DepartmentName, DepartmentHead)
PK: EmployeeID
```

Check for transitive dependencies:

- `EmployeeName` — describes the employee. ✓ Direct
- `DepartmentID` — describes the employee (which department they are in). ✓ Direct
- `DepartmentName` — describes the department, not the employee. `DepartmentID → DepartmentName`. ✗ Transitive
- `DepartmentHead` — describes the department's head, not the employee. `DepartmentID → DepartmentHead`. ✗ Transitive

3NF decomposition:

```
CREATE TABLE Departments_3NF
(
    DepartmentID    INT             NOT NULL,
```

```
    DepartmentName  NVARCHAR(60)    NOT NULL,
    DepartmentHead  NVARCHAR(100)   NULL,

    CONSTRAINT PK_Departments_3NF PRIMARY KEY (DepartmentID)
);

CREATE TABLE Employees_3NF
(
    EmployeeID      INT             NOT NULL,
    EmployeeName    NVARCHAR(100)   NOT NULL,
    DepartmentID    INT             NOT NULL,

    CONSTRAINT PK_Employees_3NF PRIMARY KEY (EmployeeID),
    CONSTRAINT FK_Emp_Dept FOREIGN KEY (DepartmentID)
        REFERENCES Departments_3NF (DepartmentID)
);
```

5.4 Recognising 3NF in ERDs

In hindsight, a correct ERD avoids transitive dependencies automatically. The `Customer` entity in the ERD has its own box and its own attributes. An `Order` entity has its own attributes and a relationship line to `Customer`. The ERD makes it visually obvious that customer attributes belong in the Customer entity — not duplicated on the Order entity.

This is why the ERD process (Chapter 2) and the mapping rules (Chapter 3) naturally produce a 3NF schema when applied correctly. Normalization analysis becomes necessary when starting from existing flat data (like a spreadsheet) rather than from a clean ERD.

---

6. The 3NF Test: Kent's Mnemonic

William Kent's 1983 paper introduced a memorable phrasing for testing 3NF:

> *"Every non-key attribute must provide a fact about the key, the whole key, and nothing but the key."*

Each phrase targets a specific normal form:

Phrase	Normal form tested	Eliminates
"…a fact about the key"	2NF	Partial dependencies (depends on part of a composite key)
"…the whole key"	2NF	(same — the entire composite key, not just a part)
"…nothing but the key"	3NF	Transitive dependencies (depends on a non-key column)

6.1 Applying the Mnemonic

For each non-key column, ask: does this column provide a fact about the key (the PK entity), and only about the key?

`CustomerName` in `Orders`: Does it provide a fact about the order (`OrderID`)? No — it provides a fact about the customer (`CustomerID`). Fails "nothing but the key." → 3NF violation.

`OrderDate` in `Orders`: Does it provide a fact about the order? Yes — when the order was placed is a fact about the order itself. Passes all three parts of the mnemonic. → No violation.

---

7. Boyce-Codd Normal Form (BCNF)

Boyce-Codd Normal Form (named after Raymond Boyce and Edgar Codd, who proposed it in 1974) is a strengthened version of 3NF that closes a subtle loophole.

7.1 The Loophole in 3NF

3NF requires that non-key attributes depend on "the key, the whole key, and nothing but the key." But it says nothing about what happens when a *key attribute* (part of a candidate key) depends on a non-key column.

In tables with multiple overlapping candidate keys, this situation can arise. A table can be in 3NF but still have a determinant that is not a candidate key.

7.2 BCNF Definition

A table is in **BCNF** if and only if for every non-trivial functional dependency X → Y, X is a **superkey** (a candidate key or a superset of a candidate key).

In other words: the only determinants allowed are superkeys. No non-key column may determine any column — including key columns.

7.3 A BCNF Violation Example

Consider a university course scheduling scenario:

```
COURSE_SCHEDULE (StudentID, CourseID, InstructorID, Room)
Business rules:
- Each student takes each course from exactly one instructor
- Each instructor teaches only one course (though many students)
- Each room hosts only one course at a time, but a course may be in multiple rooms
```

```
Candidate keys: (StudentID, CourseID) and (StudentID, InstructorID)
Functional dependencies:
- (StudentID, CourseID) → InstructorID
- InstructorID → CourseID       ← This is the BCNF violation!
```

`InstructorID → CourseID` is a functional dependency where the determinant (`InstructorID`) is **not** a superkey — it alone cannot identify a row. Yet it determines `CourseID` (a key attribute). This violates BCNF.

7.4 When BCNF Differs from 3NF

BCNF and 3NF produce the same result in most practical cases. The difference only arises in tables with:

- Multiple overlapping candidate keys
- A functional dependency where the determinant is part of one candidate key and the dependent is part of another

Such tables are relatively rare in typical business database design. When they do arise, BCNF decomposition sometimes sacrifices dependency preservation — a trade-off that must be evaluated.

7.5 BCNF in Practice

For CabotTrail and most business databases: if a table is in 3NF, it is almost certainly in BCNF. The distinction only matters in schemas with multiple complex candidate keys — more common in academic examples than in transactional business databases.

The practical guidance: **design to 3NF. Evaluate BCNF compliance when the schema has multiple overlapping candidate keys. If a BCNF decomposition sacrifices dependency preservation, weigh the trade-off carefully.**

---

8. 4NF and 5NF: Brief Overview

For completeness, the higher normal forms address increasingly specialised cases.

8.1 Fourth Normal Form (4NF)

4NF addresses **multi-valued dependencies** — a situation where one attribute independently determines two or more other attributes, creating spurious combinations when stored in the same table.

Example: A `ProductSupplierRegion` table stores which suppliers can provide which products, and in which regions a product is sold:

```
ProductID | SupplierID | Region
83        | SUP-01     | Atlantic
83        | SUP-01     | Ontario
83        | SUP-02     | Atlantic
83        | SUP-02     | Ontario
```

`ProductID` has two **independent** multi-valued facts: it determines a set of suppliers, and it determines a set of regions. These are independent — which suppliers can provide a product has nothing to do with which regions it is sold in. Storing them together forces every (Supplier, Region) combination to be represented, creating redundant rows.

4NF decomposition:

```
ProductSuppliers: (ProductID, SupplierID)
ProductRegions:   (ProductID, Region)
```

4NF is violated when there are independent multi-valued facts about the same entity in the same table. It is relevant primarily when designing junction-like tables with multiple independent M:N relationships.

8.2 Fifth Normal Form (5NF)

5NF (also called **Project-Join Normal Form**) addresses **join dependencies** — situations where a table can only be reconstructed losslessly from three or more projections (sub-tables), not just two.

5NF violations are extremely rare in practice — they arise only when three or more entities have a constrained three-way relationship where knowing any two pairs does not imply the third. Most practical business databases do not encounter 5NF issues.

8.3 Practical Perspective

For business database design:

Normal form	Address in practice?	Notes
1NF	Always	Eliminate multi-valued cells
2NF	Always	Eliminate partial dependencies
3NF	Always	Eliminate transitive dependencies
BCNF	Evaluate when relevant	Only differs from 3NF with overlapping candidate keys

Normal form	Address in practice?	Notes
4NF	Occasionally	Relevant when same entity has multiple independent M:N facts
5NF	Rarely	Complex join dependencies; extremely uncommon in business systems

The target for this course — and for most professional database work — is **3NF**. BCNF is a refinement worth understanding; 4NF and 5NF are theoretical completeness.

---

9. The Complete Normalization Process

This section applies the complete UNF → 1NF → 2NF → 3NF process to a single realistic example, showing all intermediate steps.

9.1 Starting Point: UNF

A product order flat file from a legacy system:

```
ORDER_FLAT (UNF)
OrderNum | OrderDate  | CustID | CustName      | CustProvince | Items (repeating)
         |            |        |               |              | ProdCode | ProdName |
SuppID | SuppName | Qty | Price
1001     | 2024-03-01 | C42    | Fundy Bay     | NS           | P083     | Slp. Bag |
S-01   | MtnGear  | 2   | 249.99
         |            |        |               |              | P101     | Tent     |
S-01   | MtnGear  | 1   | 649.99
1002     | 2024-03-05 | C17    | Trail Hut     | NS           | P045     | Hk. Boot |
S-02   | TrlEquip | 3   | 159.99
1003     | 2024-04-10 | C42    | Fundy Bay     | NS           | P083     | Slp. Bag |
S-01   | MtnGear  | 1   | 249.99
```

9.2 Step 1: To 1NF

Identify the repeating group: `(ProdCode, ProdName, SuppID, SuppName, Qty, Price)`.

Remove it to a new table. The composite PK of the new table: `(OrderNum, ProdCode)` — together they uniquely identify an order line.

```
ORDER_1NF:
OrderNum | OrderDate  | CustID | CustName  | CustProvince
1001     | 2024-03-01 | C42    | Fundy Bay | NS
1002     | 2024-03-05 | C17    | Trail Hut | NS
1003     | 2024-04-10 | C42    | Fundy Bay | NS
```

```
ORDERLINE_1NF:
OrderNum | ProdCode | ProdName | SuppID | SuppName  | Qty | Price
1001     | P083     | Slp. Bag | S-01   | MtnGear   | 2   | 249.99
1001     | P101     | Tent     | S-01   | MtnGear   | 1   | 649.99
1002     | P045     | Hk. Boot | S-02   | TrlEquip  | 3   | 159.99
1003     | P083     | Slp. Bag | S-01   | MtnGear   | 1   | 249.99
```

9.3 Step 2: To 2NF

Check `ORDERLINE_1NF` for partial dependencies. PK: `(OrderNum, ProdCode)`.

- `Qty` — depends on `(OrderNum, ProdCode)`. ✓ Full
- `Price` — depends on `(OrderNum, ProdCode)` if it is the price charged on this order. ✓ Full (assuming historical price capture)
- `ProdName` — depends on `ProdCode` alone. ✗ Partial dependency
- `SuppID` — depends on `ProdCode` alone (each product has one supplier). ✗ Partial dependency
- `SuppName` — depends on `ProdCode` alone (through `SuppID`). ✗ Partial dependency

Remove partial dependencies to a new Products table:

```
PRODUCT_2NF:
ProdCode | ProdName  | SuppID | SuppName
P083     | Slp. Bag  | S-01   | MtnGear
P101     | Tent      | S-01   | MtnGear
P045     | Hk. Boot  | S-02   | TrlEquip

ORDERLINE_2NF:
OrderNum | ProdCode | Qty | Price
1001     | P083     | 2   | 249.99
1001     | P101     | 1   | 649.99
1002     | P045     | 3   | 159.99
1003     | P083     | 1   | 249.99
```

`ORDER_1NF` remains unchanged after this step — its PK `OrderNum` is single-column, so no partial dependencies are possible.

9.4 Step 3: To 3NF

Check both tables for transitive dependencies.

`ORDER_1NF`: Non-key attributes: `OrderDate`, `CustID`, `CustName`, `CustProvince`.

- `OrderDate` — describes the order. ✓ Direct
- `CustID` — describes the order (which customer). ✓ Direct

- `CustName` — describes the customer, not the order. `CustID → CustName`. ✗ Transitive via `CustID`
- `CustProvince` — describes the customer, not the order. `CustID → CustProvince`. ✗ Transitive via `CustID`

`PRODUCT_2NF`: Non-key attributes: `ProdName`, `SuppID`, `SuppName`.

- `ProdName` — describes the product. ✓ Direct
- `SuppID` — describes the product (which supplier provides it). ✓ Direct
- `SuppName` — describes the supplier, not the product. `SuppID → SuppName`. ✗ Transitive via `SuppID`

Remove transitive dependencies:

```
CUSTOMER_3NF:
CustID | CustName  | CustProvince
C42    | Fundy Bay | NS
C17    | Trail Hut | NS

ORDER_3NF:
OrderNum | CustID | OrderDate
1001     | C42    | 2024-03-01
1002     | C17    | 2024-03-05
1003     | C42    | 2024-04-10

SUPPLIER_3NF:
SuppID | SuppName
S-01   | MtnGear
S-02   | TrlEquip

PRODUCT_3NF:
ProdCode | ProdName  | SuppID
P083     | Slp. Bag  | S-01
P101     | Tent      | S-01
P045     | Hk. Boot  | S-02

ORDERLINE_3NF (unchanged from 2NF — no transitive dependencies):
OrderNum | ProdCode | Qty | Price
```

9.5 Final Schema Summary

From one flat UNF table, normalization produced five tables in 3NF:

Table	PK	FKs
`CUSTOMER_3NF`	CustID	—
`SUPPLIER_3NF`	SuppID	—
`PRODUCT_3NF`	ProdCode	SuppID → SUPPLIER
`ORDER_3NF`	OrderNum	CustID → CUSTOMER

Table	PK	FKs
`ORDERLINE_3NF`	(OrderNum, ProdCode)	OrderNum → ORDER, ProdCode → PRODUCT

Every non-key attribute now depends on exactly and only the primary key of its table — directly, not transitively. The schema is in 3NF.

---

10. CabotTrail in 3NF

10.1 Does the OLTP Pass the 3NF Test?

The CabotTrail OLTP was designed with normalization in mind. A systematic check confirms it:

```
-- Check the structure of Sales.Orders
SELECT  c.name          AS ColumnName,
        c.is_nullable,
        tp.name         AS DataType
FROM    sys.tables t
INNER JOIN sys.schemas s  ON s.schema_id = t.schema_id
INNER JOIN sys.columns c  ON c.object_id = t.object_id
INNER JOIN sys.types tp   ON tp.user_type_id = c.user_type_id
WHERE   s.name = 'Sales' AND t.name = 'Orders'
ORDER BY c.column_id;
```

`Sales.Orders` columns: `OrderID`, `CustomerID`, `SalesRepID`, `PickedByPersonID`, `ContactPersonID`, `BackorderOrderID`, `OrderDate`, `ExpectedDeliveryDate`, `CustomerPurchaseOrderNumber`, `IsUndersupplyBackordered`, `Comments`, `DeliveryInstructions`, `InternalComments`, `PickingCompletedWhen`, `LastEditedBy`, `LastEditedWhen`, `CityID`, `DeliveryMethodID`.

Check for transitive dependencies:

- `CustomerID` — which customer placed the order. ✓ Direct (the FK, not the customer's name)
- `OrderDate` — when the order was placed. ✓ Direct
- `CityID` — delivery city for this order. ✓ Direct (the FK, not the city's name)
- `DeliveryMethodID` — how the order is delivered. ✓ Direct (the FK)

No customer name, city name, or delivery method name appears on `Sales.Orders`. Only FKs. This is 3NF — the transitive dependencies have been resolved by referencing dimension tables through FKs rather than duplicating their attributes.

10.2 Where 3NF Would Be Violated in a Less Careful Design

If `Sales.Orders` stored `CustomerName` (from the customer record), that would be a transitive dependency: `OrderID → CustomerID → CustomerName`. The CabotTrail design correctly stores only `CustomerID` and retrieves `CustomerName` by joining to `Sales.Customers`.

If `Sales.Orders` stored `DeliveryMethodName` directly (instead of `DeliveryMethodID`), that would be another transitive dependency. The design correctly references the method through a FK.

Every fact in `Sales.Orders` describes the order itself or identifies a related entity through a FK. There are no attributes that describe the related entity directly. This is the 3NF pattern in production.

10.3 The Geography Hierarchy as 3NF

The four-table geography hierarchy (`Countries → StateProvinces → Cities → Customers.DeliveryCityID`) is a 3NF design. Consider what would happen if `Customers` stored `CityName` and `ProvinceCode` directly:

```
Sales.Customers (bad design):
CustomerID | CustomerName | CityName | ProvinceCode
42         | Fundy Bay    | Halifax  | NS

Transitive chain: CustomerID → CityName → ProvinceCode
(Every city in Halifax belongs to NS — ProvinceCode is a fact about
the city, not directly about the customer)
```

The correct 3NF design:

```
Customers: CustomerID, DeliveryCityID (FK only)
Cities:    CityID, CityName, StateProvinceID (FK only)
StateProvinces: StateProvinceID, StateProvinceCode, CountryID (FK only)
Countries: CountryID, CountryName
```

Each fact lives in exactly the right table. The province code is stored once in `StateProvinces` and referenced everywhere — not duplicated on every customer and every city record.

---

11. Denormalization: When to Break the Rules

Normalization produces the most maintainable, least redundant structure. But it is not always the right structure for every purpose.

11.1 What Denormalization Is

Denormalization is the deliberate introduction of redundancy — the reversal of some normalization steps — to improve read performance at the cost of write complexity and storage.

Denormalization is appropriate when:

- The schema is used primarily for **reading** (analytical queries) rather than writing (transactional updates)
- **Query performance** is the primary concern, and the joins required by a normalised schema are too slow
- The **data changes infrequently** — the update anomaly risk is low
- The team has the **discipline and processes** to maintain the redundant data correctly

11.2 The DW as an Example of Intentional Denormalization

The CabotTrail data warehouse dimensions are intentionally denormalised. `dim.Customer` in `CabotTrailOutdoorsSales` stores `CustomerName`, `CityName`, `ProvinceName`, `ProvinceCode`, `CountryName`, and `SalesTerritory` all in one wide table — information that, in the normalised OLTP, is spread across four tables.

This denormalization is deliberate and justified:

- **The DW is read-only** for analysts — BI tools query it but do not update customer records
- **Joins are expensive** in queries scanning millions of fact rows — flattening the geography hierarchy into the customer dimension avoids three joins on every analytical query
- **Updates are managed by ETL** — when customer data changes in the OLTP, the ETL reloads the denormalised DW dimension

This is the Kimball star schema design philosophy: normalise the OLTP for writes; denormalise the DW for reads.

11.3 Other Denormalization Patterns

Storing aggregates: Adding a `TotalOrderValue` column to the `Orders` table to avoid summing `OrderLines` on every report. Update: the total must be recalculated when any line changes.

Duplicating a lookup value: Storing `ProvinceName` on `Customers` for display purposes even though it is available via join. Update: if the province name changes, every customer row must be updated.

Collapsing a hierarchy: Merging `Cities`, `StateProvinces`, and `Countries` into `Customers` as flat columns. Update: every customer in Halifax must be updated if Halifax's province changes.

11.4 The Denormalization Decision

Denormalization is not wrong — it is a trade-off. The question is always: **is the performance gain worth the maintenance cost and the risk of anomalies?**

For OLTP transactional systems: almost never. Normalization to 3NF is almost always the right choice because consistency and data integrity are paramount.

For OLAP analytical systems and data warehouses: frequently. The read-heavy access pattern, the managed ETL update process, and the need for query performance justify controlled denormalization.

The discipline: Know what normal form your schema is in. Know which rules you have broken and why. Document the denormalization decisions in the data dictionary so future developers understand the trade-offs.

---

12. Chapter Summary

- **Transitive dependency** occurs when a non-key attribute depends on another non-key attribute — forming a chain A → B → C where B is not a key. It causes update, insertion, and deletion anomalies.
- **Third Normal Form (3NF)** requires: in 2NF, plus every non-key attribute depends directly on the PK — "the key, the whole key, and nothing but the key."
- To identify 3NF violations: ask "does this attribute describe the primary key entity, or does it describe something referenced by a FK?" If the latter, it is transitively dependent.
- To normalise to 3NF: create a new table for the transitively dependent attributes, keyed by the intermediate non-key determinant. Keep the intermediate attribute as a FK in the original table.
- **BCNF** strengthens 3NF: every determinant must be a superkey. BCNF only differs from 3NF in tables with multiple overlapping candidate keys.
- **4NF** eliminates independent multi-valued dependencies. **5NF** eliminates join dependencies. Both are rare in practice; 3NF is the practical target.
- The **complete normalization process** (UNF → 1NF → 2NF → 3NF) transforms a flat table into a set of smaller, well-structured tables. Each step eliminates a specific class of redundancy.
- The CabotTrail OLTP is in 3NF: every column on every table describes only the entity that table represents, referencing related entities through FKs rather than copying their attributes.

- **Denormalization** deliberately introduces redundancy to improve read performance. It is appropriate for OLAP and DW designs but requires discipline, documentation, and managed update processes.

---

13. Review Questions

1. A table `EMPLOYEE_PROJECT (EmployeeID, ProjectID, ProjectName, ProjectManagerID, ProjectManagerName, HoursWorked)` has PK `(EmployeeID, ProjectID)`. After applying 2NF, you have `EMPLOYEE_PROJECT_2NF (EmployeeID, ProjectID, HoursWorked)` and `PROJECT_2NF (ProjectID, ProjectName, ProjectManagerID, ProjectManagerName)`. Does `PROJECT_2NF` contain any 3NF violations? Identify them and show the 3NF decomposition.

2. Explain the transitive dependency chain in the following table, draw the functional dependency diagram, and show the complete 3NF decomposition: `INVOICE (InvoiceID, CustomerID, CustomerName, CustomerCity, CustomerProvince, InvoiceDate, TotalAmount)`

3. Apply Kent's mnemonic ("the key, the whole key, and nothing but the key") to each non-key column in `Sales.Orders` from `CabotTrailOutdoor`. For each column, state which part of the mnemonic it satisfies and why. Use the FK catalog query from Chapter 1 to identify the columns and their relationships.

4. The following is a claim about BCNF vs 3NF: "In practice, if a table is in 3NF, it is almost always also in BCNF." Explain the conditions under which this claim is TRUE, and construct a specific example where a table can be in 3NF but NOT in BCNF.

5. Walk through the complete normalization process (UNF → 1NF → 2NF → 3NF) for the following flat data:

   ```
   STAFF_COURSE (StaffID, StaffName, DeptID, DeptName, CourseID, CourseName,
   DateCompleted, Score)
   Business rules: One staff member can complete many courses.
   Each course is offered by one department. Each staff member belongs to one
   department.
   ```

   Show intermediate tables at each step with PKs and FKs.

6. The CabotTrail data warehouse `dim.Customer` stores `CityName`, `ProvinceName`, and `CountryName` as flat columns — data that is spread across four normalised tables in the OLTP. Explain why this is acceptable in the DW but would be unacceptable in the OLTP. Reference the denormalization principles from section 11.

7. Write a SQL query against `CabotTrailOutdoor.Sales.Customers` that verifies there are no 3NF violations — that is, that no customer-level columns are storing facts that belong to other entities (cities, provinces, countries). Hint: check what FKs exist on this table using the system catalog, and confirm that geographical information is stored as FKs, not as direct attribute columns.
8. A designer argues: "Normalization is for academic exercises. Real-world databases need to be fast, so we should always denormalize." Construct a balanced response that acknowledges the valid points in this argument while explaining the risks and the contexts where each approach is appropriate.

🔍 Deeper Dive

Going Further with 3NF and Beyond

The Synthesis Algorithm for 3NF

Rather than decomposing an existing unnormalised table, it is possible to **synthesise** a 3NF schema directly from a set of functional dependencies. The synthesis algorithm (Bernstein 1976) proceeds as follows:

1. Compute the minimal cover of the functional dependencies
2. For each dependency in the minimal cover, create a table with the determinant as PK and the dependent(s) as non-key columns
3. If no table contains a candidate key of the original relation, add a table containing a candidate key
4. Eliminate redundant tables (tables whose attributes are a subset of another table's attributes)

The synthesis algorithm **guarantees**:

- The result is in 3NF
- All functional dependencies are preserved
- The decomposition is lossless

It is the theoretical basis for the practical normalization steps this chapter describes manually.

Bernstein, P. A. (1976). Synthesizing third normal form relations from functional dependencies. *ACM Transactions on Database Systems*, 1(4), 277–298.

Heath's Theorem and Lossless Decomposition

The formal guarantee of lossless decomposition comes from **Heath's Theorem** (1971):

Given a relation R with attributes ABC... if A → B holds in R, then R can be losslessly decomposed into R1 (A, B) and R2 (A, C...).

The condition: A → B must hold (A must functionally determine B, and A must be in both tables). When this condition is met, joining R1 and R2 on A reconstructs R exactly — no spurious tuples, no missing tuples.

This theorem underlies every normalization decomposition in this and the previous chapter. Each step removes a functional dependency (partial or transitive) by creating a new table keyed by the determinant — precisely the structure Heath's Theorem requires for losslessness.

Heath, I. J. (1971). Unacceptable file operations in a relational database. *Proceedings of the 1971 ACM SIGFIDET Workshop on Data Description, Access and Control*, 19–33.

Multivalued Dependencies and 4NF in Practice

Fourth Normal Form, while theoretically important, appears in practical database design more often than students expect — specifically in product catalogues, inventory systems, and configuration data.

A real-world 4NF scenario: a software product that supports multiple languages AND multiple operating systems. If language support and OS support are **independent** facts about the product (supporting English does not imply anything about Windows support), storing both in a single table forces a cross-product:

```
ProductID | Language | OS
P001      | English  | Windows
P001      | English  | Mac
P001      | French   | Windows
P001      | French   | Mac
```

Adding a third language requires adding two rows (one for each OS). Adding a third OS requires adding two rows (one for each language). The table has a **multivalued dependency** — `ProductID ↠ Language` and `ProductID ↠ OS` are independent of each other.

4NF decomposition:

```
ProductLanguages: (ProductID, Language)
ProductOS:        (ProductID, OS)
```

This reduces the N×M table to two N+M tables — significantly reducing redundancy and eliminating the coupling between language and OS management.

References and Further Reading

1. Kent, W. (1983). A simple guide to five normal forms in relational database theory. *Communications of the ACM*, 26(2), 120–125. — The paper introducing the "key, the whole key, and nothing but the key" mnemonic. Required reading; freely available.

2. Date, C. J. (2019). *Database Design and Relational Theory: Normal Forms and All That Jazz* (2nd ed.). Apress. — Chapters 5–9 cover 3NF, BCNF, 4NF, and 5NF with the mathematical rigour the subject deserves.

3. Connolly, T., & Begg, C. (2015). *Database Systems: A Practical Approach to Design, Implementation, and Management* (6th ed.). Pearson. — Chapter 14 covers 3NF and BCNF; Chapter 15 covers higher normal forms.

4. Codd, E. F. (1971). Further normalization of the data base relational model. *IBM Research Report*, RJ909. — The original 3NF paper. Codd introduced 3NF as an extension of his relational model.

5. Bernstein, P. A. (1976). Synthesizing third normal form relations from functional dependencies. *ACM Transactions on Database Systems*, 1(4), 277–298. — The synthesis algorithm.

6. Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit* (3rd ed.). Wiley. — Chapter 1 explains the denormalization philosophy of dimensional modelling — the "why" behind star schema design.

7. Elmasri, R., & Navathe, S. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson. — Chapter 14–15 cover all normal forms with formal proofs and the synthesis algorithm.

> ***Database Design for Analysts*** *| © Patrick Dolinger, NSCC Institute of Technology CC BY 4.0 — Share and adapt freely with attribution*

Chapter 6: Constraints and Data Integrity — Encoding Business Rules

Database Design for Analysts *From business requirements to relational structures*

© Patrick Dolinger, NSCC Institute of Technology Licensed under Creative Commons Attribution 4.0 International (CC BY 4.0) You are free to share and adapt this material for any purpose, provided appropriate credit is given.

Chapter Overview

Normalization ensures that data is organized correctly — that facts live in the right places and redundancy is eliminated. But organization alone is not enough. A normalized table can still store absurd or impossible values: a negative price, a future hire date for a retired employee, an order quantity of zero, a customer with a credit limit that exceeds any business rationale.

Constraints are the database's mechanism for enforcing business rules at the storage layer — rules that must be true for data to be valid. When a constraint is violated, the database rejects the operation before any data is changed. The application cannot bypass it, the ETL process cannot bypass it, and — most importantly — human error cannot bypass it.

This chapter covers every constraint type available in SQL Server, the business rules they encode, and the design decisions behind them. It also covers NULL handling in depth, default values, identity columns, and the catalog queries that reveal what constraints already exist.

By the end of this chapter you will be able to:

- Explain the purpose of each SQL Server constraint type and when to use each
- Write PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, and DEFAULT constraints with proper naming
- Design CHECK constraints that encode complex business rules
- Explain the difference between NULL and NOT NULL and choose correctly
- Configure IDENTITY columns appropriately
- Query the system catalog to discover existing constraints
- Describe the cascading referential integrity options and choose among them

- Explain computed columns and when to use them

Table of Contents

1. Why Constraints Belong in the Database
2. NOT NULL: The Simplest Constraint
3. NULL: What It Means and When to Allow It
4. PRIMARY KEY Constraints
5. IDENTITY Columns
6. FOREIGN KEY Constraints
7. Referential Integrity Actions
8. UNIQUE Constraints
9. CHECK Constraints
10. DEFAULT Constraints
11. Computed Columns
12. Constraint Naming and Documentation
13. Discovering Constraints in the System Catalog
14. Constraints in the CabotTrail OLTP
15. Chapter Summary
16. Review Questions
17. 🔍 Deeper Dive

---

1. Why Constraints Belong in the Database

1.1 The Defence-in-Depth Principle

Business rules can be enforced at multiple layers:

- **Application layer:** The web form validates that price is positive before submitting
- **ETL layer:** The transformation script checks for negative quantities before loading
- **Database layer:** A CHECK constraint rejects negative prices at the storage level

Application and ETL validation is important — but it is not sufficient. Applications change, bugs are introduced, multiple applications access the same database, direct SQL connections bypass application validation entirely, and ETL processes sometimes receive corrected data that re-violates rules.

Database constraints are the last line of defence. They are always active, always enforced, and immune to application-layer bugs. A database with proper constraints will never store data that violates the rules it encodes, regardless of where that data comes from.

1.2 Constraints as Documentation

Every constraint is a documented business rule. A CHECK constraint that reads `CHECK (UnitPrice > 0)` says: "CabotTrail has decided that unit prices are always positive. This is a business rule." A future developer reading the schema understands this rule without consulting any document outside the database.

A constraint that exists but is undocumented outside the database is still better than a rule enforced only in application code — the constraint is discoverable through the system catalog; application code may be buried in a codebase no one fully understands.

1.3 The Cost of Missing Constraints

Consider what happens without constraints:

- A data entry error creates a product with `UnitPrice = -5.00`. Reports show negative revenue.
- An ETL script loads invoice lines with `Quantity = 0`. Average unit price calculations divide by zero.
- A duplicate customer is created with the same email address. Marketing sends duplicate communications; customer service cannot determine which record is authoritative.
- An order line references `ProductID = 9999`, which does not exist. Lookups fail silently or return NULLs.

Each of these would be caught immediately by the appropriate constraint. Without them, the errors may propagate undetected for months.

---

2. NOT NULL: The Simplest Constraint

`NOT NULL` is the most frequently applied constraint. It declares that a column must always contain a value — the column cannot be empty.

2.1 Syntax

```
-- Column-level NOT NULL
CREATE TABLE Sales.Customers
(
    CustomerID      INT             NOT NULL,
    CustomerName    NVARCHAR(100)   NOT NULL,   -- Cannot be blank
    CreditLimit     DECIMAL(18,2)   NOT NULL,   -- Cannot be blank
    AccountOpenedDate DATE          NULL,        -- Can be blank (optional)
    EmailAddress    NVARCHAR(256)   NULL         -- Can be blank
);
```

`NOT NULL` is the default in some database systems but not in SQL Server — SQL Server columns are nullable by default unless `NOT NULL` is explicitly specified. Always state nullability explicitly.

2.2 When to Apply NOT NULL

Apply `NOT NULL` to every column where:

- The attribute is **mandatory** for the entity to have meaning — `CustomerName` without a name is not a meaningful customer record
- The value is **always known** at the time the row is created
- A NULL would be **misleading** — a product with NULL `UnitPrice` could be interpreted as "free" or "unknown" or "error," creating ambiguity

2.3 Adding NOT NULL to an Existing Table

Adding `NOT NULL` to a column in a table that already contains rows requires that all existing rows have a non-NULL value in that column first:

```
-- Step 1: Populate existing NULLs before adding the constraint
UPDATE Sales.Customers
SET    EmailAddress = 'unknown@cabot-trail.ca'
WHERE  EmailAddress IS NULL;

-- Step 2: Add the NOT NULL constraint
ALTER TABLE Sales.Customers
ALTER COLUMN EmailAddress NVARCHAR(256) NOT NULL;
```

If any NULL remains when you attempt to add NOT NULL, SQL Server raises an error and rejects the change.

---

3. NULL: What It Means and When to Allow It

Chapter 2 of *SQL for Analysts* covered NULL's query behaviour. Here the design question: **when should a column be nullable?**

3.1 Three Meanings of NULL

SQL Server's NULL has multiple possible interpretations:

Interpretation	Example	Correct use of NULL?
Unknown	`AccountOpenedDate` — we do not know when this account was opened	Yes — record exists but fact is unknown
Not applicable	`MiddleName` for a customer who has no middle name	Debatable — could use empty string, could use NULL
Not yet assigned	`PickedByPersonID` — order not yet picked	Yes — fact will exist later but not now
Deliberately absent	`EmailAddress` — customer has not provided one	Yes — optional attribute

The important distinction: NULL means "no value here" — it does not mean zero, empty string, 'N/A', or 'Unknown'. Using NULL consistently for its intended purpose and documenting its meaning in the data dictionary prevents misinterpretation.

3.2 NULL in Aggregate Functions

As covered in Chapter 6 of *SQL for Analysts*, aggregate functions ignore NULLs. This can produce unexpected results when designing nullable columns:

```
-- If DeliveryDays is nullable, AVG ignores NULL rows
SELECT AVG(DeliveryDays) FROM Sales.Orders;
-- Returns the average only for orders WITH a delivery date — not all orders
-- Is that the intended analysis?
```

If the business needs to include all orders in the average (treating undelivered orders as 0 days or as a separate category), storing NULL may not be the right design. Sometimes a sentinel value (e.g., 0 or -1) is more appropriate, though it reduces the clarity that NULL provides.

3.3 The Three-Valued Logic Reminder

NULL in a comparison produces UNKNOWN (not TRUE or FALSE). This is why `WHERE column = NULL` returns no rows — `= NULL` always produces UNKNOWN. Use `IS NULL` and `IS NOT NULL`.

For design purposes: a nullable column in a WHERE condition requires extra care in queries. Document nullable columns in the data dictionary with their intended interpretation.

---

4. PRIMARY KEY Constraints

The PRIMARY KEY constraint enforces two rules simultaneously: uniqueness and NOT NULL. No two rows may have the same PK value, and no row may have a NULL PK.

4.1 Syntax

```
-- Single-column PK
CREATE TABLE Sales.Customers
(
    CustomerID  INT NOT NULL,
    CustomerName NVARCHAR(100) NOT NULL,
    ...
    CONSTRAINT PK_Customers PRIMARY KEY (CustomerID)
);

-- Composite PK
CREATE TABLE Inventory.ProductCategoryAssignments
(
    ProductID           INT NOT NULL,
    ProductCategoryID   INT NOT NULL,
    ...
    CONSTRAINT PK_ProductCategoryAssignments
        PRIMARY KEY (ProductID, ProductCategoryID)
);
```

4.2 What SQL Server Creates Behind the Scenes

When you define a PRIMARY KEY constraint, SQL Server automatically creates a **clustered index** on the PK column(s). This means:

- The physical data pages of the table are sorted by PK value
- Lookups by PK are extremely fast — O(log n) binary search on the clustered index
- Only one clustered index can exist per table

This has implications: if a table is most commonly queried by a non-PK column, the clustered index on the PK may not be optimal. Chapter 7 covers this trade-off in depth.

4.3 One Table, One Primary Key

A table has exactly one primary key — though the key may span multiple columns. If you need uniqueness enforced on a non-PK column, use a UNIQUE constraint (section 8).

---

5. IDENTITY Columns

`IDENTITY(seed, increment)` is not technically a constraint — it is a column property. But it is closely associated with primary key design and belongs in this discussion.

5.1 What IDENTITY Does

An `IDENTITY` column automatically generates a unique integer value when a new row is inserted. The database manages the counter — applications do not supply the value.

```
CustomerID INT NOT NULL IDENTITY(1, 1)
-- IDENTITY(seed, increment):
--   seed = starting value (1)
--   increment = step between values (1)
-- Result: 1, 2, 3, 4, 5, ...
```

5.2 Common Variations

```
-- Start at 1000 (useful when reserving low numbers for special rows)
CustomerID INT NOT NULL IDENTITY(1000, 1)

-- Start at 0 (for the unknown member / default row)
-- Then reseed to 1 after inserting the unknown row
DBCC CHECKIDENT ('Sales.Customers', RESEED, 0);
INSERT INTO Sales.Customers (CustomerID, CustomerName, ...)
VALUES (0, 'Unknown Customer', ...);
DBCC CHECKIDENT ('Sales.Customers', RESEED, 1);
-- Next auto-generated value will be 2
```

5.3 Retrieving the Generated Value

After inserting a row with an IDENTITY column, retrieve the generated value using `SCOPE_IDENTITY()`:

```
INSERT INTO Sales.Customers (CustomerName, CreditLimit)
VALUES ('New Customer Ltd', 5000.00);

SELECT SCOPE_IDENTITY() AS NewCustomerID;
-- Returns the CustomerID generated for this insert
```

`SCOPE_IDENTITY()` returns the last IDENTITY value generated in the current scope (the current query or stored procedure). Use it rather than `@@IDENTITY` (which can return values generated by triggers in other scopes).

5.4 IDENTITY Gaps

IDENTITY values are not reused when rows are deleted. Gaps in the sequence are normal and not a problem — the IDENTITY column is a key, not a business sequence number. If business requirements demand gapless sequences (invoice numbers, cheque numbers), use a separate sequence object or a dedicated counter table, not IDENTITY.

---

6. FOREIGN KEY Constraints

A FOREIGN KEY constraint enforces **referential integrity** — every value in the FK column must either be NULL (if the column is nullable) or must match a value in the referenced PK column.

6.1 Syntax

```
CREATE TABLE Sales.Orders
(
    OrderID     INT NOT NULL IDENTITY(1,1),
    CustomerID  INT NOT NULL,
    SalesRepID  INT NULL,       -- Nullable FK: order may have no assigned rep

    CONSTRAINT PK_Orders PRIMARY KEY (OrderID),

    CONSTRAINT FK_Orders_Customers
        FOREIGN KEY (CustomerID)
        REFERENCES Sales.Customers (CustomerID),

    CONSTRAINT FK_Orders_SalesRep
        FOREIGN KEY (SalesRepID)
        REFERENCES Application.Employees (EmployeeID)
);
```

6.2 What SQL Server Enforces

A FK constraint enforces referential integrity in both directions:

On INSERT/UPDATE: A new order with `CustomerID = 999` is rejected if CustomerID 999 does not exist in `Sales.Customers`.

On DELETE/UPDATE of the referenced row: Attempting to delete `Sales.Customers` row with `CustomerID = 42` while `Sales.Orders` has rows referencing CustomerID 42 is blocked (with default `NO ACTION` behaviour).

6.3 FK Without an Index on the FK Column

SQL Server creates an index on the referenced PK automatically (as part of the PK constraint). But it does **not** automatically create an index on the FK column itself — the referencing side.

This matters for performance: when SQL Server enforces a FK, it must look up the referenced PK. When the referenced table's rows are deleted or updated, SQL Server must find all referencing rows. Without an index on the FK column, this lookup requires a full table scan.

Always create a non-clustered index on FK columns:

```
CREATE NONCLUSTERED INDEX IX_Orders_CustomerID
    ON Sales.Orders (CustomerID);

CREATE NONCLUSTERED INDEX IX_Orders_SalesRepID
    ON Sales.Orders (SalesRepID);
```

Chapter 7 covers index design in detail. The FK-index practice is so important that it is introduced here.

---

7. Referential Integrity Actions

The `ON DELETE` and `ON UPDATE` clauses of a FK constraint define what happens when the referenced row changes.

7.1 The Four Actions

```
CONSTRAINT FK_Orders_Customers
    FOREIGN KEY (CustomerID)
    REFERENCES Sales.Customers (CustomerID)
    ON DELETE NO ACTION     -- Default
    ON UPDATE NO ACTION     -- Default
```

`NO ACTION` (default): Reject the delete/update if referencing rows exist. The most conservative and safest option. The application must explicitly handle the dependents before the parent can be changed.

`CASCADE`: Propagate the change. Deleting a customer cascades to delete all their orders. Updating a CustomerID cascades to update all referencing order rows.

```
ON DELETE CASCADE   -- Deleting the parent deletes all children automatically
ON UPDATE CASCADE   -- Updating the parent's PK propagates to all FK references
```

`SET NULL`: Set the FK to NULL when the referenced row is deleted/updated. Only valid on nullable FK columns.

```
ON DELETE SET NULL  -- Deleting the SalesRep sets Orders.SalesRepID = NULL
                    -- The order still exists; it just has no assigned rep
```

`SET DEFAULT`: Set the FK to its column-level default value. Rarely used — requires a DEFAULT constraint on the FK column pointing to a valid "fallback" row (like the unknown member, CustomerID = 0).

7.2 Choosing the Right Action

The referential action choice is a **business rule** about what the business wants to happen in each scenario:

Scenario	Business question	Appropriate action
Customer is deleted	Should all their orders be deleted too?	`CASCADE` only if yes; `NO ACTION` if orders must be preserved
Sales rep leaves the company	Should their orders lose the rep assignment?	`SET NULL` if order must survive; mark order as unassigned
Product is discontinued	Should all order lines for that product be deleted?	Almost never `CASCADE` — use `NO ACTION` and soft-delete the product
PK value is updated	Should all references update automatically?	`CASCADE` is safest for PK changes if they are allowed at all

7.3 The Cascade Danger

`ON DELETE CASCADE` should be used with caution. A single DELETE statement on a parent table can trigger cascading deletes across many child tables, potentially removing thousands of rows silently. For operational data that must be preserved for audit or historical analysis, `NO ACTION` with application-level dependency management is far safer.

The CabotTrail OLTP uses `NO ACTION` throughout — an appropriate choice for a transactional system where historical integrity is paramount.

---

8. UNIQUE Constraints

A UNIQUE constraint enforces that no two rows have the same value(s) in the specified column(s). It differs from the PRIMARY KEY in two ways:

- A table can have **multiple** UNIQUE constraints
- UNIQUE columns **can be nullable** (NULL is treated as a distinct value — two NULL values are allowed, because NULL ≠ NULL)

8.1 Syntax

```
CREATE TABLE Sales.Customers
(
    CustomerID      INT             NOT NULL IDENTITY(1,1),
    CustomerName    NVARCHAR(100)   NOT NULL,
    EmailAddress    NVARCHAR(256)   NULL,

    CONSTRAINT PK_Customers             PRIMARY KEY (CustomerID),
    CONSTRAINT UQ_Customers_Email       UNIQUE (EmailAddress)
    -- No two customers can have the same email address
    -- (NULL email addresses are allowed and do not conflict with each other)
);
```

8.2 Composite UNIQUE Constraints

UNIQUE can span multiple columns. The combination must be unique; individual columns may repeat:

```
-- A product can be assigned to each category at most once
-- (same as the composite PK in ProductCategoryAssignments — but here
-- with a surrogate PK, the UNIQUE constraint enforces the combination rule)
CONSTRAINT UQ_ProductCategoryAssignments
    UNIQUE (ProductID, ProductCategoryID)
```

8.3 UNIQUE as a 1:1 Enforcer

As covered in Chapter 3, a UNIQUE constraint on the FK column of a 1:1 relationship enforces the maximum cardinality of one:

```
-- A customer has at most one loyalty account
CONSTRAINT UQ_LoyaltyAccounts_CustomerID
    UNIQUE (CustomerID)
-- Without this, multiple accounts per customer would be possible
```

8.4 UNIQUE vs PRIMARY KEY

Feature	PRIMARY KEY	UNIQUE
Limit per table	One	Many
NULL allowed	Never	Yes (treated as distinct)
Clustered index created	Yes (by default)	No (creates non-clustered index)
Purpose	Row identifier	Business uniqueness rule

---

9. CHECK Constraints

A CHECK constraint tests a condition against every row being inserted or updated. If the condition evaluates to FALSE, the operation is rejected. If it evaluates to TRUE or UNKNOWN (NULL), the operation proceeds.

9.1 Syntax

```
CONSTRAINT CK_constraintname CHECK (condition)
```

The condition is any Boolean expression involving columns of the same table. Subqueries are not allowed — CHECK constraints cannot reference other tables.

9.2 Simple Value Range Checks

```
-- Unit price must be positive
CONSTRAINT CK_Products_UnitPrice
    CHECK (UnitPrice > 0)

-- Month number must be between 1 and 12
CONSTRAINT CK_Calendar_Month
    CHECK (MonthNumber BETWEEN 1 AND 12)

-- Hire date cannot be in the future
CONSTRAINT CK_Employees_HireDate
    CHECK (HireDate <= CAST(GETDATE() AS DATE))

-- Credit limit must be zero or positive
CONSTRAINT CK_Customers_CreditLimit
    CHECK (CreditLimit >= 0)
```

9.3 Cross-Column Checks

CHECK constraints can compare multiple columns within the same row:

```
-- The due date must be after the invoice date
CONSTRAINT CK_Invoices_DueDate
    CHECK (DueDate >= InvoiceDate)

-- Picked quantity cannot exceed ordered quantity
CONSTRAINT CK_OrderLines_PickedQty
    CHECK (PickedQuantity <= Quantity)

-- Discount percentage must be between 0 and 100
CONSTRAINT CK_Promotions_Discount
    CHECK (DiscountPct BETWEEN 0.00 AND 100.00)
```

9.4 Domain Value Checks

CHECK can enforce that a column contains only specific allowed values:

```
-- Transaction type must be EARN or REDEEM
CONSTRAINT CK_LoyaltyTransactions_TransactionType
    CHECK (TransactionType IN ('EARN', 'REDEEM'))

-- Credit rating must be one of the valid ratings
CONSTRAINT CK_Customers_Rating
    CHECK (CreditRating IN ('AAA', 'AA', 'A', 'BBB', 'BB', 'B', 'NR'))

-- Province code must be a valid Canadian province/territory code
CONSTRAINT CK_Addresses_Province
    CHECK (ProvinceCode IN
('AB','BC','MB','NB','NL','NS','NT','NU','ON','PE','QC','SK','YT'))
```

9.5 Conditional Checks

CHECK constraints can express conditional business rules:

```
-- If TransactionType is EARN, PointsAmount must be positive
-- If TransactionType is REDEEM, PointsAmount must be negative
CONSTRAINT CK_LoyaltyTransactions_PointsDirection
    CHECK (
        (TransactionType = 'EARN'   AND PointsAmount > 0) OR
        (TransactionType = 'REDEEM' AND PointsAmount < 0)
    )

-- If IsActive = 1, TerminationDate must be NULL
-- If IsActive = 0, TerminationDate must be populated
CONSTRAINT CK_Employees_TerminationDate
    CHECK (
        (IsActive = 1 AND TerminationDate IS NULL) OR
```

```
        (IsActive = 0 AND TerminationDate IS NOT NULL)
    )
```

9.6 NULL Behaviour in CHECK Constraints

A CHECK constraint passes (does not reject the row) when the condition is NULL (UNKNOWN), not just when it is TRUE. This means:

```
CONSTRAINT CK_Products_Price CHECK (UnitPrice > 0)
-- This constraint PASSES for:
-- UnitPrice = 100.00 (TRUE)
-- UnitPrice = NULL   (UNKNOWN — not FALSE, so it passes)
-- This constraint FAILS for:
-- UnitPrice = -5.00  (FALSE)
-- UnitPrice = 0      (FALSE)
```

If NULL should also be rejected, combine NOT NULL with the CHECK:

```
UnitPrice   DECIMAL(18,2) NOT NULL,  -- Rejects NULL
CONSTRAINT CK_Products_Price CHECK (UnitPrice > 0)  -- Rejects zero/negative
```

The two constraints together enforce: UnitPrice must be a positive number.

---

10. DEFAULT Constraints

A DEFAULT constraint provides a value for a column when a row is inserted without explicitly specifying that column's value.

10.1 Syntax

```
-- Column-level default (inline)
OrderDate   DATE NOT NULL DEFAULT GETDATE()

-- Named constraint (preferred for production)
OrderDate   DATE NOT NULL,
CONSTRAINT DF_Orders_OrderDate DEFAULT GETDATE() FOR OrderDate
```

Named DEFAULT constraints can be dropped by name with `ALTER TABLE DROP CONSTRAINT`. Unnamed defaults require finding the system-generated name first.

10.2 Common Default Patterns

```
-- Default current date
CreatedDate     DATE        NOT NULL DEFAULT GETDATE()

-- Default to 0 (for counts, amounts that start at zero)
InvoiceCount    INT         NOT NULL DEFAULT 0
TotalRevenue    DECIMAL(18,2) NOT NULL DEFAULT 0.00

-- Default to FALSE (for bit flags)
IsDiscontinued  BIT         NOT NULL DEFAULT 0
IsActive        BIT         NOT NULL DEFAULT 1

-- Default to a specific lookup value
StatusID        INT         NOT NULL DEFAULT 1  -- 1 = 'Pending' in a Status lookup
table

-- Default NULL (explicit — makes intent clear even though NULL is already the default)
TerminationDate DATE        NULL DEFAULT NULL
```

10.3 Defaults and NOT NULL Together

A common and important pattern: combine NOT NULL with a DEFAULT to ensure a column always has a value while making inserts that omit the column simpler:

```
-- Without DEFAULT, every INSERT must supply CreatedDate
INSERT INTO Sales.Orders (CustomerID, OrderDate, CreatedDate)
VALUES (42, '2024-03-01', GETDATE());

-- With DEFAULT GETDATE(), the column can be omitted
INSERT INTO Sales.Orders (CustomerID, OrderDate)
VALUES (42, '2024-03-01');
-- CreatedDate is automatically set to the current date
```

---

11. Computed Columns

A **computed column** is a column whose value is automatically derived from an expression involving other columns in the same row. It is not stored by default — it is recalculated each time it is referenced.

11.1 Syntax

```
-- Computed column: LineTotal = Quantity * UnitPrice (recalculated on each query)
LineTotal AS (Quantity * UnitPrice)
```

```
-- Persisted computed column: stored physically, maintained automatically
LineTotalIncludingTax AS (LineTotal + TaxAmount) PERSISTED
```

`PERSISTED` stores the computed value on disk and updates it automatically when any component column changes. Persisted computed columns:

- Can be indexed (non-persisted cannot)
- Are faster to query (pre-computed)
- Use more storage
- Can be used in CHECK constraints and foreign keys

11.2 CabotTrail Examples

```
-- Sales.InvoiceLines computed columns (from the CabotTrail OLTP)
LineTotal              AS (Quantity * UnitPrice),
TaxAmount              AS (LineTotal * TaxRate / 100),
LineTotalIncludingTax  AS (LineTotal + TaxAmount) PERSISTED
```

The computed column `LineTotal` is recalculated each time it is queried. `LineTotalIncludingTax` is persisted — it is stored and indexed.

11.3 Computed Columns vs Application Logic

Computed columns enforce derivation rules at the database layer — they cannot be set to an incorrect value by an INSERT or UPDATE. The application cannot accidentally store `LineTotal = -1` when `Quantity = 2` and `UnitPrice = 249.99`. The database always computes the correct value.

This is the constraint philosophy applied to derived values: encode the rule in the database, where it is always enforced.

---

12. Constraint Naming and Documentation

Consistent constraint naming is not just a style preference — it is a professional practice that makes database maintenance dramatically easier.

12.1 The Naming Conventions

From Chapter 3, the CabotTrail convention:

Constraint	Pattern	Example
Primary key	`PK_TableName`	`PK_Customers`
Foreign key	`FK_ChildTable_ParentTable`	`FK_Orders_Customers`
Unique	`UQ_TableName_Column`	`UQ_Customers_Email`
Check	`CK_TableName_Column`	`CK_Products_UnitPrice`
Default	`DF_TableName_Column`	`DF_Orders_OrderDate`

12.2 Why Names Matter in Production

Error messages: When a constraint violation occurs, SQL Server's error message includes the constraint name:

```
The INSERT statement conflicted with the FOREIGN KEY constraint "FK_Orders_Customers".
```

With a named constraint, the error message is immediately informative — it tells you exactly which relationship was violated. With an unnamed constraint, the system-generated name (`FK__Orders__Customer__4AB81AF0`) tells you nothing.

Maintenance operations: Dropping a constraint requires knowing its name:

```
-- Named constraint: trivial to drop
ALTER TABLE Sales.Orders DROP CONSTRAINT FK_Orders_Customers;

-- Unnamed constraint: must query sys.foreign_keys to find the generated name first
SELECT name FROM sys.foreign_keys WHERE parent_object_id = OBJECT_ID('Sales.Orders');
-- Then use the cryptic name in the DROP statement
```

Documentation: Named constraints are self-documenting. Reading `CK_LoyaltyTransactions_PointsDirection` tells a developer what the constraint governs. Reading `CK__LoyaltyTrans__Points__7D2E4F12` tells them nothing.

---

13. Discovering Constraints in the System Catalog

The system catalog contains complete constraint information for every object in the database. These queries are essential for understanding an existing schema.

13.1 All Constraints on a Table

```
-- Every constraint on Sales.Orders
SELECT
    cc.name         AS ConstraintName,
    cc.type_desc    AS ConstraintType,
    -- For check constraints, the definition
    CASE WHEN cc.type = 'C' THEN cc.definition ELSE NULL END AS CheckDefinition
FROM    sys.objects cc
WHERE   cc.parent_object_id = OBJECT_ID('Sales.Orders')
AND     cc.type IN ('PK', 'UQ', 'C', 'D', 'F')
ORDER BY cc.type_desc, cc.name;
```

13.2 All Foreign Keys in the Database

```
USE CabotTrailOutdoor;

SELECT
    fk.name                         AS FKName,
    SCHEMA_NAME(tp.schema_id) + '.' + tp.name AS ChildTable,
    cp.name                         AS ChildColumn,
    SCHEMA_NAME(tr.schema_id) + '.' + tr.name AS ParentTable,
    cr.name                         AS ParentColumn,
    fk.delete_referential_action_desc AS OnDelete,
    fk.update_referential_action_desc AS OnUpdate,
    fk.is_disabled                  AS IsDisabled
FROM    sys.foreign_keys fk
INNER JOIN sys.foreign_key_columns fkc
    ON fkc.constraint_object_id = fk.object_id
INNER JOIN sys.tables tp  ON tp.object_id = fkc.parent_object_id
INNER JOIN sys.tables tr  ON tr.object_id = fkc.referenced_object_id
INNER JOIN sys.columns cp ON cp.object_id = fkc.parent_object_id
                         AND cp.column_id  = fkc.parent_column_id
INNER JOIN sys.columns cr ON cr.object_id = fkc.referenced_object_id
                         AND cr.column_id  = fkc.referenced_column_id
ORDER BY SCHEMA_NAME(tp.schema_id), tp.name, fk.name;
```

13.3 All CHECK Constraints

```
SELECT
    SCHEMA_NAME(t.schema_id) + '.' + t.name    AS TableName,
    cc.name                                     AS ConstraintName,
    cc.definition                               AS CheckExpression,
    cc.is_disabled                              AS IsDisabled
FROM    sys.check_constraints cc
INNER JOIN sys.tables t ON t.object_id = cc.parent_object_id
ORDER BY TableName, ConstraintName;
```

13.4 All DEFAULT Constraints

```
SELECT
    SCHEMA_NAME(t.schema_id) + '.' + t.name    AS TableName,
    c.name                                      AS ColumnName,
    dc.name                                     AS ConstraintName,
    dc.definition                               AS DefaultValue
FROM    sys.default_constraints dc
INNER JOIN sys.columns c ON c.object_id = dc.parent_object_id
                        AND c.column_id  = dc.parent_column_id
INNER JOIN sys.tables  t ON t.object_id = dc.parent_object_id
ORDER BY TableName, ColumnName;
```

---

14. Constraints in the CabotTrail OLTP

Running the catalog queries above against `CabotTrailOutdoor` reveals the constraint design choices.

14.1 What the OLTP Enforces

```
-- Count constraints by type across the entire OLTP
SELECT
    o.type_desc     AS ConstraintType,
    COUNT(*)        AS Count
FROM    sys.objects o
INNER JOIN sys.tables t ON t.object_id = o.parent_object_id
INNER JOIN sys.schemas s ON s.schema_id = t.schema_id
WHERE   o.type IN ('PK', 'UQ', 'C', 'D', 'F')
AND     s.name IN ('Sales', 'Purchasing', 'Inventory', 'Application')
GROUP BY o.type_desc
ORDER BY o.type_desc;
```

Run this query. You will find:

- Primary key constraints on every table
- Foreign key constraints linking all related tables
- Check constraints on key business rules
- Default constraints on dates, flags, and numeric columns
- Unique constraints on candidate keys (email addresses, product codes)

14.2 Specific Constraint Examples

```
-- Find all CHECK constraints in the OLTP and read their business rules
SELECT
```

```
    SCHEMA_NAME(t.schema_id) + '.' + t.name    AS TableName,
    cc.name                                     AS ConstraintName,
    cc.definition                               AS BusinessRule
FROM    sys.check_constraints cc
INNER JOIN sys.tables t ON t.object_id = cc.parent_object_id
WHERE   SCHEMA_NAME(t.schema_id) IN ('Sales', 'Purchasing', 'Inventory', 'Application')
ORDER BY TableName, ConstraintName;
```

Each check constraint you see is a business rule the original designer encoded permanently in the schema. Reading them is a form of system documentation — the constraints tell you what the designer knew about valid data.

14.3 The Unknown Member Rows

Several tables in the CabotTrail OLTP contain a row with PK = 0 — the **unknown member** row:

```
-- Verify the unknown member pattern
SELECT CustomerID, CustomerName FROM Sales.Customers WHERE CustomerID = 0;
SELECT ProductID,  ProductName  FROM Inventory.Products WHERE ProductID = 0;
SELECT EmployeeID, FullName     FROM Application.Employees WHERE EmployeeID = 0;
```

These rows are placeholder records that FK columns can reference when the actual value is unknown or not applicable. They prevent the need for nullable FKs in cases where the relationship is conceptually mandatory but the specific value is temporarily unknown.

The CHECK constraint and NOT NULL constraint on these FK columns enforce that they always have a value — even if that value is the unknown member.

---

15. Chapter Summary

- **Constraints** enforce business rules at the storage layer — they are always active, immune to application bugs, and discoverable through the system catalog.
- **NOT NULL** enforces mandatory attributes. All columns should explicitly declare nullability. Adding NOT NULL to an existing column requires existing NULLs to be resolved first.
- **NULL** represents missing, unknown, or not-applicable values. It is not zero or empty string. NULL in comparisons produces UNKNOWN. Three-valued logic means CHECK constraints pass on NULL unless NOT NULL is also applied.
- **PRIMARY KEY** enforces uniqueness and NOT NULL; creates a clustered index by default; one per table.

- **IDENTITY(seed, increment)** auto-generates surrogate key values. Use `SCOPE_IDENTITY()` to retrieve the generated value. Gaps are normal and not a problem.
- **FOREIGN KEY** enforces referential integrity. SQL Server does not auto-index the FK column — always create a non-clustered index on FK columns manually.
- **Referential integrity actions** (`NO ACTION` , `CASCADE` , `SET NULL` , `SET DEFAULT`) define what happens when a referenced row changes. The choice is a business rule.
- **UNIQUE** enforces uniqueness on non-PK columns; allows NULLs; multiple per table; essential for enforcing 1:1 maximum cardinality via FK + UNIQUE combination.
- **CHECK** validates column values against a condition. Can span multiple columns in the same row; cannot reference other tables. NULL makes a CHECK pass (UNKNOWN is not FALSE) unless NOT NULL is also applied.
- **DEFAULT** provides a fallback value when none is supplied. Combined with NOT NULL it simplifies inserts while ensuring mandatory columns always have a value.
- **Computed columns** enforce derivation rules at the database layer. `PERSISTED` stores the result physically and enables indexing.
- **Always name constraints** — named constraints produce informative error messages and are trivially dropped by name.

---

16. Review Questions

1. A developer creates a `Promotions` table with this column: `DiscountPct DECIMAL(5,2)` . They want to ensure the discount is between 0% and 50%. Write the complete column definition (including data type, nullability, default, and constraint) that correctly enforces this rule.
2. Explain why the following CHECK constraint does not reject NULL values for `ExpiryDate` , and write the corrected DDL that ensures `ExpiryDate` is always a date on or after `IssueDate` and is never NULL:

```
ExpiryDate DATE,
CONSTRAINT CK_Permits_Expiry CHECK (ExpiryDate > IssueDate)
```

3. The `Sales.LoyaltyAccounts` table has a FK from `CustomerID` to `Sales.Customers`. Describe what happens in each of the following scenarios under the default `NO ACTION` referential behaviour, and then explain which referential action (`CASCADE`, `SET NULL`, or `SET DEFAULT`) you would choose for each and why: a. A customer record is deleted b. A customer's `CustomerID` is updated

4. Write the complete DDL for a `Suppliers` table that includes: a surrogate PK, a supplier name (required), a contact email (unique if provided), a country code (must be one of 'CA', 'US', 'GB', 'DE', 'CN'), a credit terms value (must be 30, 60, or 90 days), and a created date (defaults to today). Name all constraints.

5. Run the CHECK constraint catalog query from section 13.3 against `CabotTrailOutdoor`. Find three CHECK constraints and for each: (a) state the table and column it applies to, (b) translate the check expression into a plain English business rule, (c) describe a specific INSERT that would be rejected.

6. The `Sales.Orders` table currently has `SalesRepID INT NULL` with no index on that column. Explain why this is a performance risk, and write the SQL to add both an appropriate index and a referential integrity constraint (assuming `SalesRepID` references `Application.Employees.EmployeeID` and a sales rep may or may not be assigned).

7. Write a computed column definition for `dbo.OrderSummary` that calculates `GrossMarginPct` from `TotalRevenue` and `TotalCost`, handling the divide-by-zero case. Should this column be PERSISTED? Justify your decision based on the likely query patterns for a summary table.

8. A colleague argues: "CHECK constraints are redundant — the application validates all inputs before they reach the database." Write a specific scenario from the CabotTrail context that demonstrates why this argument is wrong and why a database-level CHECK constraint would catch an error that application validation would miss.

🔍 Deeper Dive

Going Further with Constraints and Data Integrity

Deferred Constraint Checking

SQL Server enforces all constraints **immediately** — when an INSERT, UPDATE, or DELETE statement is executed, every constraint is checked before the statement completes. If any constraint is violated, the statement is rolled back.

Other database systems (PostgreSQL, Oracle) support **deferrable constraints** — constraints that can be checked at the end of a transaction rather than at the end of each statement. This is useful for loading data where temporary constraint violations within a transaction are acceptable (e.g., loading a set of rows where FK references within the batch are resolved by the end of the batch but not at each individual INSERT).

SQL Server does not support deferrable constraints. The workaround: load data in the correct order (parent tables before child tables), or temporarily disable FK constraints during bulk loads and re-enable them after:

```
-- Disable FK checking during bulk load
ALTER TABLE Sales.Orders NOCHECK CONSTRAINT FK_Orders_Customers;

-- [Bulk load operation]

-- Re-enable and verify all constraints
ALTER TABLE Sales.Orders WITH CHECK CHECK CONSTRAINT FK_Orders_Customers;
-- WITH CHECK re-enables AND validates existing data
-- WITHOUT: ALTER TABLE Sales.Orders CHECK CONSTRAINT -- re-enables but doesn't validate
existing rows
```

The `WITH CHECK CHECK` syntax (not a typo — `WITH CHECK` is the option; `CHECK CONSTRAINT` is the command) validates all existing rows against the constraint after re-enabling it. If existing data violates the constraint, the command fails. Use this to ensure data integrity after bulk loads.

Microsoft documentation: ALTER TABLE — Enabling Constraints

Assertion Constraints (What SQL Server Lacks)

Standard SQL defines **assertion** constraints — named constraints that can span multiple tables and reference the entire database state:

```
-- Hypothetical standard SQL assertion (not supported in SQL Server)
CREATE ASSERTION Total_Revenue_Positive
    CHECK (
        (SELECT SUM(LineTotal) FROM Sales.InvoiceLines) >= 0
    );
```

SQL Server does not support assertions. When a constraint needs to check conditions across multiple tables (e.g., "an order's total cannot exceed the customer's credit limit"), the options in SQL Server are:

1. **Application-layer validation** — check in the application before inserting
2. **Stored procedure** — encapsulate the logic in a stored procedure that performs the validation before the DML
3. **INSTEAD OF trigger** — a trigger that intercepts the INSERT/UPDATE and performs the cross-table check

None of these is as clean as a database-level assertion, which is why assertions are included in the SQL standard. SQL Server's lack of support is a known gap.

Row-Level Security as a Constraint

SQL Server's **Row-Level Security (RLS)** is sometimes described as a form of constraint — it limits which rows a user can see or modify based on their identity. While not a traditional constraint, RLS enforces data access rules at the database layer:

```
-- Security predicate: only show rows matching the current user's territory
CREATE FUNCTION Security.fn_TerritoryFilter(@SalesTerritory NVARCHAR(60))
RETURNS TABLE WITH SCHEMABINDING AS
RETURN
    SELECT 1 AS IsAllowed
    WHERE @SalesTerritory = (
        SELECT UserTerritory
        FROM   Security.UserTerritoryMapping
        WHERE  Username = USER_NAME()
    );

CREATE SECURITY POLICY TerritoryPolicy
    ADD FILTER PREDICATE Security.fn_TerritoryFilter(SalesTerritory)
    ON dim.Customer;
```

The ETL book's Chapter 5 covers RLS in detail. For database designers, understanding that access control can be encoded at the database layer (rather than only in application code) is an important design principle.

Microsoft documentation: Row-Level Security

References and Further Reading

1. Date, C. J. (2019). *Database Design and Relational Theory: Normal Forms and All That Jazz* (2nd ed.). Apress. — Chapter 10 covers constraints in the context of the relational model's integrity rules.

2. Connolly, T., & Begg, C. (2015). *Database Systems: A Practical Approach to Design, Implementation, and Management* (6th ed.). Pearson. — Chapter 4 covers integrity constraints systematically; Chapter 6 covers SQL DDL for constraints.

3. Ben-Gan, I. (2015). *T-SQL Fundamentals* (3rd ed.). Microsoft Press. — Chapters 11–12 cover DDL including constraints, computed columns, and sequence objects.

4. Microsoft. (2024). *PRIMARY AND FOREIGN KEY Constraints*. https://learn.microsoft.com/en-us/sql/relational-databases/tables/primary-and-foreign-key-constraints

5. Microsoft. (2024). *Unique Constraints and Check Constraints*. https://learn.microsoft.com/en-us/sql/relational-databases/tables/unique-constraints-and-check-constraints

6. Microsoft. (2024). *Specify Computed Columns in a Table*. https://learn.microsoft.com/en-us/sql/relational-databases/tables/specify-computed-columns-in-a-table

7. Microsoft. (2024). *IDENTITY (Property)*. https://learn.microsoft.com/en-us/sql/t-sql/statements/create-table-transact-sql-identity-property

8. Microsoft. (2024). *Row-Level Security*. https://learn.microsoft.com/en-us/sql/relational-databases/security/row-level-security

---

> ***Database Design for Analysts*** *| © Patrick Dolinger, NSCC Institute of Technology CC BY 4.0 — Share and adapt freely with attribution*

Chapter 7: Indexes and Physical Design — Making Queries Fast

> ***Database Design for Analysts*** *From business requirements to relational structures*
>
> *© Patrick Dolinger, NSCC Institute of Technology Licensed under Creative Commons Attribution 4.0 International (CC BY 4.0) You are free to share and adapt this material for any purpose, provided appropriate credit is given.*

Chapter Overview

The previous five chapters addressed the *logical* design of a database — what tables to create, how to normalise them, and what constraints to apply. This chapter shifts focus to the *physical* design — how the database stores and retrieves data, and how that physical organisation affects query performance.

At the heart of physical design is the **index**. An index is a separate data structure that SQL Server maintains alongside a table to enable fast lookups. Without indexes, every query that filters or joins must read every row of every table involved — a **full table scan**. For a table with a million rows, a full scan of every query is unacceptably slow. Indexes allow SQL Server to find qualifying rows without examining every row.

Understanding indexes is essential for both database designers and analysts. Designers create the indexes that make the schema perform well. Analysts write queries that either benefit from existing indexes or inadvertently prevent them from being used.

By the end of this chapter you will be able to:

- Explain how SQL Server stores table data using pages and the B-tree structure
- Describe the difference between a clustered and non-clustered index
- Choose the appropriate clustered index key for a table
- Design non-clustered indexes for common query patterns
- Explain covering indexes and when to use them
- Read and interpret a basic execution plan
- Identify queries that cannot use indexes (non-sargable conditions)
- Apply index design principles to the CabotTrail schema
- Explain the trade-offs between more indexes and write performance

Table of Contents

1. How SQL Server Stores Data
2. The B-Tree Index Structure
3. Clustered Indexes
4. Choosing the Clustered Index Key
5. Non-Clustered Indexes
6. Covering Indexes and Included Columns
7. Index Design for Common Query Patterns
8. The Execution Plan
9. Non-Sargable Conditions: When Indexes Cannot Help
10. Indexes and Write Performance
11. Index Maintenance
12. Composite Indexes and Column Order
13. Indexes in the CabotTrail Schema
14. Chapter Summary
15. Review Questions
16. 🔍 Deeper Dive

1. How SQL Server Stores Data

Before understanding indexes, you need a mental model of how SQL Server stores data physically. This model explains why some queries are fast and others are slow.

1.1 Pages: The Unit of Storage

SQL Server stores all data in **pages** — fixed-size blocks of 8 kilobytes (8,192 bytes). Every read and write operation works at the page level, not the row level. When SQL Server needs to read one row, it reads the entire 8 KB page containing that row.

A typical data row in the CabotTrail `fact.Sales` table uses roughly 200–300 bytes. An 8 KB page holds approximately 25–40 rows. The 16,359 rows in `fact.Sales` span roughly 400–650 pages.

1.2 Extents: Groups of Pages

Eight consecutive pages form an **extent** (64 KB). SQL Server allocates and reads data in extents during large scans, which is more efficient than reading individual pages.

1.3 The Table Scan

When no index is available for a query's filter condition, SQL Server reads every page of the table from beginning to end — a **table scan** (or **heap scan** for unordered tables). For a 400-page table, this means 400 page reads minimum. For a 1-million-page table, it means 1 million page reads.

A table scan is not always bad — for small tables or queries that need most rows, it can be the right choice. The optimizer chooses between scans and index seeks based on estimated row counts and costs.

---

2. The B-Tree Index Structure

SQL Server indexes use a **Balanced Tree (B-tree)** structure — a hierarchical structure where every lookup starts at the root and follows a path down to the leaf level. The "balanced" property means every leaf is at the same depth, guaranteeing predictable lookup time regardless of which value is searched.

2.1 The Three Levels



- **Root page:** The top of the tree. Contains the highest key value on each sub-page, pointing to the next level.
- **Intermediate pages:** Navigation level(s) between root and leaves.
- **Leaf pages:** The bottom level. Contains the actual data (for clustered indexes) or row pointers (for non-clustered indexes).

2.2 The Lookup Process

Finding CustomerID = 42:

1. Read root page: "42 is between 26 and 50, follow the pointer to Page D"

2. Read intermediate page D: "42 is between 38 and 50, follow the pointer to leaf page G"
3. Read leaf page G: "Row with CustomerID = 42 found"

Three page reads to find one row out of a million. Contrast with a table scan: a million page reads for a million-row table. The B-tree is the mechanism that makes indexes fast.

---

3. Clustered Indexes

A **clustered index** determines the physical order of rows in the table. The leaf level of a clustered index *is* the table data — the rows are stored in the order of the index key.

3.1 One Per Table

A table can have **exactly one** clustered index, because the rows can only be physically sorted one way. When you define a PRIMARY KEY constraint, SQL Server creates a clustered index on those columns by default.

3.2 What Happens Without a Clustered Index

A table without a clustered index is called a **heap** — rows are stored in insertion order with no organisation. Lookups on a heap always require a full scan (unless a non-clustered index is present). For most tables, a clustered index is preferable to a heap.

3.3 Clustered Index Seeks vs Scans

A **clustered index seek** navigates the B-tree to find specific rows — efficient for equality searches and narrow range searches.

A **clustered index scan** reads all leaf pages of the clustered index — equivalent to a table scan but organised by key order. Efficient when the query needs most rows; inefficient when it needs few rows.

```
-- This benefits from a clustered index seek on CustomerID
SELECT CustomerName FROM Sales.Customers WHERE CustomerID = 42;

-- This performs a clustered index scan (needs all rows)
SELECT CustomerName FROM Sales.Customers ORDER BY CustomerName;
```

---

4. Choosing the Clustered Index Key

Because the clustered index determines physical row order, the choice of clustered index key significantly affects both query performance and storage efficiency.

4.1 Properties of a Good Clustered Index Key

Narrow: The clustered index key is stored in every non-clustered index as a row locator. A wide key (many columns, or wide data types) inflates every non-clustered index's size.

Unique: If the clustered key is not unique, SQL Server adds a 4-byte uniquifier to every duplicate, adding storage overhead and complicating lookups.

Static: Changing a clustered index key value requires moving the row to its new sorted position — an expensive operation. Keys that frequently change (email addresses, names) make poor clustered index keys.

Ever-increasing: Keys that always increase (IDENTITY integers, sequential GUIDs) cause new rows to be appended at the end of the table — efficient. Random keys (standard GUIDs, hash values) cause pages to split when new rows are inserted between existing ones — creating **page splits** that fragment the index.

4.2 Why IDENTITY Is an Excellent Clustered Key

IDENTITY integers satisfy all four properties:

- **Narrow:** 4 bytes (INT)
- **Unique:** Each value is unique by definition
- **Static:** Surrogate keys never change
- **Ever-increasing:** New values are always higher than existing ones

This is the strongest argument for surrogate IDENTITY keys — beyond their role as stable identifiers, they produce excellent clustered index performance.

4.3 When to Choose a Different Clustered Key

Sometimes the most common query pattern suggests a different clustered key than the PK:

```
A table frequently queried by date range:
    WHERE OrderDate BETWEEN '2024-01-01' AND '2024-03-31'
```

If most queries retrieve rows by `OrderDate`, clustering by `OrderDate` means that all rows for a given date range are physically adjacent on disk — a **range scan** retrieves them in one sequential read. Clustering by `OrderID` (IDENTITY) means date-range rows are scattered across the entire table.

For such tables, consider creating the clustered index on `OrderDate` and adding a separate unique non-clustered index on the PK (`OrderID`). This is a non-default choice that requires explicit DDL:

```
-- Cluster on OrderDate instead of OrderID
CREATE TABLE Sales.Orders
(
    OrderID     INT         NOT NULL IDENTITY(1,1),
    OrderDate   DATE        NOT NULL,
    CustomerID  INT         NOT NULL,
    ...
);

-- Create clustered index on OrderDate (not the PK)
CREATE CLUSTERED INDEX CX_Orders_OrderDate
    ON Sales.Orders (OrderDate);

-- Create unique non-clustered index to enforce PK uniqueness
CREATE UNIQUE NONCLUSTERED INDEX UX_Orders_OrderID
    ON Sales.Orders (OrderID);
```

This is an advanced design choice. For most tables, the default (clustered index on the surrogate PK) is correct and simple.

---

5. Non-Clustered Indexes

A **non-clustered index** is a separate B-tree structure that contains the index key columns and a pointer back to the table row. It does not change the physical order of the table.

5.1 Structure

The leaf level of a non-clustered index contains:

- The index key column(s)
- A **row locator**: for a heap, this is a physical row address; for a table with a clustered index, this is the clustered index key value

When SQL Server finds a matching row in the non-clustered index, it uses the row locator to retrieve the full row from the clustered index (or heap) — a **key lookup**.

5.2 Creating a Non-Clustered Index

```
-- Index on a single column
CREATE NONCLUSTERED INDEX IX_Orders_CustomerID
    ON Sales.Orders (CustomerID);

-- Index on a single column with explicit name and fill factor
CREATE NONCLUSTERED INDEX IX_Orders_OrderDate
    ON Sales.Orders (OrderDate)
    WITH (FILLFACTOR = 80);  -- Leave 20% of each page empty for future inserts
```

5.3 When Non-Clustered Indexes Are Used

SQL Server uses a non-clustered index when:

1. The query filters on the index key column(s): `WHERE CustomerID = 42`
2. The estimated number of qualifying rows is small enough that a seek+lookup is faster than a scan

For large tables with selective filters (few rows per filter value), non-clustered indexes are essential. For filters that return most rows, the optimizer may prefer a scan even with an index available.

---

6. Covering Indexes and Included Columns

6.1 The Key Lookup Problem

After a non-clustered index seek, SQL Server must retrieve the full row from the clustered index to return any columns not in the non-clustered index. This **key lookup** is an additional I/O operation per row found — for large result sets, it negates the index's benefit.

```
-- Index on CustomerID only
CREATE NONCLUSTERED INDEX IX_Orders_CustomerID ON Sales.Orders (CustomerID);

-- Query: uses the index to find rows, then key-looks up SalesRepID and OrderDate
SELECT CustomerID, SalesRepID, OrderDate
FROM   Sales.Orders
WHERE  CustomerID = 42;
-- Plan: Index Seek (IX_Orders_CustomerID) → Key Lookup (clustered) for SalesRepID,
OrderDate
```

For a customer with 500 orders, this means 500 key lookups in addition to the index seek.

6.2 Covering Indexes

A **covering index** includes all the columns a query needs, so SQL Server can answer the query entirely from the index without touching the main table.

```
-- Covering index: includes all columns the query needs
CREATE NONCLUSTERED INDEX IX_Orders_CustomerID_Covering
    ON Sales.Orders (CustomerID)
    INCLUDE (SalesRepID, OrderDate);
-- Now the query above can be answered from the index alone — no key lookups
```

The `INCLUDE` clause adds columns to the leaf level of the index without including them in the key (which would change the sort order and index size). The result: the index is larger but the query avoids key lookups.

6.3 When to Use Covering Indexes

Covering indexes are most valuable for:

- Frequently executed queries on large tables
- Queries that return a small number of columns for a filtered set of rows
- Reports that run on a fixed set of columns for a common filter

The cost: each additional `INCLUDE` column adds to the index size and slows INSERT/UPDATE/DELETE operations (more indexes to maintain).

---

7. Index Design for Common Query Patterns

7.1 The Index Design Process

Good index design starts from query patterns — the actual queries that will run against the table — not from theoretical principles. The process:

1. Identify the most frequent and most performance-critical queries
2. For each query, identify the WHERE clause columns (filter), JOIN columns, and SELECT columns
3. Design an index with the most selective filter columns first in the key
4. Add additional columns to the key if they help with range scans or ordering
5. Add `INCLUDE` columns if they eliminate key lookups for high-frequency queries

7.2 Selectivity: The Key Concept

Selectivity is the proportion of rows returned by a filter condition. A highly selective filter returns few rows; a low-selectivity filter returns many rows.

- `WHERE CustomerID = 42` — highly selective (one customer out of 100)
- `WHERE IsDiscontinued = 0` — low selectivity (most products are active)

Indexes provide the greatest benefit for high-selectivity conditions. An index on `IsDiscontinued` has limited value because most queries using it will retrieve a large fraction of the table — a scan may be just as fast.

```
-- High value: index on CustomerID (1 of 100 customers — high selectivity)
CREATE NONCLUSTERED INDEX IX_Orders_CustomerID ON Sales.Orders (CustomerID);

-- Moderate value: index on OrderDate (useful for date range queries)
CREATE NONCLUSTERED INDEX IX_Orders_OrderDate ON Sales.Orders (OrderDate);

-- Low value: index on IsDiscontinued (BIT — only 2 possible values)
-- SQL Server will often ignore this index in favour of a scan
-- CREATE NONCLUSTERED INDEX IX_Products_IsDiscontinued -- probably not worth it
```

7.3 A Standard Index Set for Fact Tables

For a fact table like `fact.Sales`, the most important indexes are on the FK columns (used in JOINs) and on the date key (used in filters):

```
-- Indexes for CabotTrailOutdoorsSales.fact.Sales
-- (Clustered index on SalesKey already created by PRIMARY KEY)

-- FK indexes (essential for JOIN performance)
CREATE NONCLUSTERED INDEX IX_FactSales_CustomerKey
    ON fact.Sales (CustomerKey);

CREATE NONCLUSTERED INDEX IX_FactSales_ProductKey
    ON fact.Sales (ProductKey);

CREATE NONCLUSTERED INDEX IX_FactSales_EmployeeKey
    ON fact.Sales (EmployeeKey);

-- Date key index (essential for date range filters)
CREATE NONCLUSTERED INDEX IX_FactSales_OrderDateKey
    ON fact.Sales (OrderDateKey)
    INCLUDE (CustomerKey, ProductKey, LineTotal, GrossProfit);
-- INCLUDE columns cover the most common analytical query pattern:
-- filter by date, join to Customer and Product, sum LineTotal and GrossProfit
```

---

8. The Execution Plan

The **execution plan** is SQL Server's recipe for executing a query — which tables to read, in what order, which indexes to use, and how to combine results. Reading execution plans is the primary tool for understanding and improving query performance.

8.1 Viewing an Execution Plan in SSMS

Two options in SSMS:

- **Estimated plan** (`Ctrl+L` or Query menu → Display Estimated Execution Plan): Shows the plan without running the query
- **Actual plan** (`Ctrl+M` toggles inclusion, then run with `F5`): Shows the plan with actual row counts from the execution

For performance analysis, the actual plan is more informative — estimated row counts can be wrong when statistics are stale.

8.2 Key Plan Operators

Operator	What it means	Performance implication
Clustered Index Seek	Navigated B-tree to find specific rows	Efficient — few page reads
Clustered Index Scan	Read all pages of the clustered index	Expensive for large tables
Non-Clustered Index Seek	Navigated non-clustered B-tree	Efficient for selective queries
Key Lookup	Retrieved full row from clustered index after non-clustered seek	Extra I/O per row — consider covering index
Hash Match	Joined two inputs using a hash table	Memory-intensive; common for large joins
Nested Loops	For each row in outer input, searched inner input	Efficient when outer is small
Merge Join	Joined two sorted inputs simultaneously	Efficient when both inputs are sorted
Sort	Sorted input rows	Expensive; breaks streaming; indicates missing index
Table Spool	Spooled intermediate results to tempdb	Memory pressure; complex query

8.3 Reading Plan Cost

Each operator in the execution plan has an estimated cost (as a percentage of total query cost). Operators with high percentages are the bottlenecks — candidates for index improvement.

```
-- Enable actual execution plan, then run this query
USE CabotTrailOutdoorsSales;

SELECT
    prod.CategoryName,
    SUM(fs.LineTotal)   AS Revenue
FROM    fact.Sales fs
INNER JOIN dim.Product prod ON prod.ProductKey = fs.ProductKey
GROUP BY prod.CategoryName
ORDER BY Revenue DESC;

-- Look for: which table is scanned? Is there a Key Lookup?
-- Does it use an index seek or scan on fact.Sales?
```

8.4 The Missing Index Hint

When SQL Server determines that a query could benefit from an index that does not exist, it adds a **missing index suggestion** to the execution plan — shown as a green "Missing Index" banner at the top of the plan. The suggestion shows the exact `CREATE INDEX` statement to run.

Missing index suggestions are valuable but should not be applied blindly:

- Each suggested index has a cost (storage, write overhead)
- Multiple similar suggestions may be consolidatable into one index
- The query optimizer suggests indexes per-query; a database with many queries may have hundreds of suggestions, not all of which are worth implementing

---

9. Non-Sargable Conditions: When Indexes Cannot Help

Chapter 2 of *SQL for Analysts* and the Deeper Dive of Chapter 2 introduced **sargability**. This section deepens that concept from the designer's perspective.

9.1 What Non-Sargable Means

A **sargable** condition (Search ARGument ABLE) is one that can be resolved using an index — typically a direct comparison on a column. A **non-sargable** condition applies a function or expression to the column, forcing SQL Server to evaluate the expression for every row rather than using the B-tree.

9.2 Non-Sargable Patterns

```
-- NON-SARGABLE: function applied to column — full scan required
WHERE YEAR(OrderDate) = 2024
WHERE UPPER(CustomerName) = 'FUNDY BAY'
WHERE LEN(ProductCode) > 5

-- SARGABLE equivalents: comparison directly against column value
WHERE OrderDate >= '2024-01-01' AND OrderDate < '2025-01-01'
WHERE CustomerName = 'Fundy Bay'   -- (relies on case-insensitive collation)
WHERE ProductCode > 'AAAAA'        -- (approximate length filter via range)
```

9.3 The Designer's Responsibility

As a database designer, you can help analysts write sargable queries by:

Storing computed values as separate columns. If analysts frequently filter by year, add a `CalendarYear` column (or use the Calendar dimension) rather than making them apply `YEAR()` to a date column.

Choosing data types that support direct comparison. Storing dates as `DATE` (not `NVARCHAR`) allows `>= '2024-01-01'` to use a date index. Storing numbers as `DECIMAL` (not `NVARCHAR`) allows `> 100` to use a numeric index.

Avoiding case-sensitive collations for columns that will be searched with equality. Case-insensitive collation (the SQL Server default) allows `WHERE Name = 'value'` to be sargable without `UPPER()` transformation.

9.4 Computed Persisted Columns as an Index Workaround

If a non-sargable expression is frequently used in WHERE clauses, a **computed persisted column** plus an index on it provides a sargable alternative:

```
-- Add a persisted computed column for the year
ALTER TABLE Sales.Orders
ADD OrderYear AS YEAR(OrderDate) PERSISTED;

-- Index the computed column
CREATE NONCLUSTERED INDEX IX_Orders_OrderYear
    ON Sales.Orders (OrderYear);

-- Now this query can use the index
WHERE OrderYear = 2024   -- Sargable — directly comparing column value
```

The query `WHERE YEAR(OrderDate) = 2024` is still non-sargable. But `WHERE OrderYear = 2024` uses the index on the pre-computed column.

10. Indexes and Write Performance

Every index is a trade-off. Indexes accelerate reads; they add overhead to writes.

10.1 The Write Overhead

When a row is inserted, updated, or deleted, SQL Server must maintain every index on that table:

- **INSERT:** Add the new row's key value to every index. If a page split occurs in any index (the target page is full), SQL Server allocates a new page and redistributes the rows — expensive.
- **UPDATE:** If the updated column is part of an index key, update the index (remove the old key, insert the new key). If the updated column is only in `INCLUDE`, update the leaf-level entry.
- **DELETE:** Remove the row's key value from every index.

A table with 10 non-clustered indexes requires 11 B-tree updates for every single-row insert (1 clustered + 10 non-clustered).

10.2 The Index Design Trade-Off

```
More indexes → faster reads, slower writes, more storage
Fewer indexes → slower reads, faster writes, less storage
```

The right balance depends on the access pattern:

- **OLTP tables** (many writes, many targeted reads): conservative indexing — essential indexes only (PK, FKs, high-selectivity filter columns)
- **OLAP/DW fact tables** (few writes — nightly load — many reads): generous indexing — cover common query patterns aggressively

10.3 Filtering During Bulk Loads

When bulk-loading a large table (as in the CabotTrail ETL), temporarily disabling non-clustered indexes before the load and rebuilding them after is significantly faster than maintaining them row-by-row during the load:

```
-- Before bulk load: disable non-clustered indexes
ALTER INDEX IX_FactSales_CustomerKey ON fact.Sales DISABLE;
ALTER INDEX IX_FactSales_ProductKey  ON fact.Sales DISABLE;

-- [Bulk load operation]
```

```
-- After bulk load: rebuild (not re-enable — REBUILD also reorganises the pages)
ALTER INDEX IX_FactSales_CustomerKey ON fact.Sales REBUILD;
ALTER INDEX IX_FactSales_ProductKey  ON fact.Sales REBUILD;
```

---

11. Index Maintenance

Indexes degrade over time as rows are inserted, updated, and deleted. **Fragmentation** occurs when the logical order of index pages diverges from the physical order on disk, or when pages have excessive empty space.

11.1 Types of Fragmentation

Logical fragmentation: Index pages are not in sequential physical order. A range scan must hop between non-sequential pages rather than reading ahead.

Page density (internal fragmentation): Pages have significant empty space due to deletions or page splits. SQL Server reads more pages than necessary because each page holds fewer rows.

11.2 Detecting Fragmentation

```
-- Check fragmentation for all indexes on fact.Sales
SELECT
    i.name                      AS IndexName,
    s.avg_fragmentation_in_percent,
    s.avg_page_space_used_in_percent,
    s.page_count
FROM    sys.dm_db_index_physical_stats(
            DB_ID('CabotTrailOutdoorsSales'),
            OBJECT_ID('fact.Sales'),
            NULL, NULL, 'LIMITED') s
INNER JOIN sys.indexes i
    ON i.object_id = s.object_id AND i.index_id = s.index_id
WHERE   s.index_id > 0   -- Exclude heap
ORDER BY s.avg_fragmentation_in_percent DESC;
```

11.3 Fragmentation Thresholds and Actions

Fragmentation	Action	SQL command
< 5%	No action needed	—
5% – 30%	Reorganise (online, minimal locking)	`ALTER INDEX ... REORGANIZE`
> 30%	Rebuild (offline or online with Enterprise edition)	`ALTER INDEX ... REBUILD`

```
-- Reorganise a fragmented index (light maintenance)
ALTER INDEX IX_FactSales_CustomerKey ON fact.Sales REORGANIZE;

-- Rebuild an index (full defragmentation, updates statistics)
ALTER INDEX IX_FactSales_CustomerKey ON fact.Sales REBUILD;

-- Rebuild all indexes on a table
ALTER INDEX ALL ON fact.Sales REBUILD;
```

Index maintenance is typically run as a scheduled SQL Server Agent job during off-peak hours — part of the ETL maintenance window described in Chapter 8 of the ETL textbook.

---

12. Composite Indexes and Column Order

When an index spans multiple columns, **column order matters** — it determines which queries can use the index and which cannot.

12.1 The Leftmost Prefix Rule

A composite index can be used by queries that filter on the **leftmost prefix** of the index key — a contiguous set of columns starting from the leftmost. Queries that skip the leftmost column(s) cannot use the index efficiently.

```
-- Composite index: (SalesTerritory, CalendarYear, CategoryName)
CREATE NONCLUSTERED INDEX IX_Sales_Composite
    ON fact.Sales (SalesTerritory, CalendarYear, CategoryName);

-- CAN use this index (matches leftmost prefix):
WHERE SalesTerritory = 'Atlantic – Nova Scotia'                          -- prefix: 1
col
WHERE SalesTerritory = 'Atlantic – Nova Scotia' AND CalendarYear = 2024  -- prefix: 2
cols
WHERE SalesTerritory = 'Atlantic – Nova Scotia' AND CalendarYear = 2024
      AND CategoryName = 'Sleeping'                                       -- prefix: 3
cols

-- CANNOT use this index efficiently (skips leftmost column):
WHERE CalendarYear = 2024                   -- SalesTerritory is missing
WHERE CategoryName = 'Sleeping'             -- Both SalesTerritory and CalendarYear
missing
```

12.2 Choosing Column Order

Put the **most selective column first** when the index is used for equality searches — this narrows the range as quickly as possible.

Put the **range predicate column last** (among the key columns) for range searches — a range on an early column prevents the index from being useful for subsequent equality columns.

```
-- Good: equality columns before range column
-- Query: WHERE CustomerKey = 15 AND OrderDateKey BETWEEN 20240101 AND 20241231
CREATE NONCLUSTERED INDEX IX_FactSales_Cust_Date
    ON fact.Sales (CustomerKey, OrderDateKey);
-- CustomerKey equality narrows to one customer's rows; OrderDateKey range scans within
those

-- Poor: range column before equality column (usually)
-- Query: WHERE OrderDateKey BETWEEN 20240101 AND 20241231 AND CustomerKey = 15
-- An index on (OrderDateKey, CustomerKey) would find all 2024 rows,
-- then filter by CustomerKey — less efficient than customer first
```

---

13. Indexes in the CabotTrail Schema

13.1 Querying Existing Indexes

```
-- All indexes in CabotTrailOutdoorsSales
SELECT
    s.name                  AS SchemaName,
    t.name                  AS TableName,
    i.name                  AS IndexName,
    i.type_desc             AS IndexType,
    i.is_unique,
    i.is_primary_key,
    -- Key columns
    STRING_AGG(
        CASE WHEN ic.is_included_column = 0
             THEN c.name + ' (' + CASE ic.is_descending_key WHEN 1 THEN 'DESC' ELSE
'ASC' END + ')'
             ELSE NULL
        END,
        ', '
    ) WITHIN GROUP (ORDER BY ic.key_ordinal)        AS KeyColumns,
    -- Included columns
    STRING_AGG(
        CASE WHEN ic.is_included_column = 1
             THEN c.name
             ELSE NULL
        END,
        ', '
    ) WITHIN GROUP (ORDER BY ic.index_column_id)    AS IncludedColumns
FROM    sys.indexes i
INNER JOIN sys.tables t     ON t.object_id  = i.object_id
INNER JOIN sys.schemas s    ON s.schema_id  = t.schema_id
INNER JOIN sys.index_columns ic ON ic.object_id = i.object_id
                               AND ic.index_id   = i.index_id
```

```
INNER JOIN sys.columns c    ON c.object_id  = ic.object_id
                           AND c.column_id  = ic.column_id
WHERE   i.type > 0
AND     s.name IN ('fact', 'dim')
GROUP BY s.name, t.name, i.name, i.type_desc, i.is_unique, i.is_primary_key
ORDER BY s.name, t.name, i.name;
```

13.2 What Good Index Coverage Looks Like

For `CabotTrailOutdoorsSales.fact.Sales`, a well-indexed configuration would include:

Index	Type	Key columns	Purpose
`PK_Sales`	Clustered	`SalesKey`	Row identity and physical order
`IX_Sales_OrderDateKey`	Non-clustered	`OrderDateKey`	Date range queries
`IX_Sales_CustomerKey`	Non-clustered	`CustomerKey`	JOIN to dim.Customer
`IX_Sales_ProductKey`	Non-clustered	`ProductKey`	JOIN to dim.Product
`IX_Sales_EmployeeKey`	Non-clustered	`EmployeeKey`	JOIN to dim.Employee
`IX_Sales_DateKey_Covering`	Non-clustered	`OrderDateKey, CustomerKey` INCLUDE `LineTotal, GrossProfit`	Common analytical pattern

13.3 Index Gaps in a Fresh Schema

A freshly created schema typically has only the PK clustered index. The FK indexes and covering indexes described above must be added as part of the physical design step. This is why physical design (this chapter) is a distinct step from logical design (normalization, constraints):

1. Create tables with correct structure and constraints (Chapters 3–6)
2. Add indexes based on expected query patterns (this chapter)
3. Load data and run representative queries
4. Use execution plans to identify remaining bottlenecks
5. Add targeted indexes for identified bottlenecks

---

14. Chapter Summary

- SQL Server stores data in **8 KB pages**. Index lookups read a small number of pages; table scans read all pages.
- **B-tree indexes** use a balanced tree structure to enable fast lookups: navigate from root to leaf in O(log n) page reads.
- A **clustered index** determines the physical row order of the table. Tables can have only one. The default is the PRIMARY KEY. An ideal clustered key is narrow, unique, static, and ever-increasing — `IDENTITY INT` satisfies all four.
- A **non-clustered index** is a separate B-tree with a row pointer (key lookup) to the clustered index. Multiple non-clustered indexes can exist per table.
- A **covering index** adds `INCLUDE` columns to the leaf level, allowing queries to be answered entirely from the index without key lookups.
- **Index design** starts from query patterns: most selective filter columns first in the key, range columns last among key columns, cover frequently queried columns with INCLUDE.
- The **execution plan** (`Ctrl+L` estimated, `Ctrl+M` actual) shows the operators SQL Server uses. Key operators: Index Seek (fast), Index Scan (expensive), Key Lookup (consider covering), Sort (missing index or ORDER BY mismatch).
- **Non-sargable conditions** (functions on columns in WHERE) prevent index use. Design schemas and query patterns to keep WHERE conditions sargable.
- **Indexes cost write performance.** Every INSERT/UPDATE/DELETE must maintain all indexes. OLTP tables: conservative indexing. OLAP/DW tables: generous indexing. Disable and rebuild during bulk loads.
- **The leftmost prefix rule**: composite indexes are usable for queries filtering on any contiguous prefix of the key, starting from the leftmost column.

---

15. Review Questions

1. Explain the difference between a clustered index scan and a non-clustered index seek with key lookup. Under what circumstances would the optimizer choose the scan even when an index exists?

2. The `dim.Customer` table has a clustered index on `CustomerKey` (the surrogate PK). A common analytical query filters by `SalesTerritory`:

   ```
   SELECT CustomerName, CreditLimit FROM dim.Customer WHERE SalesTerritory = 'Atlantic
   — Nova Scotia';
   ```

   What index would improve this query? Write the complete `CREATE INDEX` statement, including `INCLUDE` columns if appropriate.

3. You create the following composite index on `fact.Sales`:

   ```
   CREATE NONCLUSTERED INDEX IX_Sales_Combo ON fact.Sales (OrderDateKey, CustomerKey);
   ```

   For each of the following queries, state whether this index can be used efficiently and explain why: a. `WHERE OrderDateKey = 20240315` b. `WHERE CustomerKey = 15` c. `WHERE OrderDateKey = 20240315 AND CustomerKey = 15` d. `WHERE OrderDateKey BETWEEN 20240101 AND 20241231 AND CustomerKey = 15`

4. A fact table has no non-clustered indexes. An analyst runs a query joining it to `dim.Customer` and the execution plan shows a clustered index scan on the fact table with an estimated 16,359 rows. They add `CREATE NONCLUSTERED INDEX IX_Fact_CustomerKey ON fact.Sales (CustomerKey)` and re-run the query. Describe what the new execution plan is likely to show, and under what condition the optimizer might still choose the scan.

5. Write a query against `sys.dm_db_index_physical_stats` to check fragmentation for all indexes in `CabotTrailOutdoorsSales.fact.Sales`. Interpret hypothetical results: fragmentation is 8% for `IX_Sales_CustomerKey` and 45% for `IX_Sales_OrderDateKey`. What action would you take for each, and why?

6. The following WHERE clause is used in a frequently executed report:

   ```
   WHERE DATEPART(year, OrderDate) = 2024 AND DATEPART(quarter, OrderDate) = 2
   ```

   Explain why this is non-sargable. Propose two design changes — one at the query level and one at the schema level — that would make the filter sargable.

7. A table used in an OLTP transaction processing system has 15 non-clustered indexes. A DBA reports that INSERT performance has degraded significantly. Explain the relationship between the number of indexes and INSERT performance, and describe the process you would use to evaluate which indexes are actually being used.

8. Design the index strategy for the `Sales.LoyaltyTransactions` table created in Chapter 3. The most common queries are:
    - "What transactions has account 42 had in the last 30 days?" (filter by AccountID and TransactionDate)
    - "How many EARN transactions were there per month?" (filter by TransactionType, group by month)
    - "What rewards has account 42 redeemed?" (filter by AccountID and RewardID where not null) Write the CREATE INDEX statements for each pattern, explaining your column order and INCLUDE choices.

🔍 Deeper Dive

Going Further with Indexes and Physical Design

Columnstore Indexes

Chapter 8 of the ETL textbook mentioned **columnstore indexes** as a performance option for large analytical fact tables. They deserve a fuller explanation here.

Traditional indexes store data **row by row** — a B-tree page contains complete rows. **Columnstore indexes** store data **column by column** — all values of a single column are stored together, heavily compressed.

For analytical queries that aggregate a few columns across millions of rows:

```
-- This query reads LineTotal and GrossProfit from all 16M rows
SELECT prod.CategoryName, SUM(fs.LineTotal), SUM(fs.GrossProfit)
FROM   fact.Sales fs
INNER JOIN dim.Product prod ON prod.ProductKey = fs.ProductKey
GROUP BY prod.CategoryName;
```

With a row-store index, SQL Server reads all columns of every qualifying row even though the query only needs two. With a columnstore index, only the `LineTotal` and `GrossProfit` columns (and the `ProductKey` for the JOIN) need to be read — dramatically reducing I/O.

```
-- Create a clustered columnstore index on a fact table
CREATE CLUSTERED COLUMNSTORE INDEX CCI_FactSales
    ON fact.Sales;

-- Or add a non-clustered columnstore alongside the existing row-store indexes
```

```
CREATE NONCLUSTERED COLUMNSTORE INDEX NCCI_FactSales_Analytics
    ON fact.Sales (OrderDateKey, CustomerKey, ProductKey, LineTotal, GrossProfit);
```

The trade-off: columnstore indexes are optimised for batch reads, not for individual row lookups or frequent small updates. They are ideal for DW fact tables loaded nightly; they are less suited for OLTP tables with many individual row insertions.

Microsoft documentation: Columnstore Indexes — Overview

Filtered Indexes

A **filtered index** is a non-clustered index that indexes only a subset of rows — rows that satisfy a filter predicate. Filtered indexes are smaller, faster to maintain, and more efficient for queries that consistently filter on the same condition.

```
-- Only index active (non-discontinued) products
CREATE NONCLUSTERED INDEX IX_Products_Active
    ON Inventory.Products (ProductName, UnitPrice)
    WHERE IsDiscontinued = 0;

-- Only index open loyalty accounts
CREATE NONCLUSTERED INDEX IX_LoyaltyAccounts_Active
    ON Sales.LoyaltyAccounts (PointBalance, TierID)
    WHERE PointBalance > 0;
```

A filtered index on `IsDiscontinued = 0` covers 95%+ of analytical queries about products (which typically filter to active products) while indexing only the active rows. The resulting index is much smaller than a full-table index and requires less maintenance during updates to discontinued products.

Filtered indexes are particularly effective for:

- Tables with a status flag where most queries filter on the active/current status
- Sparse columns (columns that are NULL for most rows)
- Tables with a "soft delete" flag

Microsoft documentation: Create Filtered Indexes

Statistics and the Query Optimizer

SQL Server's query optimizer uses **statistics** — histograms of column value distributions — to estimate how many rows each operator will process. Based on these estimates, it chooses between different execution plan options (index seek vs scan, hash join vs nested loops, etc.).

Statistics are automatically created when an index is created and automatically updated when a significant fraction of rows change. But they can become stale between updates:

```
-- View statistics for a column
DBCC SHOW_STATISTICS ('fact.Sales', 'IX_FactSales_CustomerKey');

-- Manually update statistics (run when optimizer is making poor choices)
UPDATE STATISTICS fact.Sales;

-- Update all statistics in the database
EXEC sp_updatestats;
```

When statistics are stale, the optimizer may significantly underestimate or overestimate row counts — leading to poor plan choices (using a scan when a seek would be better, or vice versa). Regular statistics maintenance — typically as part of the same maintenance window as index maintenance — keeps the optimizer making good decisions.

References and Further Reading

1. Fritchey, G. (2018). *SQL Server Query Performance Tuning* (5th ed.). Apress. — The most comprehensive practical guide to SQL Server indexes and execution plans. Chapters 3–7 cover index design; Chapters 9–12 cover execution plan reading.

2. Ben-Gan, I., Kollar, L., Sarka, D., & Talmage, R. (2015). *T-SQL Querying*. Microsoft Press. — Chapters 12–13 cover indexes and query optimisation at depth.

3. Kalen Delaney. (2014). *Microsoft SQL Server 2014 Internals*. Microsoft Press. — The authoritative reference for SQL Server storage internals including page structure, B-tree mechanics, and index architecture.

4. Microsoft. (2024). *Clustered and Non-Clustered Indexes Described*. https://learn.microsoft.com/en-us/sql/relational-databases/indexes/clustered-and-nonclustered-indexes-described

5. Microsoft. (2024). *Columnstore Indexes Overview*. https://learn.microsoft.com/en-us/sql/relational-databases/indexes/columnstore-indexes-overview

6. Microsoft. (2024). *Create Filtered Indexes*. https://learn.microsoft.com/en-us/sql/relational-databases/indexes/create-filtered-indexes

7. Microsoft. (2024). *sys.dm_db_index_physical_stats (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/relational-databases/system-dynamic-management-views/sys-dm-db-index-physical-stats-transact-sql

8. Microsoft. (2024). *Execution Plans*. https://learn.microsoft.com/en-us/sql/relational-databases/performance/execution-plans

---

Database Design for Analysts *| © Patrick Dolinger, NSCC Institute of Technology CC BY 4.0 — Share and adapt freely with attribution*

Chapter 8: OLTP vs OLAP — Two Design Philosophies

Database Design for Analysts *From business requirements to relational structures*

© Patrick Dolinger, NSCC Institute of Technology Licensed under Creative Commons Attribution 4.0 International (CC BY 4.0) You are free to share and adapt this material for any purpose, provided appropriate credit is given.

Chapter Overview

Every concept in Chapters 1 through 7 has been applied to one kind of database: the operational, transactional database — the OLTP. But you have been working with two kinds of databases throughout this course and the prerequisite: the CabotTrail OLTP and the CabotTrail data warehouse and data marts. The contrast between them is not accidental. They are built for fundamentally different purposes, and those purposes drive radically different design decisions.

This chapter examines that contrast directly. It explains *why* the same information is structured so differently in `CabotTrailOutdoor` vs `CabotTrailOutdoorsSales`, why one is normalised and the other is deliberately not, and why both are correctly designed for their respective purposes.

This chapter also positions what you have learned in this course within the broader data architecture. As an analyst, you will work with both OLTP and OLAP systems throughout your career. Understanding the design philosophy behind each — and knowing which questions to ask of each — is a core professional competency.

By the end of this chapter you will be able to:

- Define OLTP and OLAP and describe the primary purpose of each
- Explain the workload characteristics that distinguish OLTP from OLAP
- Describe the star schema design pattern and justify its trade-offs
- Compare the CabotTrail OLTP and DW schemas and explain specific design differences
- Explain the role of surrogate keys, slowly changing dimensions, and degenerate dimensions in a DW
- Describe the ETL pipeline that connects OLTP to DW
- Explain the concept of conformed dimensions and why they matter

- Position OLAP within the broader analytics architecture including data marts and the executive summary database

---

Table of Contents

1. Two Databases, Two Purposes
2. OLTP: Designed for Operations
3. OLAP: Designed for Analysis
4. The Design Consequences of Different Purposes
5. The Star Schema: OLAP's Core Structure
6. Dimension Tables in Depth
7. Fact Tables in Depth
8. Surrogate Keys in the DW Context
9. Slowly Changing Dimensions
10. Conformed Dimensions
11. The ETL Pipeline as the Bridge
12. The CabotTrail Architecture End to End
13. Chapter Summary
14. Review Questions
15. 🔍 Deeper Dive

---

1. Two Databases, Two Purposes

1.1 The Same Data, Different Structure

Consider the revenue for order number 1001. In `CabotTrailOutdoor` (OLTP), finding the total revenue for that order requires joining at least five tables:

```
-- OLTP: revenue for order 1001
USE CabotTrailOutdoor;

SELECT
    o.OrderID,
    c.CustomerName,
    SUM(il.LineTotal)   AS OrderRevenue
```

```
FROM    Sales.Orders o
INNER JOIN Sales.Customers c    ON c.CustomerID   = o.CustomerID
INNER JOIN Sales.Invoices i     ON i.OrderID      = o.OrderID
INNER JOIN Sales.InvoiceLines il ON il.InvoiceID  = i.InvoiceID
WHERE   o.OrderID = 1001
GROUP BY o.OrderID, c.CustomerName;
```

In `CabotTrailOutdoorsSales` (data mart), the same analysis is one table with a filter:

```
-- Data mart: revenue for order 1001
USE CabotTrailOutdoorsSales;

SELECT
    InvoiceID,
    SUM(LineTotal)  AS Revenue
FROM    fact.Sales
WHERE   OrderID = 1001
GROUP BY InvoiceID;
```

The data mart version is dramatically simpler. But this simplicity comes at a cost — the data mart cannot answer operational questions ("was this order picked yet? who approved the credit?") that the OLTP was designed for.

Neither design is better than the other. They are different tools for different jobs.

1.2 The Two Letters That Explain Everything

OLTP — Online Transaction Processing. **OLAP** — Online Analytical Processing.

These acronyms were coined in the 1980s and 1990s to distinguish two patterns of database usage that had emerged in practice. They capture the core distinction:

- **Transaction processing:** Recording what the business *does* — one event at a time, many users simultaneously, requiring correctness above all
- **Analytical processing:** Understanding what the business *has done* — across large datasets, typically by one or few users, requiring query performance above all

---

2. OLTP: Designed for Operations

2.1 Characteristics of OLTP Workloads

An OLTP database is the system of record for a business's operational activity. It is designed to:

- **Process many small transactions simultaneously** — hundreds or thousands of users placing orders, updating records, and querying balances at the same time
- **Record each transaction quickly and accurately** — an order that takes 10 seconds to save is unacceptable; an order that saves the wrong data is catastrophic
- **Support current state queries** — "what is Customer 42's current credit balance?" not "what was Customer 42's average monthly spend over the past three years?"
- **Maintain data integrity** — every constraint, every FK check, every transaction boundary protects against incorrect data

2.2 OLTP Query Characteristics

Characteristic	OLTP pattern
Rows accessed per query	Few — typically one record or a small set
Query complexity	Simple lookups, small JOINs
Query frequency	Very high — thousands per minute
Data currency	Must be real-time — freshness of seconds
Write-to-read ratio	High — many writes relative to reads
Typical users	Application systems, operational staff

2.3 OLTP Design Principles

These workload characteristics drive the OLTP design principles covered throughout this course:

Normalization to 3NF: Each fact stored once. Updates to one row propagate correctly everywhere. No redundancy to maintain during the many small transactions.

Many tables with targeted indexes: Small tables that can be efficiently filtered and joined. Indexes on FK columns and commonly filtered columns ensure single-row lookups are fast.

Strict constraints: Every business rule enforced at the database layer. A transaction that violates a constraint is rejected before it can corrupt the system of record.

Short transactions: OLTP transactions lock rows briefly (one order, one customer update), released quickly so other transactions can proceed.

---

3. OLAP: Designed for Analysis

3.1 Characteristics of OLAP Workloads

An OLAP database (which includes data warehouses and data marts) is designed to:

- **Answer large, complex aggregation queries** — "what was total revenue by territory by product category for each quarter of 2023 and 2024?"
- **Support historical analysis** — data is typically accumulated over years; queries must scan millions of rows efficiently
- **Serve a smaller number of analytical users** — analysts, managers, BI tools — rather than hundreds of operational users simultaneously
- **Tolerate slightly stale data** — loaded nightly or hourly from the OLTP; real-time freshness is not required

3.2 OLAP Query Characteristics

Characteristic	OLAP pattern
Rows accessed per query	Many — thousands to millions
Query complexity	Complex aggregations across multiple dimensions
Query frequency	Low to moderate — dozens per hour
Data currency	Acceptable latency — hours to one day
Write-to-read ratio	Very low — data loaded by batch ETL, read constantly
Typical users	Analysts, managers, BI tools (Power BI, Tableau)

3.3 OLAP Design Principles

The OLAP workload characteristics drive opposite design decisions from OLTP:

Denormalization: Wide, flat tables that avoid joins during queries. A customer dimension contains `CustomerName`, `CityName`, `ProvinceName`, `CountryName`, and `SalesTerritory` in one row — data that in the OLTP spans four tables. This denormalization means analytical queries need fewer joins.

Fewer, simpler tables: A star schema with one fact table and five to ten dimension tables is far simpler to query than a 3NF schema with 30–50 tables.

Optimised for reads: Generous indexing (including columnstore indexes for large fact tables), no FK enforcement constraints that would slow query planning, no write-intensive update operations.

Historical data: The DW accumulates facts over time. The OLTP may only retain current state (active customers, current prices); the DW retains everything that ever happened.

---

4. The Design Consequences of Different Purposes

The contrast between OLTP and OLAP design is most visible when you look at the same business concept in both systems.

4.1 The Customer: OLTP vs DW

In `CabotTrailOutdoor` (OLTP):

```
-- Customer in OLTP: linked to geography through normalised hierarchy
SELECT
    c.CustomerID,
    c.CustomerName,
    ci.CityName,
    sp.StateProvinceName,
    co.CountryName
FROM    Sales.Customers c
INNER JOIN Application.Cities ci         ON ci.CityID            = c.DeliveryCityID
INNER JOIN Application.StateProvinces sp ON sp.StateProvinceID   = ci.StateProvinceID
INNER JOIN Application.Countries co      ON co.CountryID         = sp.CountryID
WHERE   c.CustomerID = 42;
-- 3 joins to get customer geography — 3NF design
```

In `CabotTrailOutdoorDW` (Data Warehouse):

```
-- Customer in DW: all geography attributes in one wide row
SELECT
    CustomerKey,
    CustomerID,
    CustomerName,
    CityName,
    ProvinceName,
    ProvinceCode,
    CountryName,
    SalesTerritory
FROM    Dimension.DimCustomer
```

```
WHERE   CustomerID = 42;
-- No joins needed — denormalised for query simplicity
```

The DW `DimCustomer` is deliberately in violation of 3NF. `ProvinceName` depends on `ProvinceCode` which depends on `CityName` which depends on `CustomerID`. There are transitive dependencies. This is not a mistake — it is intentional denormalization to eliminate joins from analytical queries.

4.2 The Sale: OLTP vs DW

In `CabotTrailOutdoor` (OLTP): A sale spans `Orders`, `OrderLines`, `Invoices`, `InvoiceLines`, and references `Customers`, `Products`, `Employees`, `DeliveryMethods`, `Cities`.

In `CabotTrailOutdoorsSales` (Data Mart): A sale is one row in `fact.Sales` with 25 pre-computed columns — all joins resolved, all measures pre-calculated, all FK lookups replaced with surrogate key references to five dimension tables.

The ETL pipeline performs the transformation between these two representations nightly — this is the fundamental purpose of ETL.

4.3 The Design Decision Matrix

Design question	OLTP answer	OLAP answer
Normalise to 3NF?	Yes — mandatory	No — deliberately denormalised
How many tables?	Many (20–50+)	Few (5–15 per star schema)
Fact table exists?	No — data is spread across entities	Yes — central fact table with pre-computed measures
Store historical versions?	Generally no (current state)	Yes — multiple snapshots
Index aggressively?	Moderately (write overhead)	Aggressively (reads dominate)
FK constraints on fact table?	N/A	Usually not (ETL ensures integrity)
Primary consumer?	Applications, operational users	BI tools, analysts

---

5. The Star Schema: OLAP's Core Structure

The **star schema** is the dominant design pattern for OLAP systems. It was formalised by Ralph Kimball in the 1990s and remains the standard for analytical database design worldwide.

5.1 The Star Structure

```
                    dim.Calendar
                         |
dim.Customer ———— fact.Sales ———— dim.Product
                         |
              dim.Employee —— dim.DeliveryMethod
```

One central **fact table** surrounded by multiple **dimension tables** — connected by FK relationships from the fact table to each dimension's PK. The visual structure resembles a star.

5.2 Why It Works for Analysis

The star schema is optimised for a specific class of questions: "What was [measure] for [dimension filter] by [dimension group]?"

- "What was **total revenue** (measure) for customers in **Nova Scotia** (dimension filter) **by product category** (dimension group)?"
- "What was the **return rate** (measure) for **Sleeping products** (dimension filter) in **Q2 2024** (dimension filter) **by sales rep** (dimension group)?"

Every analytical question about business performance follows this pattern. The star schema answers all of them with the same two-step structure:

1. Join the fact table to the relevant dimensions
2. Filter and aggregate

The simplicity of this two-step structure is why BI tools can auto-generate correct SQL for star schemas — and why analysts find star schemas intuitive to query.

5.3 Snowflake Schema: A Variation

A **snowflake schema** partially normalises the dimension tables. Instead of a flat `dim.Customer` with `CityName` and `ProvinceName` as direct attributes, the snowflake has separate `dim.City` and `dim.Province` tables connected to `dim.Customer` through FKs:

```
dim.Country ——— dim.Province ——— dim.City ——— dim.Customer ——— fact.Sales
```

This recovers some storage efficiency (city names are stored once) at the cost of additional joins in analytical queries. Kimball generally argues against snowflaking: the storage savings are modest, the query complexity increases, and BI tools handle star schemas more naturally than snowflakes.

The CabotTrail DW uses a hybrid: the customer dimension is flattened (star), but products have a `DimProductCategory` table separate from `DimProduct` (a limited snowflake for categories). This is a pragmatic decision — product category details are needed independently and the one additional join is acceptable.

---

6. Dimension Tables in Depth

Dimension tables provide the descriptive context for fact measurements. They answer the "who, what, when, where, and how" of every transaction.

6.1 Dimension Table Characteristics

- **Wide:** Many descriptive columns — a dimension table with 20–30 columns is normal
- **Small:** Relatively few rows compared to the fact table — `dim.Customer` has 100 rows; `fact.Sales` has 16,359
- **Text-heavy:** Most columns are descriptive strings — names, descriptions, categories, codes
- **Slowly changing:** Dimension data changes infrequently relative to fact data (customers change addresses occasionally; products change categories rarely)
- **Include hierarchy levels:** Multiple levels of a hierarchy (City, Province, Country) stored as flat columns in the same row — the denormalization that enables hierarchy drill-down in a single table

6.2 The Date Dimension: The Most Important Dimension

Every data warehouse has a **date dimension** — a table with one row per calendar date, containing every possible date attribute that any analytical query might need:

```
-- A few columns from CabotTrailOutdoorDW.Dimension.DimDate
SELECT TOP 5
    DateKey,                -- YYYYMMDD integer (e.g., 20240315)
    FullDate,               -- Actual DATE value
    DayName,                -- 'Friday'
    MonthName,              -- 'March'
    MonthNumber,            -- 3
    CalendarYear,           -- 2024
    CalendarQuarter,        -- 'Q1'
    FiscalYearNumber,       -- 2024 (or 2025 depending on fiscal year start)
    FiscalQuarterNumber,    -- 1
    IsWeekend,              -- 0
    IsHoliday               -- 0
```

```
FROM Dimension.DimDate
ORDER BY DateKey;
```

> ***Check your warehouse's column names.*** *Chapter 8 lists the year column of* `Dimension.DimDate` *as* `CalendarYear`*, while the Chapter 9 scripts use* `YearNumber`*. Run this query against* `CabotTrailOutdoorDW` *and use the names it returns:*
>
> ```
> SELECT COLUMN_NAME, DATA_TYPE
> FROM   CabotTrailOutdoorDW.INFORMATION_SCHEMA.COLUMNS
> WHERE  TABLE_SCHEMA = 'Dimension' AND TABLE_NAME = 'DimDate'
> ORDER BY ORDINAL_POSITION;
> ```

The date dimension is populated once for a wide range of dates (CabotTrail has 4,017 rows spanning 2022–2032) and never updated. Its integer `DateKey` (YYYYMMDD format) is used as the FK on the fact table — allowing date range filters without converting dates to integers at query time.

6.3 The Unknown Member

Every dimension table includes an **unknown member** row with surrogate key = 0 and descriptive values of 'Unknown' or similar. This row serves as the FK target for fact rows whose dimension value cannot be resolved:

```
-- The unknown member in dim.Customer
SELECT CustomerKey, CustomerID, CustomerName, SalesTerritory
FROM   dim.Customer
WHERE  CustomerKey = 0;
-- Returns: 0, 0, 'Unknown Customer', 'Unknown'
```

When a fact row cannot be matched to a dimension row (a referential integrity gap in the source data), it references `CustomerKey = 0` rather than being dropped. The fact is preserved; it is attributed to the unknown member. Analysts can identify and investigate these rows:

```
-- How many sales have an unresolved customer?
SELECT COUNT(*) AS UnknownCustomerSales
FROM   fact.Sales
WHERE  CustomerKey = 0;
-- Should always return 0 in a well-managed DW
```

---

7. Fact Tables in Depth

Fact tables store the measurements of business events. They are the largest tables in the DW — typically orders of magnitude larger than any dimension table.

7.1 Types of Facts

Additive facts can be summed across all dimensions. `LineTotal`, `GrossProfit`, `OrderedQuantity` are additive — they can be summed by territory, by year, by product, or any combination.

Semi-additive facts can be summed across some dimensions but not others. `QuantityOnHand` (inventory) is semi-additive: it can be summed across products (total stock) but NOT across time (adding yesterday's and today's stock counts double-counts).

Non-additive facts cannot be meaningfully summed across any dimension. `GrossProfitMarginPct` is non-additive — summing percentages produces meaningless results. The correct aggregate is always `SUM(GrossProfit) / SUM(LineTotal)`.

7.2 Fact Table Grain

The **grain** is the precise definition of what one row in the fact table represents. Declaring the grain before designing the fact table is the first and most critical design step.

CabotTrail `fact.Sales` grain: **one row per product line on each customer invoice**. Every design decision about which columns and measures belong in the fact table follows from this grain declaration.

A finer grain (one row per item within each line) would allow tracking individual units; a coarser grain (one row per invoice) would lose line-level detail. Kimball's advice: choose the finest grain the source data supports. Aggregates can always be computed upward; detail cannot be recovered downward.

7.3 Fact Table Columns

A fact table row contains:

- **Foreign keys** to every dimension (CustomerKey, ProductKey, EmployeeKey, etc.)
- **Degenerate dimensions** — transaction identifiers that have no corresponding dimension table (InvoiceID, OrderID — identifiers that add analytical context without needing a dimension of their own)
- **Additive measures** (LineTotal, GrossProfit, OrderedQuantity, TaxAmount)

- **Pre-computed derived measures** (LineTotalIncludingTax, GrossProfitMarginPct — computed during ETL and stored for query convenience)

```
-- fact.Sales column inventory
SELECT  COLUMN_NAME, DATA_TYPE, IS_NULLABLE
FROM    INFORMATION_SCHEMA.COLUMNS
WHERE   TABLE_SCHEMA = 'fact' AND TABLE_NAME = 'Sales'
ORDER BY ORDINAL_POSITION;
```

---

8. Surrogate Keys in the DW Context

Chapter 3 of the ETL textbook introduced surrogate keys as the mechanism that links facts to dimensions in a data warehouse. This section explains *why* surrogate keys are used in the DW even when they are not strictly necessary in the OLTP.

8.1 The Problem with Natural Keys in the DW

The OLTP uses `CustomerID INT` as the natural key for customers. Could the DW use `CustomerID` directly as the FK on `fact.Sales`?

The answer is: technically yes, but practically no. Three reasons:

1. Source system changes. If a second source system is added (a second retail channel, an acquisition), its customers will have their own ID numbering — which may overlap with the OLTP's CustomerIDs. The DW needs a namespace-independent key that is unique across all sources.

2. Slowly changing dimensions. When a customer moves from Nova Scotia to Ontario, SCD Type 2 creates a new dimension row for the same customer — but with different attribute values. The surrogate key distinguishes the two versions (`CustomerKey = 15` for the old NS address, `CustomerKey = 87` for the new ON address). The natural key (`CustomerID = 42`) is the same for both.

3. Performance. Surrogate integer keys (4 bytes) are more compact than natural keys (which may be NVARCHAR(20), 40+ bytes). Every FK on every fact row uses the surrogate key — at millions of rows, the storage and join performance difference is significant.

8.2 The Surrogate Key as Insulation

The surrogate key insulates the DW from changes in the source system. If CabotTrail's OLTP resets customer numbering (unlikely but possible), the DW's CustomerKey values remain stable — the ETL mapping layer absorbs the change.

9. Slowly Changing Dimensions

Real-world dimension data changes over time. A customer moves. An employee is promoted. A product is recategorised. How the DW handles these changes is the **Slowly Changing Dimension (SCD)** design problem.

9.1 SCD Type 1: Overwrite (No History)

The simplest approach: when an attribute changes, update the dimension row in place. Historical fact rows now reflect the new attribute value — you cannot determine what the attribute was at the time of any specific transaction.

When appropriate: For corrections (fixing a typo in a customer name) or for attributes whose historical value is not analytically important.

CabotTrail: All dimensions in the CabotTrail DW use SCD Type 1. This is a deliberate design decision that simplifies the ETL — at the cost of historical dimension tracking.

9.2 SCD Type 2: Add New Row (Full History)

When an attribute changes, a new row is added to the dimension with the new attribute values. The old row is expired (its `ValidTo` date is set to today; its `IsCurrent` flag is set to 0). The new row gets a new surrogate key.

Historical fact rows continue to reference the old surrogate key — they preserve the attribute values that were current when those facts were recorded.

```
-- SCD Type 2 example: customer 42 moves from NS to ON
-- Before the move:
-- CustomerKey=15, CustomerID=42, ProvinceCode='NS', ValidFrom='2022-01-01',
ValidTo='9999-12-31', IsCurrent=1

-- After the move, the old row is expired and a new row added:
-- CustomerKey=15, CustomerID=42, ProvinceCode='NS', ValidFrom='2022-01-01',
ValidTo='2024-07-14', IsCurrent=0
-- CustomerKey=87, CustomerID=42, ProvinceCode='ON', ValidFrom='2024-07-15',
ValidTo='9999-12-31', IsCurrent=1

-- Historical facts reference CustomerKey=15 (NS province at time of sale)
-- New facts reference CustomerKey=87 (ON province)
```

When appropriate: When the business needs to answer "what province was this customer in at the time of this sale?" — i.e., when historical accuracy of the dimension attribute matters for analysis.

9.3 SCD Type 6: The Hybrid

SCD Type 6 combines Types 1, 2, and 3. Each dimension row stores both the **historical** attribute value (preserved from when the row was created — Type 2) and the **current** attribute value (updated on every load — Type 1). This allows both "what was the province at time of sale?" and "what is the customer's current province?" from a single dimension join.

```
-- SCD Type 6 example
-- CustomerKey=15, CustomerID=42, ProvinceCode='NS'(historical),
CurrentProvinceCode='ON'(current)
-- CustomerKey=87, CustomerID=42, ProvinceCode='ON'(historical),
CurrentProvinceCode='ON'(current)
```

The ETL for Business Intelligence textbook covers all SCD types in detail in Chapter 7. For the database designer, the key point is: **the SCD type choice must be made before the DW schema is finalised** — it affects the dimension table structure.

---

10. Conformed Dimensions

A **conformed dimension** is a dimension table used consistently across multiple fact tables and multiple data marts, with the same structure, the same keys, and the same meaning.

10.1 Why Conformance Matters

If `dim.Customer` in the Sales data mart defines `SalesTerritory` differently from `dim.Customer` in the Returns data mart, analysts cannot directly compare "Sales by territory" with "Returns by territory" — the territories are defined differently.

Conformed dimensions make cross-mart comparisons possible. When both data marts share the same `DimCustomer` from the DW (same surrogate keys, same attribute definitions), a query joining Sales and Returns data by customer is meaningful.

10.2 The DW as the Conformance Layer

The CabotTrail architecture uses the DW (`CabotTrailOutdoorDW`) as the conformance layer. Dimensions are defined once in the DW with canonical structure and values. The data marts load their dimensions from the DW — not independently from the OLTP:

```
OLTP → DW (canonical dimensions defined here) → Data marts (copy from DW)
```

When the Sales mart loads `dim.Customer`, it copies from `Dimension.DimCustomer` in the DW — using the same surrogate keys, the same `SalesTerritory` derivation, the same `ProvinceCode` values. When the Returns mart loads its `dim.Customer`, it copies from the same source.

The result: `CustomerKey = 15` means the same customer in both marts. A query comparing sales and returns by customer can join on CustomerKey safely.

10.3 The Bus Matrix

Kimball's **bus matrix** is a planning tool that maps which dimensions are used by which fact tables. It visualises the conformance relationships:

Fact table	Calendar	Customer	Product	Employee	Supplier	Delivery
FactSales	✓	✓	✓	✓		✓
FactReturns	✓	✓	✓	✓		
FactPurchasing	✓		✓	✓	✓	
FactInventory	✓		✓		✓	

Dimensions that appear in multiple fact tables (Calendar, Customer, Product) are the conformed dimensions — they must be defined consistently across all the fact tables that use them.

---

11. The ETL Pipeline as the Bridge

The ETL pipeline is the mechanism that transforms data from the normalised OLTP representation to the denormalised OLAP representation. Understanding this transformation bridges the two design philosophies.

11.1 What the ETL Does

The ETL pipeline for a single dimension load (`DimCustomer`) performs:

1. **Extract:** Read from `Sales.Customers` , `Application.Cities` , `Application.StateProvinces` , `Application.Countries` in the OLTP
2. **Transform:** Join the four tables, derive `SalesTerritory` from `ProvinceCode` , apply default values for missing data
3. **Load:** Write the combined, denormalised row to `Dimension.DimCustomer` in the DW

The ETL resolves the 3NF normalisation of the OLTP into the intentional denormalization of the DW. This is not a workaround — it is the designed behaviour of the architecture.

11.2 The S2T Mapping as the Link

The **source-to-target (S2T) mapping** — introduced in the ETL textbook — documents exactly how each OLTP column maps to each DW column. It is, in effect, the specification that describes how to bridge the two design philosophies for every column in every dimension and fact table.

For the database designer, the S2T mapping represents the answer to the question: "How does the OLTP design choices affect the DW design?" Every normalisation decision in the OLTP (separating customer geography into four tables) creates a corresponding structural gap in the DW S2T mapping (the gap must be resolved by JOINing four source tables during ETL).

11.3 The Design Implications

OLTP design affects ETL complexity. A well-normalised OLTP requires more complex ETL (more source tables to join, more derived columns to compute). A poorly-normalised OLTP may produce a simpler ETL extract but poorer data quality (redundant data in the source creates update anomaly risk).

DW design affects query simplicity. A wide, flat, well-structured DW star schema enables simple analytical queries. A snowflaked or partially normalised DW requires more joins in analytical queries.

The optimal architecture: a well-normalised OLTP (maintains data integrity), connected by a well-designed ETL, to a well-structured star schema DW (supports fast analytics). This is exactly the CabotTrail architecture.

---

12. The CabotTrail Architecture End to End

12.1 The Complete Pipeline

```
CabotTrailOutdoor (OLTP)
    • 3NF normalised
    • ~30 tables, 4 schemas
    • Designed for transactional correctness
    • Updated constantly (orders, invoices, picks)
         │
         │  ETL (SSIS packages, nightly 2:00 AM)
         │  Source-to-target mapping resolves:
         │    • Geography JOINs (4 tables → 1 flat column set)
         │    • Surrogate key generation
         │    • Measure derivations (UnitCost, GrossProfit)
         │    • SalesTerritory derivation
         ↓
CabotTrailOutdoorDW (Data Warehouse)
    • Dimensional model (star + limited snowflake)
    • 10 dimensions, 6 fact tables
    • Conformed dimensions for cross-mart analysis
    • Updated nightly (full reload)
         │
         │  ETL Layer 2 (datamart load packages)
         │  Simple projection from DW to each mart
         ↓
Data Marts:
    CabotTrailOutdoorsSales      — 6 tables — Sales analysis
    CabotTrailOutdoorsReturns    — 5 tables — Returns analysis
    CabotTrailOutdoorsPurchasing — 5 tables — Purchasing analysis
    CabotTrailOutdoorsInventory  — 5 tables — Inventory analysis
    CabotTrailOutdoorsTransactions — 6 tables — Financial transactions
    CabotTrailExecutive          — 3 tables + 1 view — Executive summary (Chapter 9)
         │
         ↓
BI Tools (Power BI, Excel, SQL queries)
    • Star schema queries
    • Filtered by dimension attributes
    • Aggregated on fact measures
```

12.2 Reading the Architecture as a Designer

The CabotTrail architecture embodies every principle from this course:

- The OLTP reflects Chapters 1–7: proper entities, 3NF normalisation, complete constraints, appropriate indexes
- The ETL embodies Chapter 3's mapping rules (in reverse): denormalising the 3NF structure for OLAP
- The DW reflects Chapter 8: star schema, conformed dimensions, surrogate keys, SCD choices

- The data marts reflect Chapter 8's denormalization philosophy: flattened dimensions, pre-computed measures, read-optimised structure

12.3 What Each Database Is Good At

```
-- OLTP: operational question — is this order ready to ship?
USE CabotTrailOutdoor;
SELECT o.OrderID, o.IsUndersupplyBackordered, ol.PickedQuantity, ol.Quantity
FROM   Sales.Orders o
INNER JOIN Sales.OrderLines ol ON ol.OrderID = o.OrderID
WHERE  o.OrderID = 1001;

-- DW: analytical question — which sales reps had the highest margins last quarter?
USE CabotTrailOutdoorDW;
SELECT
    e.FullName,
    ROUND(SUM(fs.GrossProfit) / NULLIF(SUM(fs.LineTotal), 0) * 100, 2) AS MarginPct
FROM   Fact.FactSales fs
INNER JOIN Dimension.DimEmployee e ON e.EmployeeKey = fs.EmployeeKey
INNER JOIN Dimension.DimDate d     ON d.DateKey      = fs.OrderDateKey
WHERE  d.FiscalYearNumber = 2024 AND d.FiscalQuarterNumber = 1
GROUP BY e.FullName
ORDER BY MarginPct DESC;

-- Data mart: the same analytical question, even simpler query
USE CabotTrailOutdoorsSales;
SELECT
    emp.FullName,
    ROUND(SUM(fs.GrossProfit) / NULLIF(SUM(fs.LineTotal), 0) * 100, 2) AS MarginPct
FROM   fact.Sales fs
INNER JOIN dim.Employee emp ON emp.EmployeeKey = fs.EmployeeKey
INNER JOIN dim.Calendar cal ON cal.DateKey     = fs.OrderDateKey
WHERE  cal.FiscalYearNumber = 2024 AND cal.FiscalQuarterNumber = 1
GROUP BY emp.FullName
ORDER BY MarginPct DESC;
```

Each database type excels at the questions it was designed for.

---

13. Chapter Summary

- **OLTP** (Online Transaction Processing) databases support business operations: many simultaneous users, small transactions, real-time freshness, write-heavy, normalised to 3NF for data integrity.
- **OLAP** (Online Analytical Processing) databases support business analysis: complex aggregations, historical data, read-heavy, denormalised for query performance.

- The **star schema** — one central fact table surrounded by dimension tables — is the standard OLAP design pattern. It optimises for the analytical query pattern: "what was [measure] for [filter] by [group]?"
- **Dimension tables** are wide, text-heavy, and slowly changing. They store descriptive context (who, what, when, where). The date dimension is universal. Every dimension has an unknown member row.
- **Fact tables** store measurements at a defined grain. Additive facts can be summed across all dimensions; semi-additive only across some; non-additive cannot be summed.
- **Surrogate keys** in the DW insulate against source system changes, support SCD Type 2 (multiple versions of the same entity), and improve join performance.
- **SCD types** define how dimension changes are handled: Type 1 (overwrite — no history), Type 2 (new row — full history), Type 6 (hybrid — both current and historical values).
- **Conformed dimensions** are shared consistently across multiple fact tables and data marts, enabling cross-mart analysis. The DW serves as the conformance layer.
- The **ETL pipeline** transforms data from normalised OLTP to denormalised OLAP — resolving JOINs, computing derived measures, generating surrogate keys, and handling SCD logic.
- The CabotTrail architecture embodies the complete pipeline: OLTP → DW → data marts → BI tools, with each layer designed for its specific purpose.

---

14. Review Questions

1. Explain why the same customer information is stored in four tables in `CabotTrailOutdoor` (OLTP) but in one wide row in `Dimension.DimCustomer` (DW). Reference both normalization principles and OLAP design principles in your answer.
2. Write two queries that answer the same business question — "what is the total revenue by product category for calendar year 2024?" — one against the OLTP (`CabotTrailOutdoor`) and one against the Sales data mart (`CabotTrailOutdoorsSales`). Compare the number of joins and the structural complexity of each.
3. `fact.Sales` stores `GrossProfitMarginPct` as a column even though it is non-additive. Explain why storing a non-additive measure in a fact table can be acceptable, what risks it introduces, and what documentation should accompany it.

4. CabotTrail decides to implement SCD Type 2 for `DimCustomer` to track province changes. Describe the structural changes required to `dim.Customer` (new columns), what the ETL process must do when a province change is detected, and how historical fact rows will correctly reference the old province after the change.

5. The bus matrix shows that `Calendar`, `Customer`, and `Product` are conformed dimensions — used by multiple fact tables. Explain what "conformed" means technically: what specifically must be true about the `CustomerKey = 15` row in the Sales mart and the `CustomerKey = 15` row in the Returns mart for them to be considered conformed?

6. A new analyst joins the team and asks why `fact.Sales` does not have a FOREIGN KEY constraint from `CustomerKey` to `dim.Customer.CustomerKey`, since CustomerKey is clearly a FK relationship. Provide a complete explanation of why the DW intentionally does not enforce FK constraints on fact tables in most production environments.

7. Compare the index strategy appropriate for `Sales.Orders` (OLTP) versus `fact.Sales` (data mart). Explain the differences in terms of write frequency, query patterns, and the acceptable trade-off between read performance and write overhead.

8. A business analyst suggests: "We should store all our data in one big database and just have both normalised OLTP tables and denormalised analytical views — that way we don't need two databases." Evaluate this suggestion: what are its advantages, what are its risks, and under what circumstances might it be the right approach?

🔍 Deeper Dive

Going Further with OLTP and OLAP Design

Kimball vs Inmon: The Data Warehouse Architecture Debate

Two competing approaches to data warehouse architecture dominated the field from the 1990s through the 2000s:

Bill Inmon's Corporate Information Factory (CIF): Build a centralised, normalised enterprise data warehouse first — a single integrated, normalised (3NF) repository of all enterprise data. Then derive denormalised departmental data marts from the enterprise DW. The DW is the single source of truth; marts are derived.

Ralph Kimball's Dimensional Modelling / Data Warehouse Bus Architecture: Build subject-area data marts first, designed around dimensional (star schema) models. Connect them through conformed dimensions (the "bus"). The enterprise DW emerges bottom-up as the collection of integrated marts.

CabotTrail uses a hybrid approach: The DW (`CabotTrailOutdoorDW`) is built directly from the OLTP using a dimensional model (Kimball-style), and the subject-area data marts are derived from the DW. This combines Inmon's idea of a centralised DW (before the marts) with Kimball's dimensional modelling methodology.

In practice, most modern enterprise DW implementations blend both approaches. Pure Inmon implementations require large upfront investment and long development cycles; pure Kimball bottom-up approaches can result in marts that are harder to integrate. The key is understanding both enough to make principled design decisions.

Inmon, W. H. (2005). *Building the Data Warehouse* (4th ed.). Wiley. Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit* (3rd ed.). Wiley.

The Modern Data Lakehouse

The traditional OLTP/OLAP separation described in this chapter is being disrupted by the **data lakehouse** architecture — a hybrid that combines the scale and flexibility of a data lake with the governance and query performance of a data warehouse.

In a data lakehouse:

- Raw data is stored in cloud object storage (S3, Azure Data Lake, Google Cloud Storage) in open formats (Parquet, Delta Lake, Apache Iceberg)
- A query engine (Databricks, Spark, DuckDB) reads directly from the object storage
- The same data can serve both operational queries and analytical queries
- The ETL pipeline becomes lighter — data is ingested as-is and transformed on read rather than on write (ELT pattern)

The star schema concepts from this chapter apply to the semantic layer of a data lakehouse — the logical model that BI tools and analysts use — even if the physical storage is different.

Tools: Delta Lake, Apache Iceberg, Databricks Lakehouse, Microsoft Fabric, Snowflake.

Operational Data Stores (ODS)

An **Operational Data Store (ODS)** is a middle tier between the OLTP and the DW — a normalised, subject-oriented, integrated, volatile store of current operational data from multiple source systems.

The ODS:

- Integrates data from multiple OLTP source systems (in a multi-system enterprise, the OLTP is rarely just one database)
- Is normalised (not dimensional) — it is an integration layer, not an analytical layer
- Contains current data only (no history) — it is operational, not historical
- Supports operational reporting (current-state analysis) that the OLTP cannot easily support (because it requires integrating multiple systems)

The ODS feeds the DW (which adds history and dimensional structure). Many enterprises have an ODS between their OLTP systems and their DW, though smaller organisations like CabotTrail can load directly from OLTP to DW.

References and Further Reading

1. Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling* (3rd ed.). Wiley. — The primary reference for star schema design, dimensional modelling, and SCD types.

2. Inmon, W. H. (2005). *Building the Data Warehouse* (4th ed.). Wiley. — The foundational text for the enterprise data warehouse approach.

3. Kimball, R., & Caserta, J. (2004). *The Data Warehouse ETL Toolkit*. Wiley. — The ETL perspective on bridging OLTP and OLAP.

4. Connolly, T., & Begg, C. (2015). *Database Systems: A Practical Approach to Design, Implementation, and Management* (6th ed.). Pearson. — Chapter 31 covers data warehousing and OLAP in the context of relational database design.

5. Microsoft. (2024). *Azure Synapse Analytics Documentation*. https://learn.microsoft.com/en-us/azure/synapse-analytics/ — The cloud-native evolution of the OLTP/OLAP architecture described in this chapter.

6. Databricks. (2024). *Delta Lake Documentation*. https://delta.io/ — The leading open-source data lakehouse format.

7. Dolinger, P. (2027). *ETL for Business Intelligence: A practical guide to data provisioning, dimensional modelling, and pipeline design*. NSCC Institute of Technology. CC BY 4.0. — The companion textbook covering the ETL pipeline that connects OLTP to DW in detail.

Database Design for Analysts *| © Patrick Dolinger, NSCC Institute of Technology CC BY 4.0 — Share and adapt freely with attribution*

Chapter 9: Putting It Together — Designing CabotTrailExecutive

Database Design for Analysts *From business requirements to relational structures*

© Patrick Dolinger, NSCC Institute of Technology Licensed under Creative Commons Attribution 4.0 International (CC BY 4.0) You are free to share and adapt this material for any purpose, provided appropriate credit is given.

Chapter Overview

This final chapter executes the complete database design process from start to finish. Every concept from the preceding eight chapters converges here: gathering requirements, identifying entities and relationships, drawing an ERD, normalising the model, mapping to DDL, applying constraints, designing indexes, and considering both OLTP and OLAP perspectives — all applied to one real design problem.

The problem is `CabotTrailExecutive` — an executive summary database that CabotTrail Outdoor's leadership team needs for monthly performance reporting. You have seen this database referenced throughout the course and the prerequisite SQL course. Now you will design it from requirements.

This chapter models the professional workflow: a business narrative leads to questions; questions refine requirements; requirements become a logical model; the logical model becomes a physical schema; the physical schema is populated and verified. The output is a complete, production-ready database design — and the skills to repeat the process for any future design challenge.

By the end of this chapter you will be able to:

- Extract entities, attributes, and relationships from a business narrative
- Surface and resolve design ambiguities through business questions
- Draw a complete ERD for a new database
- Apply the full normalization process to verify the design
- Write complete, production-quality DDL with all constraints
- Design an appropriate index strategy for the schema
- Populate the database from existing CabotTrail data
- Verify the design with reconciliation queries

- Describe how CabotTrailExecutive fits into the broader ETL architecture

---

Table of Contents

1. The Design Brief
2. Requirements Analysis
3. Identifying Entities and Attributes
4. Drawing the ERD
5. Normalization Verification
6. Mapping to DDL
7. Constraint Design
8. Index Strategy
9. The Complete DDL Script
10. Populating from Existing Data
11. Verification and Reconciliation
12. Positioning in the ETL Architecture
13. Looking Back, Looking Forward
14. Chapter Summary
15. Review Questions
16. 🔍 Deeper Dive

---

1. The Design Brief

1.1 The Business Narrative

> *CabotTrail Outdoor's executive team meets monthly to review company performance. Currently, the CFO prepares a report manually by running queries against multiple data marts and assembling the results in Excel. This takes three days and the report is often out of date by the time it circulates.*
>
> *The executive team needs a dedicated database — `CabotTrailExecutive` — that contains pre-aggregated monthly summary data they can query directly. The database should support the following key questions:*
>
> - *"What was total revenue by territory and product category for each month?"*
> - *"How does this month compare to the same month last year?"*
> - *"Which territories are growing and which are declining?"*
> - *"What is our gross profit margin by category?"*
> - *"How many invoices and customers did we serve in each territory?"*
>
> *The database should be refreshable from existing data — updated monthly from the data marts that already exist. It does not need to track individual transactions; it should contain only monthly summaries at the territory × category × month grain.*

1.2 Reading the Brief

Before designing anything, read the brief carefully and identify what is explicit and what is ambiguous.

Explicit:

- Monthly grain (one row per month per territory per category)
- Key measures: revenue, gross profit margin, invoice count, customer count
- Territory and product category as dimensions
- Time comparison needed (year-over-year)

Ambiguous:

- Does "territory" mean `SalesTerritory` (derived) or `ProvinceCode` (source)? If CabotTrail opens in a new territory, does it appear immediately or need to be pre-defined?
- Does "category" mean the 13 existing product categories? Are all 13 included, or only major ones?
- Does "customer count" mean distinct customers who placed orders, or total customer records?

- Should the database be refreshable incrementally (append new months) or fully replaced each month?
- Does "same month last year" mean the calculation should be pre-computed, or computed at query time?

These questions must be answered before the design can be finalised. The next section shows how.

---

2. Requirements Analysis

A good database designer does not proceed on ambiguous requirements. This section demonstrates the question-and-answer process that resolves ambiguities.

2.1 Clarifying Questions and Decisions

Q: What is the territory definition? *A:* Use `SalesTerritory` as derived in `dim.Customer` — the 10-value derived field ('Atlantic — Nova Scotia', 'Ontario', etc.). New territories will be added to the derivation logic and the database will pick them up on the next refresh.

Q: What categories are included? *A:* All 13 product categories currently in `dim.Product`. The database should accommodate new categories without structural change.

Q: What does "customer count" mean? *A:* Distinct customers who placed at least one order in that month, in that territory. Not total customer records.

Q: Incremental or full refresh? *A:* Full replacement monthly — truncate and reload from the current state of the data marts. The CFO wants the cleanest possible data; partial updates would complicate verification.

Q: Should year-over-year be pre-computed? *A:* No — the query can compute it at runtime by joining the same summary table to itself with a one-year offset. Pre-computing it would either require storing an extra column (and recomputing it on every refresh) or creating a separate view. A view is the better approach.

Q: Is a calendar/period dimension needed? *A:* Yes — a separate `Period` table with CalendarYear and MonthNumber as the PK, plus MonthName and other calendar attributes, will make queries cleaner than storing year and month as plain integers in the summary table.

Q: Should the database include all time or only recent data? *A:* All available history — January 2022 through the most recent complete month. The database must be able to support year-over-year comparisons going back to the first full year of data.

2.2 The Refined Requirements

After clarification, the requirements are:

1. **Grain:** One row per (SalesTerritory, CategoryName, CalendarYear, MonthNumber)
2. **Measures per row:** TotalRevenue, TotalGrossProfit, InvoiceCount, CustomerCount, SalesLineCount
3. **Dimensions:** Period (CalendarYear + MonthNumber), SalesTerritory (string, from DW), ProductCategory (CategoryName)
4. **History:** All available data (January 2022 through the last complete month)
5. **Refresh:** Monthly full replacement (clear the tables, then INSERT)
6. **YoY comparison:** Delivered as a view, not stored columns
7. **No individual transaction data** — summaries only

---

3. Identifying Entities and Attributes

3.1 Entities from the Requirements

Reading the refined requirements for nouns that have multiple instances:

- **Period** — each unique (CalendarYear, MonthNumber) combination; has MonthName and other calendar attributes
- **Territory** — each unique SalesTerritory value; could have descriptive attributes
- **ProductCategory** — each unique CategoryName; could have margin tier or description attributes
- **MonthlySummary** — the central fact: one row per (Period, Territory, Category) with measures

3.2 Is Territory a Full Entity?

Should `SalesTerritory` be a separate table with its own attributes, or just a column in the summary?

Arguments for a separate Territory table:

- Allows adding territory descriptors (regional manager, geographic description) without changing the summary table
- Makes territory name changes manageable (update once, reflects everywhere)
- More normalized

Arguments against:

- The territory is a derived string value, not a business entity with independent existence
- A Territory table would have very few rows (10) and no attributes beyond the name
- Adding a JOIN to a 10-row lookup table for a summary database adds complexity without much value

Decision: Keep `SalesTerritory` as a column in the summary table. Its cardinality (10 values) is small enough that a separate table adds more complexity than value. The period and category are the true independent entities.

3.3 The Entity Set

Three entities in the final design:

Period — calendar period for grouping:

- PeriodKey (PK, surrogate)
- CalendarYear (SMALLINT)
- MonthNumber (TINYINT)
- MonthName (NVARCHAR)
- YearMonthLabel (derived: e.g., 'Mar 2024')
- FiscalYearNumber (SMALLINT)
- FiscalQuarterNumber (TINYINT)
- QuarterLabel (e.g., 'Q1 2024/25')

ProductCategory — product category reference:

- CategoryKey (PK, surrogate)
- CategoryName (NVARCHAR, unique)
- MarginTier (derived: 'High', 'Medium', 'Low' based on typical margins)
- IsActive (BIT — future-proofing for discontinued categories)

ExecutiveSummary — the central fact/summary:

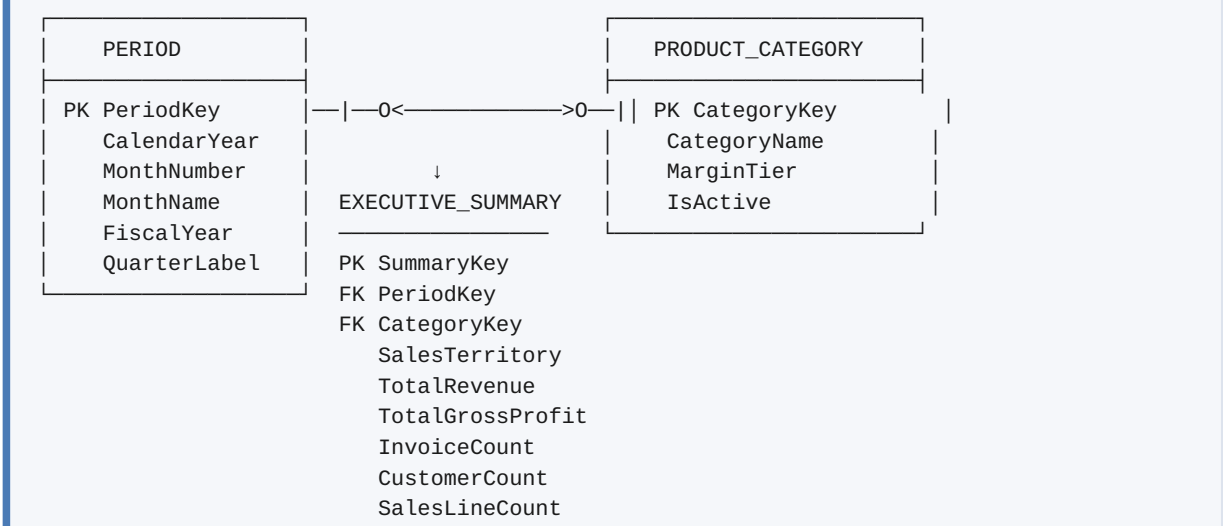
- SummaryKey (PK, surrogate)
- PeriodKey (FK → Period)
- CategoryKey (FK → ProductCategory)
- SalesTerritory (NVARCHAR — stored inline, not a FK)
- TotalRevenue (DECIMAL)

- TotalGrossProfit (DECIMAL)
- InvoiceCount (INT)
- CustomerCount (INT)
- SalesLineCount (INT)

---

4. Drawing the ERD

4.1 The ERD in Text Notation



Relationships:

- `Period` to `ExecutiveSummary`: 1:N — one period has many summary rows (one per territory × category combination)
- `ProductCategory` to `ExecutiveSummary`: 1:N — one category has many summary rows (one per period × territory)
- `SalesTerritory` is embedded in `ExecutiveSummary` as a column, not a FK relationship

Cardinality note: For any given (PeriodKey, CategoryKey, SalesTerritory) combination, there should be exactly one `ExecutiveSummary` row — enforced by a UNIQUE constraint on those three columns.

4.2 Design Decisions Recorded

Decision	Choice	Rationale
Territory as separate entity	No — inline string	Too small and too simple to justify a lookup table
Period as separate entity	Yes — lookup table	Enables clean calendar attributes without redundancy in summary table
Category as separate entity	Yes — lookup table	Allows category descriptors; `MarginTier` has independent meaning
SCD handling	None — full monthly replacement	Simplifies ETL; no history within the executive database needed
YoY as stored or computed	View — computed at query time	Avoids redundant storage; view is the right tool for derived comparative data

---

5. Normalization Verification

5.1 Checking ExecutiveSummary

Let PK = `SummaryKey` (single-column surrogate).

Check for transitive dependencies in non-key attributes:

- `PeriodKey` — describes the summary row (which period). ✓ Direct FK
- `CategoryKey` — describes the summary row (which category). ✓ Direct FK
- `SalesTerritory` — describes the summary row (which territory). ✓ Direct
- `TotalRevenue` — a fact about the (period, category, territory) combination. ✓ Direct
- `TotalGrossProfit` — same. ✓ Direct
- `InvoiceCount`, `CustomerCount`, `SalesLineCount` — same. ✓ Direct

No transitive dependencies in ExecutiveSummary. ✓ 3NF.

5.2 Checking Period

PK = `PeriodKey`.

- `CalendarYear`, `MonthNumber` — describe the period directly. ✓
- `MonthName` — is it a fact about the period? Yes — MonthName is determined by MonthNumber, but MonthNumber is also a direct attribute of the period. This is a transitive-ish dependency: `MonthNumber`

`→ MonthName`. But since `MonthNumber` is part of the logical period identity, not a separate entity, and MonthName adds no redundancy (it is a canonical attribute of months), this is acceptable. In strict 3NF, `MonthName` would belong in a separate `Month` lookup. For a small reference table, the pragmatic choice is to keep it in `Period`.

- `FiscalYearNumber`, `FiscalQuarterNumber` — facts about the period's position in the fiscal calendar. ✓ Direct
- `QuarterLabel` — derived from `FiscalYearNumber` and `FiscalQuarterNumber`. This is a derived attribute. Design choice: compute it as a computed column or store it for query convenience.

Period is in 3NF with a pragmatic exception for MonthName. Acceptable for a small reference table (one row per month of history).

5.3 Checking ProductCategory

PK = `CategoryKey`.

- `CategoryName` — describes the category. ✓ Direct
- `MarginTier` — is it a fact about the category? Yes, if it is a derived classification applied per category. It could also be a derived value from average margins in the fact data — in which case it is not a property of the category entity itself, but a computed aggregate.

Design clarification: `MarginTier` will be loaded from the ETL as a static classification derived from `dim.Product.CategoryName` in the DW — it is treated as a property of the category in this design, not a computed aggregate. ✓ 3NF.

---

6. Mapping to DDL

6.1 Applying the Mapping Rules

Rule 1 (Strong entities → tables): Three tables: `dim.Period`, `dim.ProductCategory`, `fact.ExecutiveSummary`.

Rule 2 (Attributes → columns): Each attribute maps to a column with an appropriate data type.

Rule 5 (1:N relationships → FK on many side): `ExecutiveSummary` carries `PeriodKey` and `CategoryKey` as FKs.

No composite PK: `SummaryKey` is a surrogate PK for `ExecutiveSummary`. The business uniqueness rule (one row per period × category × territory) is enforced by a UNIQUE constraint.

6.2 Data Type Decisions

Column	Type	Rationale
PeriodKey, CategoryKey, SummaryKey	`INT NOT NULL IDENTITY(1,1)`	Surrogate PKs
CalendarYear, FiscalYearNumber	`SMALLINT NOT NULL`	2-byte; year range 2000–2100 easily fits
MonthNumber, FiscalQuarterNumber	`TINYINT NOT NULL`	1-byte; 1–12 or 1–4
MonthName	`NVARCHAR(10) NOT NULL`	Longest value, 'September', is 9 characters
YearMonthLabel	`NVARCHAR(10) NOT NULL`	Format 'Mar 2024' — 8 characters
QuarterLabel	`NVARCHAR(12) NOT NULL`	Format 'Q1 2024/25' — 10 characters
CategoryName	`NVARCHAR(60) NOT NULL`	Matches the length of category names in the DW
SalesTerritory	`NVARCHAR(60) NOT NULL`	Matches dim.Customer.SalesTerritory
TotalRevenue, TotalGrossProfit	`DECIMAL(18,2) NOT NULL DEFAULT 0`	Financial amounts; 2 decimal places
InvoiceCount, CustomerCount, SalesLineCount	`INT NOT NULL DEFAULT 0`	Counts; non-negative
MarginTier	`NVARCHAR(10) NOT NULL`	'High', 'Medium', 'Low'
IsActive	`BIT NOT NULL DEFAULT 1`	Boolean flag

---

7. Constraint Design

For each table, identify all constraints needed to enforce the business rules:

`dim.Period` constraints:

- PK on PeriodKey
- UNIQUE on (CalendarYear, MonthNumber) — each calendar month appears once
- CHECK: MonthNumber BETWEEN 1 AND 12
- CHECK: CalendarYear BETWEEN 2000 AND 2100
- CHECK: FiscalQuarterNumber BETWEEN 1 AND 4

`dim.ProductCategory` constraints:

- PK on CategoryKey
- UNIQUE on CategoryName — no duplicate category names
- CHECK: MarginTier IN ('High', 'Medium', 'Low')
- No CHECK on IsActive — the `BIT` data type already restricts it to 0 and 1 (or NULL, which `NOT NULL` prevents), so a CHECK would add nothing

`fact.ExecutiveSummary` constraints:

- PK on SummaryKey
- UNIQUE on (PeriodKey, CategoryKey, SalesTerritory) — one row per combination
- FK: PeriodKey → dim.Period.PeriodKey
- FK: CategoryKey → dim.ProductCategory.CategoryKey
- CHECK: TotalRevenue >= 0
- No CHECK on TotalGrossProfit — gross profit can legitimately be negative for a territory and category in a month (returns exceeding sales, or sales below cost), so a `>= 0` rule would reject valid data
- CHECK: InvoiceCount >= 0, CustomerCount >= 0, SalesLineCount >= 0

---

8. Index Strategy

`CabotTrailExecutive` is a read-heavy analytical database. The ETL loads it monthly (full reload); analysts query it constantly. The index strategy favours reads.

`dim.Period`:

- Clustered on PeriodKey (PK default)
- UNIQUE non-clustered on (CalendarYear, MonthNumber) — supports queries filtering by year and month
- Non-clustered on (FiscalYearNumber, FiscalQuarterNumber) — supports fiscal period queries

`dim.ProductCategory`:

- Clustered on CategoryKey (PK default)
- UNIQUE non-clustered on CategoryName — supports joins and lookups by name

`fact.ExecutiveSummary`:

- Clustered on SummaryKey (PK default)
- UNIQUE non-clustered on (PeriodKey, CategoryKey, SalesTerritory) — enforces grain, also supports the most common query pattern
- Non-clustered on (PeriodKey) INCLUDE (SalesTerritory, TotalRevenue, TotalGrossProfit) — covers time-series queries
- Non-clustered on (SalesTerritory, PeriodKey) INCLUDE (TotalRevenue, TotalGrossProfit) — covers territory trend queries

---

9. The Complete DDL Script

```
-- ============================================================
-- CabotTrailExecutive Database
-- Executive summary database for monthly performance reporting
-- Author: Patrick Dolinger, NSCC Institute of Technology
-- Version: 1.0
-- Date: 2027-01-06
-- ============================================================

-- Create the database
IF NOT EXISTS (SELECT name FROM sys.databases WHERE name = 'CabotTrailExecutive')
BEGIN
    CREATE DATABASE CabotTrailExecutive;
END;
GO

USE CabotTrailExecutive;
GO

-- ── Create schemas ──────────────────────────────────────────
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = 'dim')
    EXEC ('CREATE SCHEMA dim');
GO
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = 'fact')
    EXEC ('CREATE SCHEMA fact');
GO

-- ── Drop existing tables (idempotent) ──────────────────────
DROP TABLE IF EXISTS fact.ExecutiveSummary;
DROP TABLE IF EXISTS dim.Period;
DROP TABLE IF EXISTS dim.ProductCategory;
GO

-- ── dim.Period ──────────────────────────────────────────────
CREATE TABLE dim.Period
(
    PeriodKey           INT             NOT NULL IDENTITY(1,1),
```

```
    CalendarYear        SMALLINT        NOT NULL,
    MonthNumber         TINYINT         NOT NULL,
    MonthName           NVARCHAR(10)    NOT NULL,
    YearMonthLabel      NVARCHAR(10)    NOT NULL,   -- e.g., 'Mar 2024'
    FiscalYearNumber    SMALLINT        NOT NULL,
    FiscalQuarterNumber TINYINT         NOT NULL,
    QuarterLabel        NVARCHAR(12)    NOT NULL,   -- e.g., 'Q1 2024/25'
    IsCurrentMonth      BIT             NOT NULL DEFAULT 0,   -- 1 = most recent
complete month loaded

    CONSTRAINT PK_Period                    PRIMARY KEY (PeriodKey),
    CONSTRAINT UQ_Period_CalendarYearMonth   UNIQUE (CalendarYear, MonthNumber),
    CONSTRAINT CK_Period_MonthNumber         CHECK (MonthNumber BETWEEN 1 AND 12),
    CONSTRAINT CK_Period_CalendarYear        CHECK (CalendarYear BETWEEN 2000 AND 2100),
    CONSTRAINT CK_Period_FiscalQuarterNumber CHECK (FiscalQuarterNumber BETWEEN 1 AND 4)
);
GO

-- ── dim.ProductCategory ──────────────────────────────
CREATE TABLE dim.ProductCategory
(
    CategoryKey         INT             NOT NULL IDENTITY(1,1),
    CategoryName        NVARCHAR(60)    NOT NULL,
    MarginTier          NVARCHAR(10)    NOT NULL DEFAULT 'Medium',
    IsActive            BIT             NOT NULL DEFAULT 1,

    CONSTRAINT PK_ProductCategory             PRIMARY KEY (CategoryKey),
    CONSTRAINT UQ_ProductCategory_CategoryName UNIQUE (CategoryName),
    CONSTRAINT CK_ProductCategory_MarginTier   CHECK (MarginTier IN ('High', 'Medium',
'Low'))
);
GO

-- ── fact.ExecutiveSummary ────────────────────────────
CREATE TABLE fact.ExecutiveSummary
(
    SummaryKey          INT             NOT NULL IDENTITY(1,1),
    PeriodKey           INT             NOT NULL,
    CategoryKey         INT             NOT NULL,
    SalesTerritory      NVARCHAR(60)    NOT NULL,
    TotalRevenue        DECIMAL(18,2)   NOT NULL DEFAULT 0,
    TotalGrossProfit    DECIMAL(18,2)   NOT NULL DEFAULT 0,
    InvoiceCount        INT             NOT NULL DEFAULT 0,
    CustomerCount       INT             NOT NULL DEFAULT 0,
    SalesLineCount      INT             NOT NULL DEFAULT 0,

    CONSTRAINT PK_ExecutiveSummary       PRIMARY KEY (SummaryKey),
    CONSTRAINT UQ_ExecutiveSummary_Grain UNIQUE (PeriodKey, CategoryKey,
SalesTerritory),
    CONSTRAINT FK_ExecutiveSummary_Period
        FOREIGN KEY (PeriodKey)    REFERENCES dim.Period (PeriodKey),
    CONSTRAINT FK_ExecutiveSummary_ProductCategory
        FOREIGN KEY (CategoryKey)  REFERENCES dim.ProductCategory (CategoryKey),
    CONSTRAINT CK_ExecutiveSummary_TotalRevenue
        CHECK (TotalRevenue >= 0),
    CONSTRAINT CK_ExecutiveSummary_InvoiceCount
        CHECK (InvoiceCount >= 0),
```

```
    CONSTRAINT CK_ExecutiveSummary_CustomerCount
        CHECK (CustomerCount >= 0),
    CONSTRAINT CK_ExecutiveSummary_SalesLineCount
        CHECK (SalesLineCount >= 0)
);
GO

-- ── Indexes ──────────────────────────────────────────────
CREATE NONCLUSTERED INDEX IX_Period_FiscalYear
    ON dim.Period (FiscalYearNumber, FiscalQuarterNumber);

CREATE NONCLUSTERED INDEX IX_ExecutiveSummary_Period
    ON fact.ExecutiveSummary (PeriodKey)
    INCLUDE (SalesTerritory, TotalRevenue, TotalGrossProfit, InvoiceCount,
CustomerCount);

CREATE NONCLUSTERED INDEX IX_ExecutiveSummary_Territory
    ON fact.ExecutiveSummary (SalesTerritory, PeriodKey)
    INCLUDE (TotalRevenue, TotalGrossProfit, InvoiceCount);
GO

-- ── YoY View ─────────────────────────────────────────────
CREATE OR ALTER VIEW fact.vw_ExecutiveSummaryYoY AS
SELECT
    es.SummaryKey,
    p.CalendarYear,
    p.MonthNumber,
    p.MonthName,
    p.YearMonthLabel,
    p.FiscalYearNumber,
    p.QuarterLabel,
    cat.CategoryName,
    cat.MarginTier,
    es.SalesTerritory,
    es.TotalRevenue,
    es.TotalGrossProfit,
    es.InvoiceCount,
    es.CustomerCount,
    es.SalesLineCount,
    -- Gross profit margin percentage (computed, non-additive)
    ROUND(
        es.TotalGrossProfit / NULLIF(es.TotalRevenue, 0) * 100,
    2)                                      AS GrossMarginPct,
    -- Same period last year: join back to the same table 12 months ago
    prior.TotalRevenue                      AS PriorYearRevenue,
    prior.TotalGrossProfit                  AS PriorYearGrossProfit,
    -- Year-over-year revenue change
    ROUND(
        (es.TotalRevenue - ISNULL(prior.TotalRevenue, 0))
        / NULLIF(prior.TotalRevenue, 0) * 100,
    1)                                      AS YoYRevenueChangePct,
    -- Revenue growth flag
    CASE
        WHEN prior.TotalRevenue IS NULL     THEN 'New'
        WHEN es.TotalRevenue > prior.TotalRevenue THEN 'Growth'
        WHEN es.TotalRevenue < prior.TotalRevenue THEN 'Decline'
        ELSE                                     'Flat'
```

```
    END                                         AS YoYTrend
FROM    fact.ExecutiveSummary es
INNER JOIN dim.Period p      ON p.PeriodKey   = es.PeriodKey
INNER JOIN dim.ProductCategory cat ON cat.CategoryKey = es.CategoryKey
-- Self-join: find the same territory+category combination 12 months prior
LEFT JOIN (
    SELECT
        es2.CategoryKey,
        es2.SalesTerritory,
        p2.CalendarYear,
        p2.MonthNumber,
        es2.TotalRevenue,
        es2.TotalGrossProfit
    FROM   fact.ExecutiveSummary es2
    INNER JOIN dim.Period p2 ON p2.PeriodKey = es2.PeriodKey
) prior
    ON  prior.CategoryKey   = es.CategoryKey
    AND prior.SalesTerritory = es.SalesTerritory
    AND prior.CalendarYear  = p.CalendarYear - 1
    AND prior.MonthNumber   = p.MonthNumber;
GO
```

---

10. Populating from Existing Data

With the schema in place, populate `CabotTrailExecutive` from the existing data in `CabotTrailOutdoorDW`.

10.1 Populate dim.Period

> ***Check your warehouse's column names.*** *Chapter 8 lists the year column of `Dimension.DimDate` as `CalendarYear`, while the Chapter 9 scripts use `YearNumber`. Run this query against `CabotTrailOutdoorDW` and use the names it returns:*
>
> ```
> SELECT COLUMN_NAME, DATA_TYPE
> FROM   CabotTrailOutdoorDW.INFORMATION_SCHEMA.COLUMNS
> WHERE  TABLE_SCHEMA = 'Dimension' AND TABLE_NAME = 'DimDate'
> ORDER BY ORDINAL_POSITION;
> ```

```
USE CabotTrailExecutive;

-- Load window: January 2022 through the last complete month.
-- Each script declares the window itself. If you run several scripts together, keep a
GO between them.
DECLARE @FirstDateKey INT = 20220101;
DECLARE @LastDateKey  INT = CONVERT(INT, CONVERT(CHAR(8), EOMONTH(GETDATE(), -1),
```

```
112));  -- e.g. 20261031

-- Load one row per year/month from the DW date dimension
INSERT INTO dim.Period
(
    CalendarYear, MonthNumber, MonthName, YearMonthLabel,
    FiscalYearNumber, FiscalQuarterNumber, QuarterLabel, IsCurrentMonth
)
SELECT DISTINCT
    d.YearNumber            AS CalendarYear,
    d.MonthNumber,
    d.MonthName,
    -- Build the label: 'Mar 2024'
    LEFT(d.MonthName, 3) + ' ' + CAST(d.YearNumber AS NVARCHAR(4)) AS YearMonthLabel,
    d.FiscalYearNumber,
    d.FiscalQuarterNumber,
    -- Build the quarter label: 'Q1 2024/25'
    'Q' + CAST(d.FiscalQuarterNumber AS NVARCHAR(1))
        + ' ' + CAST(d.FiscalYearNumber AS NVARCHAR(4))
        + '/' + RIGHT(CAST(d.FiscalYearNumber + 1 AS NVARCHAR(4)), 2) AS QuarterLabel,
    -- Flag the most recent complete month in the load window
    CASE WHEN d.DateKey / 100 = @LastDateKey / 100
         THEN 1 ELSE 0 END AS IsCurrentMonth
FROM    CabotTrailOutdoorDW.Dimension.DimDate d
WHERE   d.DateKey BETWEEN @FirstDateKey AND @LastDateKey
GROUP BY d.YearNumber, d.MonthNumber, d.MonthName,
         d.FiscalYearNumber, d.FiscalQuarterNumber, d.DateKey / 100
ORDER BY d.YearNumber, d.MonthNumber;
```

10.2 Populate dim.ProductCategory

```
-- Load product categories from the DW
INSERT INTO dim.ProductCategory (CategoryName, MarginTier, IsActive)
SELECT
    pc.ProductCategoryName  AS CategoryName,
    -- MarginTier derived from known category margin rates
    CASE
        WHEN pc.ProductCategoryName IN
            ('Accessories','Hydration','Navigation','Safety and First Aid')
        THEN 'High'
        WHEN pc.ProductCategoryName IN
            ('Climbing and Rappelling','Cooking and Food',
             'Apparel - Outerwear','Apparel - Tops','Backpacks and Bags')
        THEN 'Medium'
        ELSE 'Low'
    END                     AS MarginTier,
    1                       AS IsActive
FROM    CabotTrailOutdoorDW.Dimension.DimProductCategory pc
WHERE   pc.ProductCategoryID <> 0   -- Exclude unknown member
ORDER BY pc.ProductCategoryName;
```

10.3 Populate fact.ExecutiveSummary

```
-- The main summary load: aggregate from DW FactSales
-- This is the source-to-target script for the executive summary
-- Load window: January 2022 through the last complete month.
-- Each script declares the window itself. If you run several scripts together, keep a
GO between them.
DECLARE @FirstDateKey INT = 20220101;
DECLARE @LastDateKey  INT = CONVERT(INT, CONVERT(CHAR(8), EOMONTH(GETDATE(), -1),
112));  -- e.g. 20261031

INSERT INTO fact.ExecutiveSummary
(
    PeriodKey, CategoryKey, SalesTerritory,
    TotalRevenue, TotalGrossProfit,
    InvoiceCount, CustomerCount, SalesLineCount
)
SELECT
    p.PeriodKey,
    cat.CategoryKey,
    cust.SalesTerritory,
    ROUND(SUM(fs.LineTotal),        2) AS TotalRevenue,
    ROUND(SUM(fs.GrossProfit),      2) AS TotalGrossProfit,
    COUNT(DISTINCT fs.InvoiceID)       AS InvoiceCount,
    COUNT(DISTINCT fs.CustomerKey)     AS CustomerCount,
    COUNT(*)                           AS SalesLineCount
FROM    CabotTrailOutdoorDW.Fact.FactSales fs
-- Join to DW dimensions
INNER JOIN CabotTrailOutdoorDW.Dimension.DimDate d
    ON d.DateKey = fs.OrderDateKey
INNER JOIN CabotTrailOutdoorDW.Dimension.DimCustomer cust
    ON cust.CustomerKey = fs.CustomerKey
    AND cust.IsCurrent  = 1      -- Current version of customer (SCD Type 1 — all
current)
INNER JOIN CabotTrailOutdoorDW.Dimension.DimProduct dp
    ON dp.ProductKey = fs.ProductKey
    AND dp.ProductID <> 0
INNER JOIN CabotTrailOutdoorDW.Dimension.DimProductCategory dpc
    ON dpc.ProductCategoryKey = dp.ProductCategoryKey
    AND dpc.ProductCategoryID <> 0
-- Join to Executive dimension tables to get surrogate keys
INNER JOIN dim.Period p
    ON p.CalendarYear = d.YearNumber
    AND p.MonthNumber = d.MonthNumber
INNER JOIN dim.ProductCategory cat
    ON cat.CategoryName = dpc.ProductCategoryName
-- Exclude unknown members and the current, incomplete month
WHERE   fs.CustomerKey <> 0
AND     fs.ProductKey  <> 0
AND     fs.OrderDateKey BETWEEN @FirstDateKey AND @LastDateKey
GROUP BY
    p.PeriodKey,
    cat.CategoryKey,
    cust.SalesTerritory;
```

11. Verification and Reconciliation

After populating the database, verify that the data is correct and complete.

11.1 Row Count Verification

```
USE CabotTrailExecutive;

-- How many periods, categories, territories, and summary rows?
SELECT 'dim.Period'          AS TableName, COUNT(*) AS Rows FROM dim.Period
UNION ALL
SELECT 'dim.ProductCategory', COUNT(*) FROM dim.ProductCategory
UNION ALL
SELECT 'fact.ExecutiveSummary', COUNT(*) FROM fact.ExecutiveSummary;

-- Expected:
-- dim.Period: one row per month from January 2022 through the last complete month
--   (for example, 58 rows when loaded on 1 November 2026)
-- dim.ProductCategory: 13 rows
-- fact.ExecutiveSummary: up to (dim.Period rows) × 13 categories × 10 territories
--   (fewer if some territory/category combinations have no sales in some months)
```

11.2 Revenue Reconciliation

```
-- Total revenue in ExecutiveSummary must match total revenue in the DW FactSales
-- Load window: January 2022 through the last complete month.
-- Each script declares the window itself. If you run several scripts together, keep a
GO between them.
DECLARE @FirstDateKey INT = 20220101;
DECLARE @LastDateKey  INT = CONVERT(INT, CONVERT(CHAR(8), EOMONTH(GETDATE(), -1),
112));  -- e.g. 20261031

SELECT
    'DW FactSales'              AS Source,
    ROUND(SUM(LineTotal), 2)    AS TotalRevenue
FROM    CabotTrailOutdoorDW.Fact.FactSales
WHERE   CustomerKey <> 0
AND     ProductKey  <> 0
AND     OrderDateKey BETWEEN @FirstDateKey AND @LastDateKey   -- Same window as the load

UNION ALL

SELECT
    'CabotTrailExecutive',
    ROUND(SUM(TotalRevenue), 2)
FROM    fact.ExecutiveSummary;

-- Both values must be equal (within $0.01 tolerance for rounding)
```

A reconciliation is only valid if the source query applies **exactly the same filters and joins** as the load. The load in §10.3 also excludes rows through its joins — for example, products whose category is the unknown member. If the totals differ, first check whether the difference is made up of rows the load excluded on purpose. Review Question 4 asks you to diagnose exactly this kind of mismatch.

11.3 Testing the YoY View

```
-- Test the year-over-year view
SELECT TOP 20
    YearMonthLabel,
    CategoryName,
    SalesTerritory,
    TotalRevenue,
    PriorYearRevenue,
    YoYRevenueChangePct,
    YoYTrend
FROM    fact.vw_ExecutiveSummaryYoY
WHERE   CalendarYear = 2024
ORDER BY CalendarYear, MonthNumber, CategoryName, SalesTerritory;
```

11.4 The Executive Report Query

The executive report query is now a simple SELECT against the view — exactly what the CFO needs:

```
-- Monthly territory performance: YTD 2024 vs prior year
SELECT
    QuarterLabel,
    SalesTerritory,
    SUM(TotalRevenue)               AS Revenue,
    SUM(PriorYearRevenue)           AS PriorYearRevenue,
    ROUND(
        (SUM(TotalRevenue) - SUM(ISNULL(PriorYearRevenue, 0)))
        / NULLIF(SUM(PriorYearRevenue), 0) * 100,
    1)                              AS YoYChangePct,
    ROUND(
        SUM(TotalGrossProfit) / NULLIF(SUM(TotalRevenue), 0) * 100,
    2)                              AS MarginPct
FROM    fact.vw_ExecutiveSummaryYoY
WHERE   CalendarYear = 2024
GROUP BY QuarterLabel, FiscalYearNumber, FiscalQuarterNumber, SalesTerritory
ORDER BY FiscalYearNumber, FiscalQuarterNumber, SalesTerritory;
```

This is the query the CFO was previously assembling manually over three days. With `CabotTrailExecutive`, it returns in under a second.

---

12. Positioning in the ETL Architecture

12.1 Where CabotTrailExecutive Fits

```
CabotTrailOutdoor (OLTP)
        ↓ ETL Layer 1 (nightly, 2:00 AM)
CabotTrailOutdoorDW (Data Warehouse)
        ↓ ETL Layer 2 (nightly, 2:05 AM)
Data Marts (Sales, Returns, Purchasing, etc.)
        ↓ ETL Layer 3 (monthly, 1st of month, 3:00 AM)
CabotTrailExecutive
```

`CabotTrailExecutive` is loaded from the DW after the nightly DW load completes. The monthly load runs on the first of each month:

1. **Clear** all three tables: `TRUNCATE` the fact table, then `DELETE` from both dimensions and reset their IDENTITY values (SQL Server cannot truncate a table that a foreign key references)
2. **Reload** `dim.Period` from the DW calendar
3. **Reload** `dim.ProductCategory` from the DW product dimension
4. **Reload** `fact.ExecutiveSummary` from the DW fact table
5. **Verify** revenue reconciliation
6. **Alert** on failure

The order matters. `fact.ExecutiveSummary` has foreign keys to both dimensions, so SQL Server refuses `TRUNCATE TABLE dim.Period` (error 4712) even when the fact table is empty. Clear the fact table first, then the dimensions:

```
TRUNCATE TABLE fact.ExecutiveSummary;    -- allowed: no table references the fact table

DELETE FROM dim.Period;
DELETE FROM dim.ProductCategory;

DBCC CHECKIDENT ('dim.Period', RESEED, 0);           -- next PeriodKey will be 1
DBCC CHECKIDENT ('dim.ProductCategory', RESEED, 0);  -- next CategoryKey will be 1
```

12.2 The SSIS Package Design

In DBAS 2103, you will build the SSIS package for this load. Its structure follows the master/child package pattern from the ETL textbook:

```
Master_Executive_Load.dtsx
    ↓
[Execute SQL: TRUNCATE fact.ExecutiveSummary]
```

```
[Execute SQL: DELETE dim.Period; DBCC CHECKIDENT reseed]
[Execute SQL: DELETE dim.ProductCategory; DBCC CHECKIDENT reseed]
    ↓
[Data Flow: Load dim.Period]
    ↓
[Data Flow: Load dim.ProductCategory]
    ↓
[Data Flow: Load fact.ExecutiveSummary]
    ↓
[Execute SQL: Reconciliation test — Revenue match]
    ↓
[On failure: Send alert email]
```

The T-SQL source-to-target scripts in section 10 of this chapter are the direct specification for what each Data Flow Task implements.

---

13. Looking Back, Looking Forward

13.1 What This Course Covered

`CabotTrailExecutive` was designed using every concept in this course:

Chapter	Skill applied in this chapter
1 — The Relational Model	Understanding entities, attributes, relationships; the designer's perspective
2 — ERD	Drawing the three-entity ERD; recording design decisions
3 — Mapping to DDL	Seven mapping rules applied; FK placement; surrogate PKs
4 — 1NF and 2NF	Verifying no multi-valued attributes, no partial dependencies
5 — 3NF	Verifying no transitive dependencies; pragmatic MonthName decision
6 — Constraints	Named constraints for every business rule
7 — Indexes	Read-optimised index strategy; covering indexes for common patterns
8 — OLTP vs OLAP	Positioned as OLAP; denormalised SalesTerritory; summary grain

13.2 The Three Courses Together

This course is the middle of a three-course sequence:

DBAS 5010 (SQL for Analysts) taught you to *read* databases — SELECT, JOIN, GROUP BY, window functions. You worked with the CabotTrail databases as existing structures.

DBAS 2010 (Database Design for Analysts) taught you to *design* databases — ERDs, normalization, constraints, indexes, OLTP vs OLAP. You learned why the CabotTrail databases are the way they are, and built `CabotTrailExecutive`.

DBAS 2103 (Data Provisioning with ETL) will teach you to *connect* databases — the SSIS pipelines that move data from OLTP through DW to data marts to `CabotTrailExecutive`. The designs you built in this course are the targets you will load in the next.

The three courses together give you a complete picture of the data lifecycle: design the structures, query the data, build the pipelines that keep the structures current.

---

14. Chapter Summary

- The complete database design process proceeds: **narrative → clarifying questions → refined requirements → entity identification → ERD → normalization verification → DDL mapping → constraints → indexes → population → verification**.
- **Requirements analysis** is not optional — ambiguities in the brief (territory definition, customer count meaning, refresh strategy, YoY pre-computation) must be resolved before design begins.
- `CabotTrailExecutive` has three tables: `dim.Period`, `dim.ProductCategory`, and `fact.ExecutiveSummary`, connected by two 1:N relationships. SalesTerritory is stored inline rather than as a separate entity.
- The **ERD** documents three entities with two 1:N relationships. Design decisions (territory inline, period as entity, YoY as view) are recorded and justified.
- **Normalization verification** confirms 3NF with one pragmatic exception (MonthName in dim.Period), explicitly documented.
- The **complete DDL** includes all three tables, all constraints (named), indexes, and the YoY view — 100+ lines of production-quality SQL.
- **Population** uses cross-database INSERT INTO ... SELECT from `CabotTrailOutdoorDW`, with three sequential loads in dependency order (Period first, Category second, Summary last).
- **Reconciliation** verifies row counts and total revenue match between source (DW) and target (Executive).

- `CabotTrailExecutive` feeds into the ETL architecture as Layer 3 — loaded from the DW monthly, positioned as the terminal layer before BI tools.

---

15. Review Questions

1. The design brief asked for "customer count" — the number of customers served in each territory/category/month. The design uses `COUNT(DISTINCT fs.CustomerKey)` in the population query. Describe a scenario where this count could be misleading, and propose an alternative definition that would be clearer for executive reporting.

2. `SalesTerritory` was stored as an inline column in `fact.ExecutiveSummary` rather than as a separate entity with its own table. Write the alternative DDL for a `dim.Territory` table and the modified `fact.ExecutiveSummary` with a FK to it. Then justify the original decision (inline) vs the alternative (separate table) — under what future requirement would the separate table become the better choice?

3. The `vw_ExecutiveSummaryYoY` view computes year-over-year revenue by joining the summary table to itself. Explain the JOIN logic: what columns are joined, why a LEFT JOIN is used rather than INNER JOIN, and what a NULL `PriorYearRevenue` means in the context of executive reporting.

4. Run the revenue reconciliation query from section 11.2 against your loaded `CabotTrailExecutive` database. If the totals do not match exactly (within $0.01), identify three possible causes and show the diagnostic query you would run for each.

5. The `dim.Period` table stores `MonthName` even though it is transitively dependent on `MonthNumber`. Explain why this is an acceptable pragmatic exception to strict 3NF for this specific case, and describe the conditions under which this pragmatism would become a maintenance problem.

6. Design the monthly SSIS package structure (in text notation, following the process flow diagram format from Chapter 6 of the ETL book) for `Master_Executive_Load.dtsx`. Include all Control Flow tasks, the error paths, and the reconciliation steps. Which package from DBAS 2103 is this most similar to?

7. A month after deploying `CabotTrailExecutive`, a new territory is added to CabotTrail's operations: 'Northern Canada' (covering Yukon, Northwest Territories, and Nunavut). Describe every place in the architecture — from the OLTP through the DW to the Executive database — that would need to change to support the new territory, and in what order those changes must be made.

8. The executive team asks: "Could we have built this in Power BI directly from the data marts, without a separate database?" Evaluate this suggestion technically: what would Power BI need to do to replicate the YoY comparison, what are the limitations, and under what scale or complexity would a dedicated database like `CabotTrailExecutive` become clearly preferable?

🔍 Deeper Dive

The Design Process in Professional Practice

Agile Database Development

Traditional database design assumes requirements are fully known before design begins. In practice, requirements emerge and change throughout a project. **Agile database development** adapts the design process to accommodate iterative requirements:

- Start with the entities and relationships that are clearly understood; design those first
- Use evolutionary design — add tables and columns incrementally as requirements are clarified
- Version control the schema (as SQL migration scripts) so changes are tracked
- Use non-destructive changes where possible (add columns as nullable before constraining)

Tools like **Liquibase** and **Flyway** manage database schema migrations in version-controlled environments — tracking which migration scripts have been applied to each environment and applying new ones in sequence.

The tension between "design everything upfront" and "evolve as you go" is resolved by: design what you know clearly, leave designed flexibility where you anticipate change, and use tools that make changes safe and trackable.

Dimensional Modelling for the Executive Layer

`CabotTrailExecutive` is a small dimensional model — one fact table and two dimension tables. Kimball's guidance for executive summary databases emphasises:

- **Keep the grain coarse.** Monthly territory × category grain is appropriate for executive reporting; finer grain is for operational or tactical analysis.
- **Pre-compute everything the executives will always need.** Gross margin percentage belongs in the view (or as a stored derived measure) — executives should not have to write `SUM(GP)/SUM(Rev)*100` themselves.

- **Make the schema self-evident.** An executive database should be queryable by a senior analyst who has not read any documentation. Column names, table names, and the YoY view should be self-explaining.

Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit* (3rd ed.). Wiley. — Chapter 5 covers enterprise data warehouse bus architecture including summary and executive databases.

Database Design Reviews

A professional database design is reviewed before implementation — by a peer developer, a senior DBA, or a business stakeholder, depending on the organisation. Common review checkpoints:

Correctness review: Is the ERD accurate? Does the DDL correctly implement the ERD? Are all constraints named? Are all FKs indexed?

Business review: Are the entity names and column names recognisable to business stakeholders? Does the grain match the business requirement? Are the measures defined correctly?

Performance review: Are the indexes appropriate for the expected query patterns? Are there any non-sargable patterns baked into the schema? Should the clustered index key be reconsidered?

Standards review: Do all names follow the established conventions? Are all constraints named? Is the DDL idempotent (safe to re-run)?

For `CabotTrailExecutive`, a business review would confirm: does the executive team agree with the territory definition, the category list, and the monthly grain? A single wrong assumption here invalidates months of data.

References and Further Reading

1. Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit* (3rd ed.). Wiley. — The entire book; Chapter 5 specifically covers enterprise DW bus architecture and summary databases.

2. Connolly, T., & Begg, C. (2015). *Database Systems: A Practical Approach to Design, Implementation, and Management* (6th ed.). Pearson. — Chapter 10 covers the end-to-end database design methodology including requirements analysis and review processes.

3. Date, C. J. (2019). *Database Design and Relational Theory: Normal Forms and All That Jazz* (2nd ed.). Apress. — Chapter 12 covers pragmatic normalisation decisions — when and how to deviate from strict forms.

4. Dolinger, P. (2027). *ETL for Business Intelligence: A practical guide to data provisioning, dimensional modelling, and pipeline design*. NSCC Institute of Technology. CC BY 4.0. — Chapter 9 covers the complete ETL pipeline including the executive summary layer.

5. Liquibase. (2024). *Liquibase Documentation*. https://docs.liquibase.com/ — Database change management and migration tracking.

6. Microsoft. (2024). *CREATE DATABASE (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/t-sql/statements/create-database-transact-sql

7. Microsoft. (2024). *CREATE VIEW (Transact-SQL)*. https://learn.microsoft.com/en-us/sql/t-sql/statements/create-view-transact-sql

---

> ***Database Design for Analysts*** | *© Patrick Dolinger, NSCC Institute of Technology CC BY 4.0 — Share and adapt freely with attribution*
>
> *Thank you for reading. If you use or adapt this material, please attribute as: Dolinger, P. (2027). Database Design for Analysts: From business requirements to relational structures. NSCC Institute of Technology. CC BY 4.0.*